

La Guía del Profesor Frisby en su Mayor Parte Adecuada para la Programación Funcional



Tabla de contenido

Introducción	1.1
Sobre este libro	1.1.1
Sobre la traducción	1.1.2
Léelo Online	1.1.3
Juega Con el Código	1.1.4
Descárgalo	1.1.5
Hazlo tú mismo	1.1.6
Contenido	1.1.7
Planes para el futuro	1.1.8
Capítulo 01: ¿Qué Estamos Haciendo?	1.2
Presentaciones	1.2.1
Un Breve Encuentro	1.2.2
Capítulo 02: Funciones de Primera Clase	1.3
Un Repaso Rápido	1.3.1
¿Por Qué Favorecer a las Funciones de Primera Clase?	1.3.2
Capítulo 03: Pura Felicidad con Funciones Puras	1.4
Oh Ser Puro Otra Vez	1.4.1
Los Efectos Secundarios Pueden Incluir...	1.4.2
Matemáticas de Octavo Grado	1.4.3
Los Argumentos Para La Pureza	1.4.4
En Resumen	1.4.5
Capítulo 04: Currificación	1.5
No Puedo Vivir Si Vivir Es Sin Ti	1.5.1
Más Que Un Juego de Palabras / Salsa Especial	1.5.2
En resumen	1.5.3
Ejercicios	1.5.4
Capítulo 05: Programar Mediante Composición	1.6
Cultivo Funcional	1.6.1
Pointfree	1.6.2
Depurando	1.6.3
Teoría de Categorías	1.6.4
En Resumen	1.6.5
Ejercicios	1.6.6
Capítulo 06: Aplicación de Ejemplo	1.7
Programación Declarativa	1.7.1
Un Flickr Con Programación Funcional	1.7.2

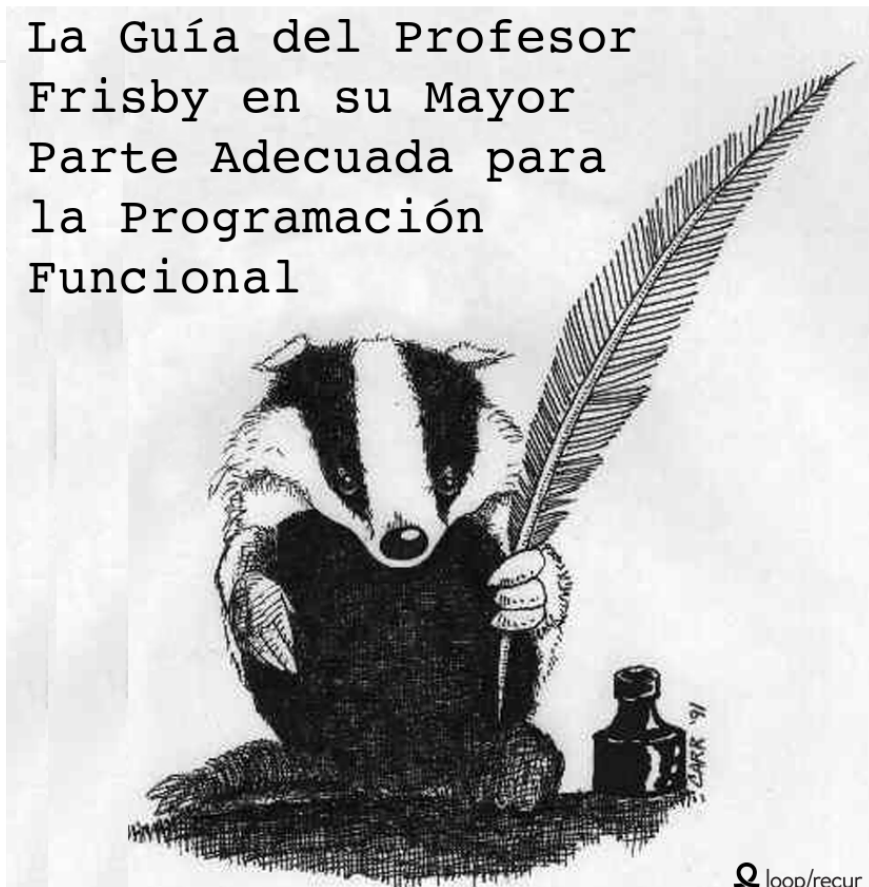
Una Refactorización Basada En Principios	1.7.3
En Resumen	1.7.4
Capítulo 07: Hindley-Milner y Yo	1.8
¿Cuál Es Tu Tipo?	1.8.1
Cuentos De Lo Críptico	1.8.2
Reduciendo las Posibilidades	1.8.3
Teoremas Gratuitos	1.8.4
Restricciones	1.8.5
En Resumen	1.8.6
Capítulo 08: Tupperware	1.9
El Poderoso Contenedor	1.9.1
Mi Primer Funtor	1.9.2
El Maybe de Schrödinger	1.9.3
Casos de Uso	1.9.4
Liberando el Valor	1.9.5
Manejo de Errores Puro	1.9.6
El Viejo McDonald Tenía Efectos...	1.9.7
Tareas Asíncronas	1.9.8
Un Poco de Teoría	1.9.9
En Resumen	1.9.10
Ejercicios	1.9.11
Capítulo 09: Cebollas Monádicas	1.10
Fábrica de Futores Puntiagudos	1.10.1
Mezclando Metáforas	1.10.2
Mi Cadena Me Golpea El Pecho	1.10.3
Borrachera de Poder	1.10.4
Teoría	1.10.5
En Resumen	1.10.6
Ejercicios	1.10.7
Capítulo 10: Futores Aplicativos	1.11
Aplicando Aplicativos	1.11.1
Barcos en Botellas	1.11.2
Motivación para la Coordinación	1.11.3
Colega, ¿Alguna Vez Levantas?	1.11.4
Operadores	1.11.5
Abrelatas Gratis	1.11.6
Leyes	1.11.7
En Resumen	1.11.8

Ejercicios	1.11.9
Capítulo 11: Transforma Otra Vez, Naturalmente	1.12
Maldice Este Nido	1.12.1
Una Comedia de Situación	1.12.2
Todo Natural	1.12.3
Conversión de Tipos Basada en Principios	1.12.4
Envidia de Características	1.12.5
JavaScript Isomórfico	1.12.6
Una Definición Más Amplia	1.12.7
Una Solución Para El Anidamiento	1.12.8
En Resumen	1.12.9
Ejercicios	1.12.10
Capítulo 12: Atravesando la Piedra	1.13
Tipos y Tipos	1.13.1
Feng Shui de Tipos	1.13.2
Surtido de Efectos	1.13.3
El Vals de los Tipos	1.13.4
Sin Ley Ni Orden	1.13.5
En Resumen	1.13.6
Ejercicios	1.13.7
Capítulo 13: Los Monoides Lo Unen Todo	1.14
Salvaje Combinación	1.14.1
Abstrayendo La Suma	1.14.2
Todos Mis Funtores Favoritos Son Semigrupos	1.14.3
Monoides A Cambio De Nada	1.14.4
Plegando La Casa	1.14.5
No Un Monoide Exactamente	1.14.6
Gran Teoría Unificadora	1.14.7
¿Teoría De Grupos O Teoría De Categorías?	1.14.8
En Resumen	1.14.9
Ejercicios	1.14.10
Apéndice A: Funciones Esenciales de Soporte	1.15
always	1.15.1
compose	1.15.2
curry	1.15.3
either	1.15.4
identity	1.15.5
inspect	1.15.6

left	1.15.7
liftA2	1.15.8
liftA3	1.15.9
maybe	1.15.10
nothing	1.15.11
reject	1.15.12
Apéndice B: Estructuras Algebraicas de Soporte	1.16
Compose	1.16.1
Either	1.16.2
Identity	1.16.3
IO	1.16.4
List	1.16.5
Map	1.16.6
Maybe	1.16.7
Task	1.16.8
Apéndice C: Utilidades Pointfree	1.17
add	1.17.1
append	1.17.2
chain	1.17.3
concat	1.17.4
eq	1.17.5
filter	1.17.6
flip	1.17.7
forEach	1.17.8
head	1.17.9
intercalate	1.17.10
join	1.17.11
last	1.17.12
map	1.17.13
match	1.17.14
prop	1.17.15
reduce	1.17.16
replace	1.17.17
reverse	1.17.18
safeHead	1.17.19
safeLast	1.17.20
safeProp	1.17.21
sequence	1.17.22

sortBy	1.17.23
split	1.17.24
take	1.17.25
toLowerCase	1.17.26
toString	1.17.27
toUpperCase	1.17.28
traverse	1.17.29
unsafePerformIO	1.17.30

La Guía del Profesor Frisby en su Mayor Parte Adecuada para la Programación Funcional



Introducción

Sobre este libro

Este es un libro sobre el paradigma funcional en general. Utilizaremos el lenguaje de programación funcional más popular del mundo: JavaScript. Hay quien vaya a pensar que es una mala elección, ya que va en contra de la cultura que, por el momento, es predominantemente imperativa. Sin embargo, creo que esta es la mejor forma de aprender programación funcional por diversas razones:

- **Seguramente lo utilices cada día en el trabajo.**

Esto hace posible practicar y aplicar cada día en programas del mundo real los conocimientos adquiridos en vez de en proyectos de una sola noche o de un fin de semana con un lenguaje de programación funcional esotérico.

- **No tenemos que aprenderlo todo desde cero para empezar a escribir programas.**

En un lenguaje funcional puro, no puedes registrar una variable o leer un nodo DOM sin usar mónadas. Aquí podemos hacer un poco de trampas mientras aprendemos a purificar nuestra base de código. También es más

fácil empezar con este lenguaje debido a su paradigma mixto y a que te puedes apoyar en lo que ya conoces mientras haya huecos en tu conocimiento.

- **El lenguaje está completamente capacitado para escribir código funcional de primera categoría.**

Tenemos todas las características necesarias para imitar a un lenguaje como Scala o Haskell con la ayuda de una o dos pequeñas librerías. La programación orientada a objetos domina actualmente la industria, pero es claramente torpe en JavaScript. Es similar a acampar en una autopista o bailar claqué con botas de agua. Tenemos que usar `bind` por todas partes para que `this` no cambie sin que nos demos cuenta, tenemos varias alternativas al peculiar comportamiento cuando olvidamos utilizar `new`, los miembros privados solo están disponibles mediante clausuras [*closures*]. En fin, para muchas personas la programación funcional parece más natural.

Dicho esto, los lenguajes funcionales tipados serán, sin ninguna duda, el mejor lugar para programar con el estilo que se presenta en este libro. JavaScript será nuestro medio para aprender un paradigma, dónde lo apliques depende de tí. Afortunadamente, las interfaces son matemáticas y, como tal, ubicuas. Te sentirás como en casa con Swiftz, Scalaz, Haskell, PureScript, y otros entornos con inclinación por las matemáticas.

Sobre la traducción

Se han añadido notas de traducción donde se ha visto necesario. Para no interferir con el ritmo de la lectura se ha optado por incluir la nota entre corchetes y en cursiva seguidamente de aquello que se esté anotando. Por ejemplo, "solo necesitas saber cómo encontrar y matar algunos bugs [*bichos*]". En caso de anotar un título la anotación se incluirá al comienzo del párrafo que le siga.

También se han modificado algunas frases para mantener un género neutro. Por ejemplo en vez de traducir "Some will argue that" a "Algunos argumentarán que", se ha traducido a "Hay quien argumentará que".

Léelo Online

Para una mejor experiencia en la lectura, [léelo online a través de Gitbook](#).

- Barra lateral de acceso rápido
- Ejercicios en el propio navegador
- Ejemplos en profundidad

Juega Con el Código

Para que el entrenamiento sea efectivo y no te aburras demasiado mientras te cuento otra historia, asegúrate de jugar con los conceptos introducidos en este libro. Algunos pueden ser difíciles de entender a la primera y se comprenden mejor cuando te ensucias las manos. Todas las funciones y estructuras de datos algebraicas presentadas en el libro están reunidas en los apéndices. El correspondiente código también está disponible como un módulo de npm:

```
$ npm i @mostly-adequate/support
```

Alternativamente, ¡los ejercicios de cada capítulo son ejecutables y pueden ser completados en tu editor! Por ejemplo, completa `exercise_*.js` en `exercises/ch04` y después ejecuta:

```
$ npm run ch04
```

Descárgalo

Encuentra archivos **PDF** y **EPUB** pregenerados como [artefactos construidos desde la última versión](#).

Hazlo tú mismo

⚠ La preparación del proyecto es un poco antigua, puedes encontrarte con distintos problemas cuando lo construyas localmente. Recomendamos el uso de node v10.22.1 y la última versión de Calibre si es posible.

```
git clone https://github.com/MostlyAdequate/mostly-adequate-guide-es.git
cd mostly-adequate-guide-es/
npm install
npm run setup
npm run generate-pdf
npm run generate-epub
```

¡Nota! Para generar la versión ebook necesitarás instalar `ebook-convert`. [Instrucciones de instalación](#).

Contenido

Ver [SUMMARY-es.md](#)

Contribuir

Ver [CONTRIBUTING-es.md](#)

Traducciones

Ver [TRANSLATIONS-es.md](#)

FAQ

Ver [FAQ-es.md](#)

Planes para el futuro

- **Parte 1** (capítulos 1-7) es una guía básica. La actualizaré a medida que encuentre errores, ya que esto es un borrador inicial. ¡Siéntete libre de ayudar!
- **Parte 2** (capítulos 8-13) aborda clases de tipos como funtores y mónadas llegando hasta traversable. Espero poder alcanzar transformadores y una aplicación pura.
- **Parte 3** (capítulos 14+) cruzará la delgada línea entre la programación práctica y la absurdidad académica. Veremos comónadas, f-algebras, mónadas libres, yoneda, y otras construcciones categóricas.



Este trabajo está licenciado bajo [licencia Creative Commons Atribución-CompartirIgual 4.0 Internacional](#).

Capítulo 01: ¿Qué Estamos Haciendo?

Presentaciones

¡Hola! Soy el Profesor Franklin Frisby, encantado de conocerte. Vamos a pasar algo de tiempo juntos, pues se supone que voy a enseñarte algo de programación funcional. Pero basta de hablar sobre mí, ¿qué hay de ti? Espero que el lenguaje JavaScript te sea por lo menos familiar, que tengas algo de experiencia en programación orientada a objetos, y que te apetezca convertirte en un programador de bandera. No necesitas tener un doctorado en entomología, solo necesitas saber cómo encontrar y matar algunos bugs [*bichos*].

No asumo que tengas ningún conocimiento previo sobre programación funcional porque ya sabemos lo que sucede cuando uno presupone, pero espero que hayas encontrado problemas al trabajar con estados mutables, con efectos secundarios no restringidos, y con diseño sin principios. Ahora que ya nos hemos presentado, sigamos adelante.

El propósito de este capítulo es darte una idea de lo que buscamos cuando escribimos programas funcionales. Para poder entender los próximos capítulos, hemos de tener una idea sobre qué hace que un programa sea *funcional*. De lo contrario, acabaremos garabateando sin rumbo, evitando objetos a toda costa; un esfuerzo sin sentido. Necesitamos una diana a la que lanzar nuestro código, una brújula celestial para cuando las aguas se agiten.

Hay ciertos principios de programación, varios acrónimos, que nos guiarán a través de los túneles oscuros de cualquier aplicación: DRY (don't repeat yourself [*no te repitas*]), YAGNI (ya ain't gonna need it [*no lo vas a necesitar*]), alta cohesión bajo acoplamiento, principio de mínima sorpresa, responsabilidad única, etc.

No voy a alargarme enumerando cada una de las guías que he escuchado a lo largo de los años... La cuestión es que siguen vigentes en un entorno funcional, aunque de forma meramente tangencial a nuestro objetivo final. Lo que me gustaría que entendieses por ahora, antes de seguir adelante, es cuál será nuestra intención cuando nos aferremos al teclado; nuestro Xanadú funcional.

Un Breve Encuentro

Vamos a empezar con un toque de locura. He aquí una aplicación de gaviotas [*seagulls*]. Cuando una bandada [*flock*] se junta con otra [*conjoin*], se convierten en una bandada más grande y cuando se reproducen [*breed*], aumentan por el número de gaviotas con las que se reproducen. Ahora bien, este no pretende ser un buen ejemplo de código orientado a objetos, ojo, este código está aquí para resaltar los peligros de nuestro moderno enfoque basado en asignación. Contempla:

```
class Flock {
  constructor(n) {
    this.seagulls = n;
  }

  conjoin(other) {
    this.seagulls += other.seagulls;
    return this;
  }

  breed(other) {
    this.seagulls = this.seagulls * other.seagulls;
    return this;
  }
}

const flockA = new Flock(4);
const flockB = new Flock(2);
const flockC = new Flock(0);
const result = flockA
  .conjoin(flockC)
  .breed(flockB)
  .conjoin(flockA.breed(flockB))
  .seagulls;

// 32
```

¿Quién en la faz de la tierra, sería capaz de crear tan espantosa abominación? Es irrazonablemente difícil seguir el rastro del estado interno mientras muta. Y, por si esto fuera poco, ¡la respuesta es incorrecta! Debería ser `16`, pero `flockA` ha sido alterado permanentemente durante el proceso. Pobre `flockA`. ¡Esto es anarquía en la informática! ¡Esto es aritmética de animales salvajes!

Si no entiendes este programa, no pasa nada, yo tampoco lo entiendo. La cuestión es que el estado y los valores mutables son difíciles de seguir, incluso en un ejemplo tan pequeño.

Vamos a intentarlo de nuevo, esta vez con un enfoque más funcional:

```
const conjoin = (flockX, flockY) => flockX + flockY;
const breed = (flockX, flockY) => flockX * flockY;

const flockA = 4;
const flockB = 2;
const flockC = 0;
const result =
  conjoin(breed(flockB, conjoin(flockA, flockC)), breed(flockA, flockB));

// 16
```

Bueno, esta vez la respuesta es correcta. Con mucho menos código. La anidación de funciones es algo confusa... (pondremos remedio a esto en el capítulo 5). Está mejor, pero profundicemos un poco más. Llamar a las cosas por su nombre tiene sus ventajas. Si hubiéramos examinado nuestras funciones más de cerca, habríamos descubierto que estamos utilizando simples sumas (`conjoin`) y multiplicaciones (`breed`).

Realmente no hay nada especial en estas dos funciones aparte de sus nombres. Vamos a renombrarlas a `multiply` [*multiplicar*] y `add` [*añadir*] para revelar sus verdaderas identidades.

```
const add = (x, y) => x + y;
const multiply = (x, y) => x * y;

const flockA = 4;
const flockB = 2;
const flockC = 0;
const result =
  add(multiply(flockB, add(flockA, flockC)), multiply(flockA, flockB));
// 16
```

Y con esto obtenemos el conocimiento de los antiguos:

```
// asociativa
add(add(x, y), z) === add(x, add(y, z));

// conmutativa
add(x, y) === add(y, x);

// identidad
add(x, 0) === x;

// distributiva
multiply(x, add(y, z)) === add(multiply(x, y), multiply(x, z));
```

Ah, sí, esas viejas y fieles propiedades matemáticas serán de ayuda. No te preocupes si no las sabes de memoria. Puede que haya pasado mucho tiempo desde que las estudiamos. Vamos a ver si podemos utilizar estas propiedades para simplificar nuestra pequeña aplicación de gaviotas.

```
// Línea original
add(multiply(flockB, add(flockA, flockC)), multiply(flockA, flockB));

// Aplicamos la propiedad de identidad para eliminar la suma sobrante
// (add(flockA, flockC) == flockA)
add(multiply(flockB, flockA), multiply(flockA, flockB));

// Aplicamos la propiedad distributiva para llegar a nuestro resultado
multiply(flockB, add(flockA, flockA));
```

¡Brillante! No hemos tenido que escribir ni una pizca de código aparte de las llamadas a las funciones. Hemos incluido las implementaciones de `add` y `multiply` por completitud, pero en realidad no hacía falta escribirlas, puesto que seguro que ya existen en alguna librería.

Seguramente estarás pensando "qué pícaro, al poner este ejemplo". O "en la realidad los programas no son tan simples y no se puede razonar sobre ellos de esta manera". He seleccionado este ejemplo porque la mayoría ya sabemos sumar y multiplicar, y así es fácil ver cómo las matemáticas pueden sernos útiles.

No te desespere, a lo largo de este libro hablaremos un poco sobre teoría de categorías, teoría de conjuntos, y cálculo lambda para escribir ejemplos de la vida real que consigan la misma elegante simplicidad y que resulten como nuestro ejemplo de la bandada de gaviotas. Tampoco necesitas tener un título en matemáticas, será sencillo y natural, como utilizar otro framework u otra API.

Puede resultar sorprendente oír que se pueden escribir aplicaciones completas y reales utilizando programación funcional tal y como hemos mostrado en el ejemplo anterior. Programas con sólidas propiedades. Programas cortos sobre los que razonar fácilmente. Programas que no reinventan la rueda una y otra vez. La falta de leyes es buena si vas a cometer un crimen, pero en este libro, vamos a reconocer y a obedecer a las leyes de las matemáticas.

Querremos utilizar una teoría en la que todas las piezas tiendan a encajar limpiamente. Querremos representar nuestro problema específico en términos de pequeñas piezas genéricas y combinables, para luego explotar sus propiedades en nuestro propio beneficio. Será necesaria un poco más de disciplina que en el enfoque del "todo vale" de la programación imperativa (más adelante definiremos más precisamente qué es la programación imperativa, pero por ahora considérala cualquier cosa que no sea programación funcional). La recompensa de trabajar dentro de un marco matemático basado en principios verdaderamente te asombrará.

Hemos visto un destello de nuestra estrella del norte funcional, pero hay unos cuantos conceptos que necesitamos entender antes de realmente poder emprender nuestro viaje.

[Capítulo 2: Funciones de Primera Clase](#)

Capítulo 02: Funciones de Primera Clase

Un Repaso Rápido

Cuando decimos que las funciones son de "primera clase", queremos decir que son como cualquier otra clase... o sea, una clase normal. Podemos tratar a las funciones como a cualquier otro tipo de dato y no hay nada particularmente especial en ellas; pueden ser almacenadas en arreglos, pasadas como parámetros de otras funciones, asignadas a variables, y lo que quieras.

Esto es de primero de JavaScript, pero vale la pena mencionarlo, pues una búsqueda rápida de código en GitHub mostrará la evasión colectiva, o tal vez la ignorancia generalizada de este concepto. ¿Deberíamos poner un ejemplo ficticio? Deberíamos.

```
const hi = name => `Hi ${name}`;  
const greeting = name => hi(name);
```

Aquí, en `greeting`, la función que envuelve a `hi` es completamente redundante. ¿Por qué? Porque las funciones son *llamables* en JavaScript. Cuando `hi` tiene los `()` al final, se ejecutará y devolverá un valor. Cuando no los tiene, simplemente devolverá la función almacenada en la variable. Solo para asegurarte, echa un vistazo tú mismo.

```
hi; // name => `Hi ${name}`  
hi("jonas"); // "Hi jonas"
```

Dado que `greeting` tan solo está llamando a `hi` con el mismo argumento, podríamos simplemente escribir:

```
const greeting = hi;  
greeting("times"); // "Hi times"
```

En otras palabras, `hi` ya es una función que espera un argumento, ¿por qué colocar otra función alrededor de ella que simplemente llame a `hi` con el mismo condenado argumento? No tiene ningún maldito sentido. Es como ponerte tu parka más pesada al final de un julio mortal solo para subir el aire acondicionado y pedir un helado.

Rodear una función con otra función simplemente para retrasar la evaluación es demasiado detallado y también una mala práctica. (Veremos por qué en un momento, pero tiene que ver con el mantenimiento.)

Es fundamental comprender bien esto antes de continuar, así que vamos a examinar algunos otros divertidos ejemplos extraídos de paquetes de npm.

```
// ignorante
const getServerStuff = callback => ajaxCall(json => callback(json));

// iluminado
const getServerStuff = ajaxCall;
```

El mundo está repleto de código ajax exactamente igual a este. He aquí la razón por la que ambos son equivalentes:

```
// esta línea
ajaxCall(json => callback(json));

// es lo mismo que esta línea
ajaxCall(callback);

// así que refactorizamos getServerStuff
const getServerStuff = callback => ajaxCall(callback);

// ...la cual es equivalente a esto
const getServerStuff = ajaxCall; // <-- mira mama, sin ()'s
```

Y así, amigos, es cómo se hace. Uno más para que entendamos por qué estoy siendo tan insistente.

```
const BlogController = {
  index(posts) { return Views.index(posts); },
  show(post) { return Views.show(post); },
  create(attrs) { return Db.create(attrs); },
  update(post, attrs) { return Db.update(post, attrs); },
  destroy(post) { return Db.destroy(post); },
};
```

Este ridículo controlador es 99% aire. Podríamos reescribirlo como:

```
const BlogController = {
  index: Views.index,
  show: Views.show,
  create: Db.create,
  update: Db.update,
  destroy: Db.destroy
};
```

...o desecharlo por completo, puesto que no hace más que agrupar `Views` y `Db`.

¿Por Qué Favorecer a las Funciones de Primera Clase?

Vale, vayamos a las razones por las que favorecer a las funciones de primera clase. Como vimos en los ejemplos `getServerStuff` y `BlogController`, es fácil agregar capas de indirección que no añaden ningún valor y que lo único que hacen es incrementar la cantidad de código en el que rebuscar y que mantener.

Además, si cambia una función que estamos envolviendo innecesariamente, también deberemos cambiar la función que la envuelve.

```
httpGet('/post/2', json => renderPost(json));
```

Si `httpGet` cambiase para enviar un posible error (`err`), necesitaríamos cambiar la función interna.

```
// ir a cada llamada a httpGet en la aplicación y pasar explícitamente err.  
httpGet('/post/2', (json, err) => renderPost(json, err));
```

Si la hubiéramos escrito como una función de primera clase, mucho menos necesitaríamos cambiar:

```
// renderPost es llamada desde dentro de httpGet, con todos los argumentos que  
httpGet('/post/2', renderPost);
```

Si no eliminamos las funciones innecesarias deberemos nombrar a los argumentos y hacer referencia a ellos. Los nombres son algo problemáticos, ya sabes. Existen potenciales errores de nombrado, especialmente cuando la base de código envejece y los requerimientos cambian.

Tener múltiples nombres para el mismo concepto suele ser una fuente de confusión en los proyectos. También existe el problema del código genérico. Por ejemplo, estas dos funciones hacen exactamente lo mismo, pero una es infinitamente más general y reusable.

```
// específico para nuestro blog actual  
const validArticles = articles =>  
  articles.filter(article => article !== null && article !== undefined),  
  
// mucho más relevante para futuros proyectos  
const compact = xs => xs.filter(x => x !== null && x !== undefined);
```

Usando nombres concretos, aparentemente nos atamos a datos específicos (en este caso `articles`). Esto sucede bastante a menudo y es una fuente de mucha de la reinención.

Debo mencionar que, al igual que con código orientado a objetos, debes ser consciente de que `this` puede morderte en la yugular. Si una función subyacente usa `this` y la llamamos como si fuese de primera clase, estamos sujetos a esta cólera de la abstracción con fugas.

```
const fs = require('fs');

// aterrador
fs.readFile('freaky_friday.txt', Db.save);

// no tanto
fs.readFile('freaky_friday.txt', Db.save.bind(Db));
```

Después de haber sido enlazada a sí misma, `Db` es libre de acceder a su prototípico código basura. Yo evito usar `this` de la misma manera que evito usar un pañal sucio. Realmente no hay ninguna necesidad cuando se escribe código funcional. Sin embargo, al interactuar con otras bibliotecas, tendrás que aceptar el loco mundo que nos rodea.

Hay quien argumentará que `this` es necesario para optimizar la velocidad. Si te va la micro-optimización, por favor cierra este libro. Si no puedes recuperar tu dinero, quizás puedas intercambiarlo por algo más complejo.

Y con esto estamos listos para seguir adelante.

[Capítulo 3: Pura Felicidad con Funciones Puras](#)

Capítulo 03: Pura Felicidad con Funciones Puras

Oh Ser Puro Otra Vez

Una cosa que necesitamos para comenzar correctamente es la idea de una función pura.

Una función pura es una función que, dada la misma entrada, siempre devolverá la misma salida y que no contiene ningún efecto secundario observable.

Toma por ejemplo `slice` y `splice`. Son dos funciones que hacen exactamente lo mismo, eso sí, de una forma muy diferente, pero lo mismo al fin y al cabo. Decimos que `slice` es *pura* porque siempre devuelve la misma salida para cada entrada, garantizado. `splice`, sin embargo, se comerá su array y lo escupirá cambiado para siempre, lo cual es un efecto observable.

```
const xs = [1,2,3,4,5];

// pura
xs.slice(0,3); // [1,2,3]

xs.slice(0,3); // [1,2,3]

xs.slice(0,3); // [1,2,3]

// impura
xs.splice(0,3); // [1,2,3]

xs.splice(0,3); // [4,5]

xs.splice(0,3); // []
```

En programación funcional, no nos gustan las funciones poco manejables como `splice`, que muta los datos. Esto no es aceptable, ya que nos esforzamos por tener funciones en las que podamos confiar, que devuelvan siempre la misma salida, no funciones que dejan un desastre a su paso como `splice`.

Veamos otro ejemplo.

```
// impuro
let minimum = 21;
const checkAge = age => age >= minimum;

// puro
const checkAge = (age) => {
  const minimum = 21;
  return age >= minimum;
};
```

En la parte impura, `checkAge` depende de la variable mutable `minimum` para determinar el resultado. En otras palabras, depende del estado del sistema, lo que es decepcionante porque incrementa la *carga cognitiva* al introducir un entorno externo.

Puede que no parezca mucho en este ejemplo, pero esta dependencia sobre el estado es una de las mayores contribuciones a la complejidad de los sistemas(<http://www.curtclifton.net/storage/papers/MoseleyMarks06a.pdf>). Esta `checkAge` puede devolver un resultado diferente dependiendo de factores externos a la entrada, lo que no solo la descalifica como pura, sino que además pone a prueba a nuestra mente cada vez que razonamos sobre el software.

Por otro lado, su forma pura, es completamente autosuficiente. También podemos hacer que `minimum` sea inmutable, lo que preserva la pureza, ya que el estado nunca cambia. Para hacer esto, debemos crear un objeto para poder congelarlo.

```
const immutableState = Object.freeze({ minimum: 21 });
```

Los Efectos Secundarios Pueden Incluir...

Veamos más cosas sobre estos "efectos secundarios" para mejorar nuestra intuición. Entonces, ¿qué es este indudablemente nefasto *efecto secundario* mencionado en la definición de *función pura*? Nos referiremos a *efecto* como a cualquier cosa que ocurra en nuestra computación que no sea calcular un resultado.

No hay nada intrínsecamente malo acerca de los efectos y los usaremos por todas partes en futuros capítulos. Es la parte *secundaria* la que tiene connotaciones negativas. El agua por sí sola no es una incubadora de larvas, es su *estancamiento* lo que produce los enjambres, y te aseguro que en tus propios programas los efectos *secundarios* son un criadero similar.

Un *efecto secundario* es un cambio en el estado del sistema, o una *interacción observable* con el mundo exterior, que sucede durante el cálculo de un resultado.

Los efectos secundarios pueden incluir, pero no limitarse a

- cambiar el sistema de ficheros
- insertar un registro en una base de datos
- hacer una llamada http
- mutar valores
- imprimir en pantalla/registro
- obtener entrada del usuario
- consultar el DOM
- acceder al estado del sistema

Y el listado sigue y sigue. Cualquier interacción de una función con el mundo exterior es un efecto secundario, hecho que puede llevarte a sospechar de la practicidad de programar sin ellos. La filosofía de la programación funcional postula que los efectos secundarios son la principal causa de las incorrecciones en el comportamiento.

No es que tengamos prohibido usarlos, más bien queremos contenerlos y ejecutarlos de manera controlada. Aprenderemos como hacerlo cuando lleguemos a los funtores y mónadas en capítulos posteriores, pero por ahora, trataremos de mantener a estas insidiosas funciones apartadas de las puras.

Los efectos secundarios descalifican a una función para ser *pura* y tiene sentido: las funciones puras, por definición, deben devolver siempre la misma salida dada la misma entrada, lo que no es garantizable cuando se manejan asuntos externos a nuestra función local.

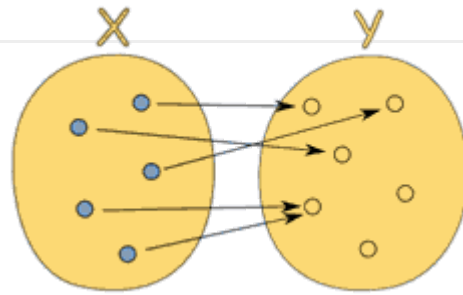
Veamos con más detalle por qué insistimos en lo de la misma salida para cada entrada. Levantaos el cuello de las camisas, vamos a ver algo de matemáticas de octavo grado [*estudiantes de entre 13 y 14 años].

Matemáticas de Octavo Grado

De mathisfun.com:

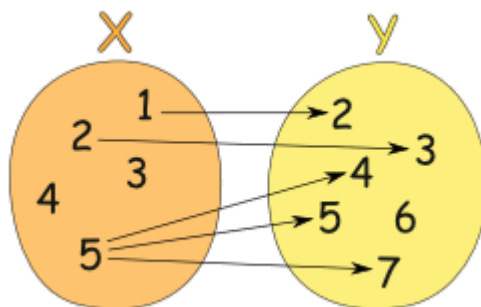
Una función es una relación especial entre valores: Cada uno de sus valores de entrada devuelve exactamente un valor de salida.

En otras palabras, tan solo es una relación entre dos valores: la entrada y la salida. Aunque cada entrada tiene exactamente una salida, esa salida no tiene que ser necesariamente única por cada entrada. El siguiente diagrama muestra una función de x a y perfectamente válida;



(<http://www.mathsisfun.com/sets/function.html>)

Para contrastar, el siguiente diagrama muestra una relación que *no* es una función, ya que el valor de entrada 5 apunta a varias salidas:



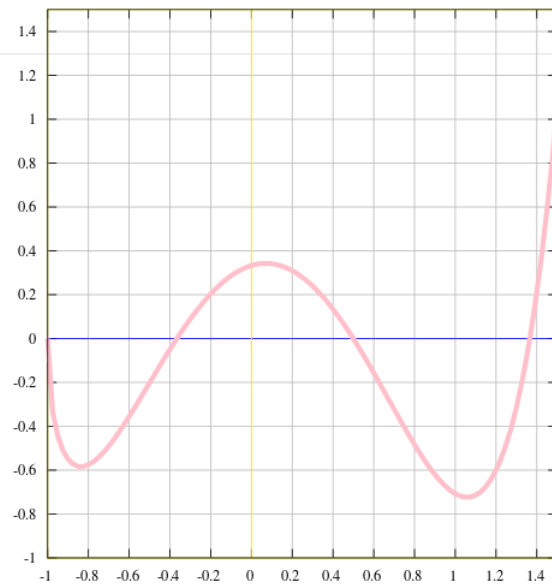
(<http://www.mathsisfun.com/sets/function.html>)

Las funciones pueden ser descritas como un conjunto de pares con la posición (entrada, salida): $[(1, 2), (3, 6), (5, 10)]$ (Parece que esta función dobla su entrada).

O tal vez una tabla:

Entrada	Salida
1	2
2	4
3	6

O incluso como un gráfico con x como la entrada e y como la salida:



No hay necesidad de detalles de implementación si la entrada dicta la salida. Ya que las funciones tan solo son mapeos de entrada a salida, podemos simplemente escribir objetos literales y ejecutarlos con `[]` en lugar de `()`.

```
const toLowerCase = {  
  A: 'a',  
  B: 'b',  
  C: 'c',  
  D: 'd',  
  E: 'e',  
  F: 'f',  
};  
toLowerCase['C']; // 'c'  
  
const isPrime = {  
  1: false,  
  2: true,  
  3: true,  
  4: false,  
  5: true,  
  6: false,  
};  
isPrime[3]; // true
```

Por supuesto, puedes querer calcular en lugar de apuntar valores a mano, pero esto ilustra una forma diferente de pensar sobre las funciones. (Debes estar pensando "¿qué pasa con las funciones con múltiples argumentos?". Ciertamente, esto presenta un pequeño inconveniente cuando se piensa en términos matemáticos. Por ahora, podemos empaquetarlos en un array o

simplemente pensar que como entrada pasamos el objeto `arguments`. Cuando aprendamos sobre *currying*, veremos cómo podemos modelar directamente la definición matemática de función.)

Aquí viene la dramática revelación: Las funciones puras *son* funciones matemáticas y son todo sobre lo que trata la programación funcional. Programar con estos pequeños ángeles puede tener grandes beneficios. Veamos algunas de las razones por las que estamos dispuestos a recorrer tan grandes distancias para preservar la pureza.

Los Argumentos Para La Pureza

Almacenable en Caché

Para empezar, las funciones puras siempre pueden ser almacenadas en caché por su entrada. Esto se hace típicamente con una técnica llamada memoización:

```
const squareNumber = memoize(x => x * x);

squareNumber(4); // 16

squareNumber(4); // 16, devuelve lo almacenado en caché para la entrada 4

squareNumber(5); // 25

squareNumber(5); // 25, devuelve lo almacenado en cache para la entrada 5
```

Aquí hay una implementación simplificada, aunque haya disponibles otras mucho más robustas.

```
const memoize = (f) => {
  const cache = {};

  return (...args) => {
    const argStr = JSON.stringify(args);
    cache[argStr] = cache[argStr] || f(...args);
    return cache[argStr];
  };
};
```

Algo a tener en cuenta es que puedes transformar funciones impuras en puras retrasando su evaluación:

```
const pureHttpCall = memoize((url, params) => () => $.getJSON(url, params));
```

Lo interesante aquí es que realmente no hacemos la llamada http; en su lugar devolvemos una función que lo hará cuando sea llamada. Esta función es pura porque siempre devolverá la misma salida dada la misma entrada: la función que hará esa llamada http en particular dados `url` y `params`.

Nuestra función `memoize` funciona bien, aunque no guarda en caché los resultados de la llamada http, sino que guarda la función generada.

Esto todavía no es muy útil, pero pronto aprenderemos algunos trucos que harán que lo sea. La lección es que podemos guardar en caché cualquier función sin importar cuán destructiva parezca.

Portables / Autodocumentadas

Las funciones puras son completamente autocontenidas. Todo lo que necesita la función se le pasa en bandeja de plata. Considera esto por un momento...

¿Cómo puede esto ser beneficioso? Para empezar, las dependencias de la función son explícitas y, por lo tanto, más fáciles de ver y entender; nada extraño sucede a escondidas.

```
// impura
const signUp = (attrs) => {
  const user = saveUser(attrs);
  welcomeUser(user);
};

// pura
const signUp = (Db, Email, attrs) => () => {
  const user = saveUser(Db, attrs);
  welcomeUser(Email, user);
};
```

Este ejemplo demuestra que la función pura debe ser honesta acerca de sus dependencias y que como tal debe decirnos exactamente qué es lo que hace. Solo por su firma, sabemos que usará una `Db`, `Email` y `attrs`, lo que debería ser, cuanto menos, revelador.

Aprenderemos a crear funciones puras como esta sin limitarnos a tan solo aplazar la evaluación, pero debería quedar claro que la forma pura es mucho más informativa que su escurridiza contraparte que trama quién sabe qué.

Algo más a tener en cuenta es que se nos obliga a "inyectar" dependencias, pasándolas como argumentos, lo que hace a nuestra aplicación más flexible, pues hemos parametrizado nuestra base de datos, cliente de email o lo que sea (no te preocupes, veremos una manera de hacer esto menos tedioso de lo que parece). Si decidimos usar una base de datos diferente solo necesitaremos llamar con ella a nuestra función. Si nos encontramos escribiendo una nueva aplicación en la que nos gustaría reutilizar esta confiable función, simplemente tendremos que pasar a esta función la `Db` y el `Email` que tengamos en ese momento.

En un entorno JavaScript, portabilidad puede significar serializar y enviar funciones por un socket. Puede significar ejecutar toda nuestra aplicación con Web Workers. La portabilidad es un rasgo poderoso.

Al contrario de los "típicos" métodos y procedimientos de la programación imperativa profundamente enraizados a sus entornos a través de estado, dependencias y efectos, las funciones puras pueden ejecutarse allá donde nuestro corazón desee.

¿Cuándo fué la última vez que copiaste un método en una nueva app? Una de mis citas favoritas proviene del creador de Erlang, Joe Armstrong: "El problema con los lenguajes orientados a objetos es todo ese entorno implícito que llevan a todos lados con ellos. Querías una banana, pero tienes un gorila sosteniendo una banana... y la jungla entera".

Testeable

Después de lo anterior, nos damos cuenta de que las funciones puras hacen que el testing sea mucho más fácil. No necesitamos mockear una pasarela de pagos "real" o configurar y verificar el estado del mundo después de cada test. Simplemente, pasamos la entrada a la función y verificamos su salida.

De hecho, la comunidad funcional está siendo pionera nuevas herramientas de pruebas que pueden bombardear nuestra función con entradas generadas y verificar que sus propiedades se mantienen en la salida. Está fuera del alcance de este libro, pero os animo encarecidamente a que busquéis y probéis *Quickcheck*; una herramienta de pruebas que está hecha a medida para un entorno puramente funcional.

Comprensible

Muchas personas creen que la mayor victoria cuando trabajas con funciones puras es la *transparencia referencial*. Un trozo de código es referencialmente transparente cuando puede ser sustituido por su valor resultante sin cambiar el comportamiento del programa.

Dado que las funciones puras no tienen efectos secundarios, tan solo pueden influir en el comportamiento de un programa a través de sus valores de salida. Además, puesto que sus valores de salida pueden calcularse de forma fiable con tan solo utilizar sus valores de entrada, las funciones puras siempre mantendrán la transparencia referencial. Veamos un ejemplo.

```
const { Map } = require('immutable');

// Alias: p = player, a = attacker, t = target
const jobe = Map({ name: 'Jobe', hp: 20, team: 'red' });
const michael = Map({ name: 'Michael', hp: 20, team: 'green' });
const decrementHP = p => p.set('hp', p.get('hp') - 1);
const isSameTeam = (p1, p2) => p1.get('team') === p2.get('team');
const punch = (a, t) => (isSameTeam(a, t) ? t : decrementHP(t));

punch(jobe, michael); // Map({name:'Michael', hp:19, team: 'green'})
```

`decrementHP`, `isSameTeam` y `punch` son todas puras y, por tanto, referencialmente transparentes. Podemos usar la técnica llamada *razonamiento ecuacional*, donde podemos sustituir "iguales por iguales" para razonar sobre el código. Es un poco como evaluar manualmente el código sin tener en cuenta las peculiaridades de la evaluación programática. Usando transparencia referencial, juguemos un poco con este código.

Primero reemplazamos la función `isSameTeam`.

```
const punch = (a, t) => (a.get('team') === t.get('team') ? t : decrementHP(t));
```

Ya que nuestros datos son inmutables, podemos simplemente reemplazar cada equipo [`team`] por su valor real

```
const punch = (a, t) => ('red' === 'green' ? t : decrementHP(t));
```

Vemos que en este caso la condición es falsa por lo que podemos quitar toda la rama del `if`

```
const punch = (a, t) => decrementHP(t);
```

Y si también reemplazamos `decrementHP`, vemos que, en este caso, `punch` se convierte en una llamada para reducir `hp` en 1.

```
const punch = (a, t) => t.set('hp', t.get('hp') - 1);
```

Esta habilidad para razonar acerca del código es excelente para, en general, refactorizarlo y entenderlo. De hecho, hemos utilizado esta técnica para refactorizar nuestro programa de bandada de gaviotas. Usamos razonamiento ecuacional para aprovechar las propiedades de adición y multiplicación. De hecho, utilizaremos estas técnicas a lo largo de todo el libro.

Código Paralelo

Finalmente, y aquí está el golpe de gracia, podemos ejecutar en paralelo cualquier función pura, ya que no necesita acceder a memoria compartida y no puede, por definición, tener una condición de carrera debido a algún efecto secundario.

Esto podría usarse tanto en un servidor con entorno js e hilos de ejecución como en un navegador con web workers, aunque la cultura actual parece evitarlo debido a lo complejo que resulta tratar con funciones impuras.

En Resumen

Hemos visto qué son las funciones puras y por qué en programación funcional creemos que son extraordinarias. De aquí en adelante, nos esforzaremos en escribir todas nuestras funciones de una forma pura. Necesitaremos algunas herramientas adicionales para ayudarnos, pero mientras tanto, trataremos de separar las funciones impuras del resto del código puro.

Resulta un poco laborioso escribir programas con funciones puras al no tener algunas herramientas extra en nuestro cinturón. Hemos de hacer malabares con los datos pasando argumentos por todas partes, tenemos prohibido utilizar estado y sin mencionar lo de los efectos secundarios. ¿Cómo afrontar la escritura de estos programas de masoquista? Obtengamos una nueva herramienta llamada curry.

[Capítulo 4: Currying](#)

Capítulo 04: Currificación

No Puedo Vivir Si Vivir Es Sin Ti

[El título en inglés es 'Can't Live If Livin' Is without You' que recuerda a la canción "Without You" de Badfinger]

Mi padre una vez me explicó como hay ciertas cosas sin las que se puede vivir hasta que las compras. Un microondas es una de esas cosas. Los smartphones, otra. Quien sea ya mayor recordará una vida de plenitud sin internet. Para mí, la currificación [currying en inglés] está en esta lista.

El concepto es sencillo: Puedes llamar a una función con menos argumentos de los que espera. Esta devolverá una función que esperará los argumentos restantes.

Puedes elegir llamarla con todos sus argumentos de una vez o simplemente pasarle cada argumento poco a poco.

```
const add = x => y => x + y;
const increment = add(1);
const addTen = add(10);

increment(2); // 3
addTen(2); // 12
```

Aquí hemos hecho una función `add` que acepta un argumento y devuelve una función. A partir de entonces, al llamarla, la función devuelta recuerda el primer argumento mediante el cierre [closure]. Sin embargo, llamarla con ambos argumentos de una vez es un poco molesto, por lo que podemos utilizar una función de soporte especial llamada `curry` para facilitar la definición y la llamada de funciones como esta.

Vamos a preparar unas pocas funciones currificadas para nuestro disfrute. Desde ahora, nos apoyaremos en nuestra función `curry` definida en el [Apéndice A - Funciones Esenciales de Soporte](#).

```
const match = curry((what, s) => s.match(what));
const replace = curry((what, replacement, s) => s.replace(what, replacement));
const filter = curry((f, xs) => xs.filter(f));
const map = curry((f, xs) => xs.map(f));
```

El patrón que he seguido es uno simple, pero importante. He posicionado estratégicamente los datos sobre los que vamos a operar (String, Array) como último argumento. Quedará mas claro por qué cuando lo usemos.

(La sintaxis `/r/g` es una expresión regular que significa *encuentra todas las letras 'r'*. Lee [más sobre expresiones regulares](#) si quieres.)

```

match(/r/g, 'hello world'); // [ 'r' ]

const hasLetterR = match(/r/g); // x => x.match(/r/g)
hasLetterR('hello world'); // [ 'r' ]
hasLetterR('just j and s and t etc'); // null

filter(hasLetterR, ['rock and roll', 'smooth jazz']); // ['rock and roll']

const removeStringsWithoutRs = filter(hasLetterR); // xs => xs.filter(x => x.ma
removeStringsWithoutRs(['rock and roll', 'smooth jazz', 'drum circle']); // ['r

const noVowels = replace(/[aeiou]/ig); // (r,x) => x.replace(/[aeiou]/ig, r)
const censored = noVowels('*'); // x => x.replace(/[aeiou]/ig, '*')
censored('Chocolate Rain'); // 'Ch*c*1*t* R**n'

```

Lo que demostramos aquí es la habilidad para precargar una función con un argumento o dos con el fin de recibir una nueva función que recuerde dichos argumentos.

Os animo a clonar el repositorio de Mostly Adequate (`git clone https://github.com/MostlyAdequate/mostly-adequate-guide-es.git`), copiar el código anterior y probarlo en la consola REPL. La función curry, igual que cualquier cosa definida en los apéndices, están disponibles en el módulo `support/index.js`.

Alternativamente, dale un vistazo a la versión en inglés publicada en `npm`:

```
npm install @mostly-adequate/support
```

Más Que Un Juego de Palabras / Salsa Especial

La currificación es útil para muchas cosas. Podemos hacer nuevas funciones solo pasando algunos argumentos a nuestras funciones base, tal y como hemos visto en `hasLetterR`, `removeStringsWithoutRs` y `censored`.

También tenemos la habilidad para transformar cualquier función que trabaje con un solo elemento en una función que trabaje con una lista, simplemente envolviéndola con `map`:

```

const getChildren = x => x.childNodes;
const allTheChildren = map(getChildren);

```

Pasar a una función menos argumentos de los que espera, típicamente se conoce como *aplicación parcial*. Aplicar parcialmente una función puede quitar mucho código repetitivo. Considera como sería la función anterior

`allTheChildren` con el `map` sin currificación de `lodash` (fíjate que los argumentos están en diferente orden):

```
const allTheChildren = elements => map(elements, getChildren);
```

Normalmente, no definimos funciones que trabajan con arrays, porque simplemente podemos llamar en línea a `map(getChildren)`. Lo mismo con `sort`, `filter`, y otras funciones de alto orden (una *función de alto orden* es una función que acepta o devuelve una función).

Cuando hablábamos de *funciones puras*, dijimos que llevan de 1 entrada a 1 salida. La currificación hace exactamente eso: cada uno de los argumentos devuelve una nueva función que espera los argumentos restantes. Eso, viejo amigo, es 1 entrada a 1 salida.

No importa si la salida es otra función; califica como pura. Permitimos más de un argumento a la vez, pero viéndolo tan solo como para quitar por conveniencia los `()` adicionales.

En resumen

Las currificación es práctica y disfruto mucho trabajando diariamente con funciones currificadas. Es una herramienta para el cinturón que hace a la programación funcional menos verbosa y tediosa.

Podemos crear al vuelo nuevas y útiles funciones, simplemente pasándole unos pocos argumentos y además, retenemos la definición de función matemática a pesar de sus múltiples argumentos.

Adquiramos otra herramienta esencial llamada `compose`.

[Capítulo 5: Programando por Composición](#)

Ejercicios

Nota sobre los ejercicios

A lo largo del libro, podrás encontrar una sección 'Ejercicios' como esta. Los ejercicios pueden hacerse directamente en el navegador siempre que estés leyendo desde [gitbook](#) (recomendado).

Fíjate que, para todos los ejercicios del libro, siempre tienes en el contexto global unas útiles funciones de ayuda. Por lo tanto, ¡cualquier cosa definida en el [Apéndice A](#), [Apéndice B](#) y [Apéndice C](#) está disponible para ti! Y, por si esto fuera poco, algunos ejercicios también definen funciones específicas al problema que presentan; de hecho, también considéralas disponibles.

Sugerencia: ¡puedes enviar tu solución con `ctrl + Enter` en el editor embebido!

Ejecutando Los Ejercicios En Tu Máquina (opcional)

En caso de preferir hacer los ejercicios directamente en archivos usando tu propio editor:

- clona el repositorio (`git clone git@github.com:MostlyAdequate/mostly-adequate-guide-es.git`)
- ve a la sección *exercises* (`cd mostly-adequate-guide-es/exercises`)
- instala lo necesario usando `npm` (`npm install`)
- completa las respuestas modificando los archivos llamados *exercise_** dentro del directorio del correspondiente capítulo
- ejecuta la corrección con `npm` (p. ej. `npm run ch04`)

Los tests unitarios se ejecutarán con tus respuestas y darán sugerencias en caso de error. Por cierto, las respuestas a los ejercicios están en los archivos llamados *solution_**.

¡Practiquemos!

Exercise

Refactoriza para quitar todos los argumentos aplicando parcialmente la función.

```
// words :: String -> [String]
const words = str => split(' ', str);
```

Exercise

Refactoriza para quitar todos los argumentos aplicando parcialmente las funciones.

```
// filterQs :: [String] -> [String]
const filterQs = xs => filter(x => x.match(/q/i), xs);
```

Teniendo en cuenta la siguiente función:

```
const keepHighest = (x, y) => (x >= y ? x : y);
```

Exercise

Refactoriza ``max`` para que no haga referencia a ningún argumento usando la función de ayuda ``keepHighest``.

```
// max :: [Number] -> Number  
const max = xs => reduce((acc, x) => (x >= acc ? x : acc), -Infinity, xs);
```

Capítulo 05: Programar Mediante Composición

Cultivo Funcional

Aquí tenemos `compose` :

```
const compose = (...fns) => (...args) => fns.reduceRight((res, fn) => [fn.call
```

... ¡No te asustes! Este es el nivel-9000-super-Saiyan de *compose*. En aras del razonamiento, ignoremos la implementación variádica y consideremos una forma más simple capaz de componer juntas a dos funciones. Una vez te hayas hecho a la idea, puedes llevar la abstracción más allá y considerar que simplemente funciona para cualquier número de funciones (¡incluso podemos aportar pruebas de ello!) Aquí tenemos una función *compose* más amigable para quiénes me estáis leyendo:

```
const compose2 = (f, g) => x => f(g(x));
```

`f` y `g` son funciones y `x` es el valor que está siendo canalizado a través de ellas.

La composición es una suerte de cultivo de funciones. Tú, horticultor de funciones, seleccionas dos con las características que te gustaría combinar y las mezclas para engendrar una nueva. Su uso es el siguiente:

```
const toUpperCase = x => x.toUpperCase();
const exclaim = x => `${x}!`;
const shout = compose(exclaim, toUpperCase);

shout('send in the clowns'); // "SEND IN THE CLOWNS!"
```

La composición de dos funciones devuelve una nueva función. Esto tiene todo el sentido: componer dos unidades de algún tipo (en este caso función) debería devolver una nueva unidad de ese mismo tipo. No conectas dos legos entre sí y obtienes un "Lincoln Log" [*juego de construcción de casitas de madera*]. Existe una teoría aquí, una ley subyacente que descubriremos a su debido tiempo.

En nuestra definición de `compose`, la `g` se ejecutará antes que la `f`, creando un flujo de datos de derecha a izquierda. Esto es mucho más legible que tener un montón de funciones anidadas. Sin `compose`, lo anterior sería:

```
const shout = x => exclaim(toUpperCase(x));
```

En vez de adentro hacia afuera, lo ejecutamos de derecha a izquierda, lo cual supongo que es un paso a la izquierda (¡buu!) *[chiste malo que pierde la gracia en la traducción. En inglés, un paso a la derecha también quiere decir un paso en la buena dirección. En este chiste, el paso es hacia la izquierda, o sea, en la mala dirección]*.

Veamos un ejemplo donde el orden en la secuencia importa:

```
const head = x => x[0];
const reverse = reduce((acc, x) => [x, ...acc], []);
const last = compose(head, reverse);

last(['jumpkick', 'roundhouse', 'uppercut']); // 'uppercut'
```

`reverse` le da la vuelta a la lista, mientras que `head` coge el elemento inicial. Esto resulta en una efectiva a la vez que ineficiente función `last`. La secuencia de funciones en la composición debería ser evidente aquí. Podríamos definir una versión de izquierda a derecha, sin embargo, tal y como está se asemeja más a la versión matemática. Sí, eso es correcto, la composición viene directamente de los libros de matemáticas. De hecho, quizás es el momento de ver una propiedad que es válida para cualquier composición.

```
// asociatividad
compose(f, compose(g, h)) === compose(compose(f, g), h);
```

La composición es asociativa, lo cual significa que no importa como agrupes dos de ellas. Entonces, si elegimos pasar la cadena de caracteres a mayúsculas, podemos escribir:

```
compose(toUpperCase, compose(head, reverse));
// o también
compose(compose(toUpperCase, head), reverse);
```

Dado que no importa cómo agrupemos nuestras llamadas a `compose`, el resultado será el mismo. Esto nos permite escribir `compose` como una función *variádica* y usarla así:

```
// anteriormente hubiésemos tenido que escribir dos composiciones, pero como es
// podemos dar a compose tantas funciones como queramos y dejar que decida como
const arg = ['jumpkick', 'roundhouse', 'uppercut'];
const lastUpper = compose(toUpperCase, head, reverse);
const loudLastUpper = compose(exclaim, toUpperCase, head, reverse);

lastUpper(arg); // 'UPPERCUT'
loudLastUpper(arg); // 'UPPERCUT!'
```

Aplicar la propiedad asociativa nos da esta flexibilidad y la tranquilidad de que el resultado será equivalente. La definición variádica y un poco más complicada está incluida en las librerías de soporte de este libro y es la definición que se suele encontrar en librerías como [lodash](#), [underscore](#), y [ramda](#).

Un agradable beneficio de la asociatividad es que cualquier grupo de funciones puede ser extraído y agrupado en su propia composición. Juguemos a refactorizar nuestro ejemplo anterior:

```
const loudLastUpper = compose(exclaim, toUpperCase, head, reverse);

// -- o también -----

const last = compose(head, reverse);
const loudLastUpper = compose(exclaim, toUpperCase, last);

// -- o también -----

const last = compose(head, reverse);
const angry = compose(exclaim, toUpperCase);
const loudLastUpper = compose(angry, last);

// más variaciones...
```

No hay repuestas correctas o incorrectas; solo estamos juntando nuestras piezas de lego de la manera que nos plazca. Normalmente, lo mejor es agrupar las cosas de manera que se puedan reutilizar, como `last` y `angry`. Si te es familiar "Refactoring" de Fowler, quizás reconozcas a este proceso como "extract function"... excepto por no tener que preocuparte por el estado de ningún objeto.

Pointfree

El estilo pointfree [*que se puede encontrar traducido como programación tácita*] se refiere a no hablar nunca sobre tus datos. Perdona. Se refiere a funciones que nunca mencionan los datos sobre los que operan. Las funciones de primera clase, la currificación, y la composición hacen un buen equipo para crear este estilo.

Sugerencia: En el [Apéndice C - Utilidades Pointfree](#) hay definidas versiones pointfree de `replace` y `toLowerCase`. ¡No dudes en echar un vistazo!

```
// no es pointfree porque mencionamos a los datos: word
const snakeCase = word => word.toLowerCase().replace(/s+/ig, '_');

// pointfree
const snakeCase = compose(replace(/s+/ig, '_'), toLowerCase);
```

¿Ves como hemos aplicado `replace` parcialmente? Lo que estamos haciendo es canalizar nuestros datos a través de cada una de las funciones de un solo argumento. La currificación nos permite preparar cada función para que solo coja los datos, opere con ellos, y se los pase a quien siga. Algo más a destacar es como en la versión pointfree no necesitamos a los datos para construir nuestra función, mientras que con la función no pointfree, necesitamos tener disponible a nuestra `word` [*palabra*] antes que nada.

Veamos otro ejemplo.

```
// no es pointfree porque mencionamos a los datos: name
const initials = name => name.split(' ').map(compose(toUpperCase, head)).join(' ');

// pointfree
// NOTA: utilizamos 'intercalate' del apéndice en lugar de 'join' presentada en el apéndice
const initials = compose(intercalate(' '), map(compose(toUpperCase, head)), split(' '));

initials('hunter stockton thompson'); // 'H. S. T'
```

El código pointfree puede de nuevo ayudarnos a eliminar nombres innecesarios y mantenernos concisos y genéricos. Pointfree es un buen indicador para saber si nuestro código funcional está compuesto de pequeñas funciones que toman un input y devuelven un output. No se puede componer un bucle while, por ejemplo. Sin embargo, ten cuidado porque pointfree es un arma de doble filo y a veces puede ofuscar la intención. No todo el código funcional es pointfree y eso está bien. Lo utilizaremos cuando podamos y si no, de lo contrario, utilizaremos funciones normales.

Depurando

Un error común es componer algo como `map`, una función de dos argumentos, sin antes aplicarla parcialmente.

```
// incorrecto - terminamos pasando un array a angry y aplicamos map parcialmente
const latin = compose(map, angry, reverse);

latin(['frog', 'eyes']); // error

// correcto - cada función espera 1 argumento.
const latin = compose(map(angry), reverse);

latin(['frog', 'eyes']); // ['EYES!', 'FROG!']
```

Si estás teniendo problemas para depurar una composición, podemos utilizar esta útil pero impura función `trace` para ver qué es lo que está pasando.

```
const trace = curry((tag, x) => {
  console.log(tag, x);
  return x;
});

const dasherize = compose(
  intercalate('-'),
  toLower,
  split(' '),
  replace(/\s{2,}/ig, ' '),
);

dasherize('The world is a vampire');
// TypeError: Cannot read property 'apply' of undefined
```

Parece que algo fue mal, vamos a probar con `trace`

```
const dasherize = compose(
  intercalate('-'),
  toLower,
  trace('después de split'),
  split(' '),
  replace(/\s{2,}/ig, ' '),
);

dasherize('The world is a vampire');
// después de split [ 'The', 'world', 'is', 'a', 'vampire' ]
```

¡Ah! Necesitamos usar `toLower` con `map` ya que está trabajando con un array.

```
const dasherize = compose(
  intercalate('-'),
  map(toLower),
  split(' '),
  replace(/\s{2,}/ig, ' '),
);

dasherize('The world is a vampire'); // 'the-world-is-a-vampire'
```

A la hora de depurar, la función `trace` nos permite observar los datos en un cierto punto. Lenguajes como Haskell y PureScript tienen funciones similares para facilitar el desarrollo.

La composición será nuestra herramienta para construir programas y, afortunadamente, está respaldada por una poderosa teoría que asegura que las cosas funcionarán. Examinemos esta teoría.

Teoría de Categorías

La teoría de categorías es una rama abstracta de las matemáticas que puede formalizar conceptos a partir de distintas ramas como la teoría de conjuntos, la teoría de tipos, la teoría de grupos, lógica, y más. Principalmente maneja objetos, morfismos, y transformaciones, lo cual se asemeja bastante a programar. He aquí una gráfica de los mismos conceptos vistos según las distintas teorías.

Types	Logic	Sets	Homotopy
A	proposition	set	space
$a : A$	proof	element	point
$B(x)$	predicate	family of sets	fibration
$b(x) : B(x)$	conditional proof	family of elements	section
$0, 1$	$\perp, \top$	$\emptyset, \{\emptyset\}$	$\emptyset, *$
$A + B$	$A \vee B$	disjoint union	coproduct
$A \times B$	$A \wedge B$	set of pairs	product space
$A \rightarrow B$	$A \Rightarrow B$	set of functions	function space
$\sum_{(x:A)} B(x)$	$\exists_{x:A} B(x)$	disjoint sum	total space
$\prod_{(x:A)} B(x)$	$\forall_{x:A} B(x)$	product	space of sections
Id_A	equality $=$	$\{ (x, x) \mid x \in A \}$	path space A^I

Lo siento, no pretendía asustarte. No espero que estés íntimamente familiarizado con todos estos conceptos. Mi intención es mostrarte cuanta duplicación existe y así puedas ver cómo la teoría de categorías tiene por objeto unificar estas cosas.

En la teoría de categorías, tenemos algo que se llama... una categoría. Está definida como una colección con los siguientes componentes:

- Una colección de objetos
- Una colección de morfismos
- Una noción de composición en los morfismos
- Un morfismo en particular llamado identidad

La teoría de categorías es suficientemente abstracta como para modelar muchas cosas, pero vamos a aplicar esto a tipos y funciones, que es lo que nos importa en este momento.

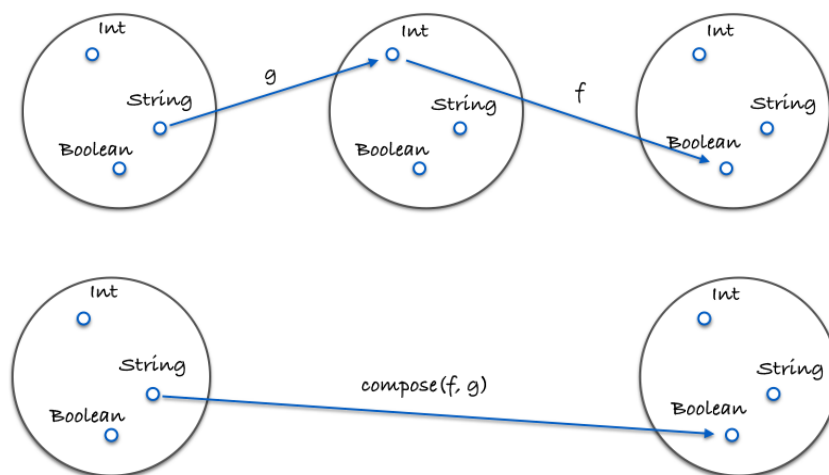
Una colección de objetos Los objetos serán tipos de datos. Por ejemplo, `String`, `Boolean`, `Number`, `Object`, etc. Frecuentemente vemos a los tipos de datos como un conjunto de todos los valores posibles. Se puede ver a `Boolean` como el conjunto de `[true, false]` y a `Number` como el conjunto de todos los valores numéricos posibles. Tratar a los tipos como conjuntos es útil porque podemos utilizar la teoría de conjuntos con ellos.

Una colección de morfismos Los morfismos serán nuestras funciones puras estándar de cada día.

Una noción de composición en los morfismos Esto, como ya habrás adivinado, es nuestro flamante juguete nuevo: `compose`. Hemos visto que nuestra función `compose` es asociativa, lo cual no es una coincidencia, ya que es

una propiedad que debe mantenerse para cualquier composición en la teoría de categorías.

Aquí tenemos una imagen que demuestra la composición:



He aquí un ejemplo concreto en código:

```
const g = x => x.length;
const f = x => x === 4;
const isFourLetterWord = compose(f, g);
```

Un morfismo en particular llamado identidad Introduzcamos una útil función llamada `id`. Esta función simplemente acepta una entrada y te la escupe de vuelta. Echa un vistazo:

```
const id = x => x;
```

Quizás te preguntes a tí mismo "¿Para qué demonios puede ser esto útil?". En los siguientes capítulos haremos un uso intensivo de esta función, pero por ahora piensa en ella como una función que puede sustituir a nuestro valor; una función que se hace pasar por datos normales y corrientes.

`id` ha de interactuar bien con `compose`. Aquí tenemos una propiedad que siempre se cumple para cualquier función unaria `f` (unaria: función de un solo argumento):

```
// identidad
compose(id, f) === compose(f, id) === f;
// true
```

¡Eh, es como la propiedad de identidad en los números! Si esto no lo ves claro inmediatamente, dedícale algún tiempo. Entiende la futilidad. Pronto veremos a `id` usada en todas partes, pero por ahora veámosla como a una función que actúa como sustituta de un valor dado. Esto es bastante útil a la hora de escribir código pointfree.

Así que ahí lo tienes, una categoría de tipos y funciones. Si esta es tu primera introducción, imagino que seguirás algo confuso sobre qué es una categoría y su utilidad. Aprenderemos más sobre todo esto a lo largo del libro. Por el momento, en este capítulo, en esta línea, puedes verlo al menos como que nos provee de algo de sabiduría sobre la composición; concretamente sobre las propiedades de identidad y asociatividad.

¿Qué otras categorías hay, te preguntarás? Bien, podemos definir una para grafos dirigidos en la que los nodos son objetos, las aristas son morfismos, y la composición es simplemente una concatenación de caminos. Podemos definir otra con Números como objetos y $\geq$ como morfismos (en realidad cualquier orden parcial o total puede ser una categoría). Hay montones de categorías, pero para el propósito de este libro solo tendremos en cuenta la definida anteriormente. Hemos mirado por encima lo suficiente y tenemos que seguir.

En Resumen

La composición conecta nuestras funciones como si de una especie de tuberías se tratase. Los datos fluirán a través de nuestra aplicación como es debido; después de todo las funciones puras van de entrada a salida, por lo que romper esta cadena invalidaría la salida, convirtiendo a nuestro software en inútil.

Consideramos a la composición como el principio de diseño que está por encima de todos los demás. Esto se debe a que mantiene a nuestra app simple y razonable. La teoría de categorías desempeñará un papel importante en la arquitectura de aplicaciones, modelando los efectos secundarios, y asegurando que está libre de errores.

Hemos llegado a un punto donde nos será útil ver algo de esto en la práctica. Hagamos una aplicación de ejemplo.

[Capítulo 6: Aplicación de Ejemplo](#)

Ejercicios

En cada uno de los siguientes ejercicios, consideraremos objetos Car [*Coche*] con la siguiente forma:

```
{
  name: 'Aston Martin One-77',
  horsepower: 750,
  dollar_value: 1850000,
  in_stock: true,
}
```

Exercise

Utiliza `compose()` para reescribir la función de abajo.

```
// isLastInStock :: [Car] -> Boolean
const isLastInStock = (cars) => {
  const lastCar = last(cars);
  return prop('in_stock', lastCar);
};
```

Teniendo en cuenta la siguiente función:

```
const average = xs => reduce(add, 0, xs) / xs.length;
```

Exercise

Usa la función de soporte `average` para refactorizar `averageDollarValue` a una composición.

```
// averageDollarValue :: [Car] -> Int
const averageDollarValue = (cars) => {
  const dollarValues = map(c => c.dollar_value, cars);
  return average(dollarValues);
};
```

Exercise

Refactoriza `fastestCar` utilizando `compose()` y otras funciones en estilo pointfree. Pista, la función `append` puede resultar útil.

```
const fastestCar = (cars) => {
  const sorted = sortBy(car => car.horsepower, cars);
  const fastest = last(sorted);
  return concat(fastest.name, ' is the fastest');
};
```

Capítulo 06: Aplicación de Ejemplo

Programación Declarativa

Vamos a cambiar nuestra mentalidad. A partir de ahora, dejaremos de decirle al ordenador cómo hacer su trabajo y, en cambio, escribiremos una especificación de lo que nos gustaría obtener como resultado. Estoy seguro que lo encontrarás mucho menos estresante que intentar microgestionarlo todo continuamente.

Declarativo, al contrario que imperativo, significa que escribiremos expresiones en lugar de instrucciones paso a paso.

Piensa en SQL. No existe un "primero haz esto, luego haz lo otro". Existe una expresión que especifica lo que nos gustaría obtener de la base de datos. No decidimos como hacer el trabajo, es la base de datos quien lo decide. Cuando se actualiza la base de datos y el motor de SQL es optimizado, no tenemos que cambiar nuestra consulta. Esto se debe a que existen muchas maneras de interpretar nuestra especificación y conseguir el mismo resultado.

Para algunas personas, yo incluido, cuesta entender de primeras el concepto de programación declarativa, así que vamos a mostrar algunos ejemplos para hacernos una idea.

```
// imperativo
const makes = [];
for (let i = 0; i < cars.length; i += 1) {
  makes.push(cars[i].make);
}

// declarativo
const makes = cars.map(car => car.make);
```

El bucle imperativo primero debe crear el array. El intérprete debe evaluar esta sentencia antes de continuar. Luego itera directamente sobre la lista de automóviles, incrementando manualmente un contador y mostrándonos sus partes y piezas en una vulgar exhibición de iteración explícita.

La versión con `map` es una sola expresión. No requiere ningún orden de evaluación. Hay mucha libertad en cuanto a la forma en que la función `map` itera y a como debe ser ensamblado el array devuelto. Especifica el *qué*, no el *cómo*. Así pues, es quien lleva puesta la brillante banda declarativa.

Además de ser más clara y más concisa, la función `map` puede ser optimizada a voluntad sin que el valioso código de nuestra aplicación necesite cambiar.

Para quien esté pensando "Sí, pero es mucho más rápido hacer el bucle imperativo", les sugiero que se informen sobre cómo el JIT optimiza su código. Aquí hay un [excelente video que puede arrojar algo de luz](#).

He aquí otro ejemplo.

```
// imperativo
const authenticate = (form) => {
  const user = toUser(form);
  return logIn(user);
};

// declarativo
const authenticate = compose(logIn, toUser);
```

Aunque no hay nada necesariamente malo en la versión imperativa, esta sigue escondiendo una evaluación por pasos. La expresión con `compose` simplemente afirma un hecho: la autenticación es la composición de `toUser` y `logIn`. Nuevamente, esto deja margen de maniobra para permitir cambios en el código, y hace que nuestro código de aplicación sea una especificación de alto nivel.

Como no tenemos que codificar el orden de evaluación, la programación declarativa se presta a la computación paralela. Esto, junto con las funciones puras, es la razón por la que la programación funcional es una buena opción para el futuro paralelo; en realidad no tenemos que hacer nada especial para conseguir sistemas paralelos/concurrentes.

Un Flickr Con Programación Funcional

Ahora construiremos una aplicación de ejemplo de manera declarativa y usando composición. Por ahora seguiremos haciendo trampas y utilizando efectos secundarios, pero los mantendremos al mínimo y los separaremos de nuestra base de código puro. Vamos a construir un widget para el navegador que obtenga imágenes de Flickr y las muestre. Empecemos creando el andamiaje de la aplicación. Aquí está el html:

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <title>Flickr App</title>
  </head>
  <body>
    <main id="js-main" class="main"></main>
    <script src="https://cdnjs.cloudflare.com/ajax/libs/require.js/2.2.0/require.js"></script>
    <script src="main.js"></script>
  </body>
</html>
```

Y aquí está esqueleto de main.js

```
const CDN = s => `https://cdnjs.cloudflare.com/ajax/libs/${s}`;
const ramda = CDN('ramda/0.21.0/ramda.min');
const jquery = CDN('jquery/3.0.0-rc1/jquery.min');

requirejs.config({ paths: { ramda, jquery } });
requirejs(['jquery', 'ramda'], ($, { compose, curry, map, prop }) => {
  // la aplicación va aquí
});
```

Estamos usando **ramda** en lugar de lodash o cualquier otra librería de utilidades. Incluye `compose`, `curry`, y más. He usado requirejs, hecho que puede parecer exagerado, pero la usaremos a lo largo de todo el libro y mantener la consistencia es clave.

Ahora que hemos dejado esto claro, vamos a la especificación. Nuestra aplicación hará 4 cosas.

1. Construir una url para nuestro término de búsqueda en particular
2. Hacer una llamada a la api de flickr
3. Transformar el json resultante en imágenes html
4. Colocarlas en la pantalla

Arriba se mencionan 2 acciones impuras. ¿Puedes verlas? Esos pedacitos donde se obtienen datos de la api de flickr y donde se muestran en la pantalla. Definémoslos primero para así poder ponerlos en cuarentena. Además, añadiré nuestra bonita función `trace` para poder depurar fácilmente.

```
const Impure = {
  getJSON: curry((callback, url) => $.getJSON(url, callback)),
  setHtml: curry((sel, html) => $(sel).html(html)),
  trace: curry((tag, x) => { console.log(tag, x); return x; }),
};
```

Aquí simplemente hemos envuelto los métodos de jQuery para ser currificados y hemos intercambiado los argumentos a una posición más favorable. He agregado las funciones al espacio de nombres `Impure` para saber que son funciones peligrosas. En un futuro ejemplo, haremos que estas dos funciones sean puras.

A continuación debemos construir una url para pasársela a nuestra función

`Impure.getJSON` .

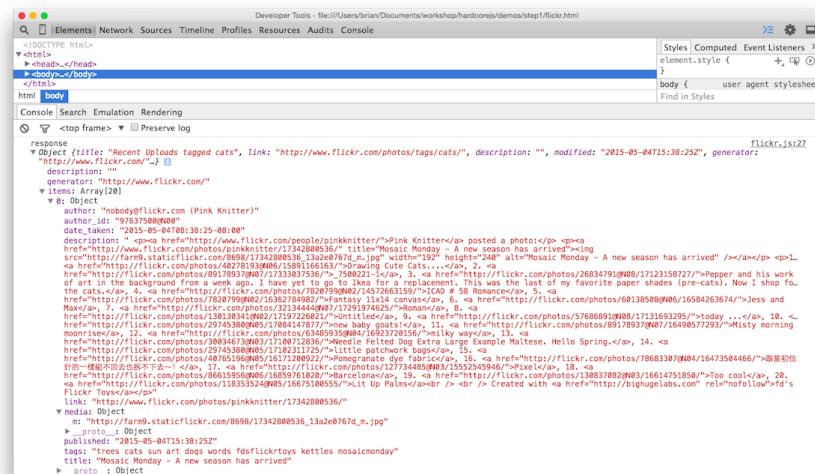
```
const host = 'api.flickr.com';
const path = '/services/feeds/photos_public.gne';
const query = t => `?tags=${t}&format=json&jsoncallback=?`;
const url = t => `https://${host}${path}${query(t)}`;
```

Hay formas extravagantes y demasiado complejas de escribir `ur1` en estilo pointfree usando monoides (aprenderemos sobre ello más adelante) o combinadores. Hemos optado por quedarnos con una versión más legible y ensamblar esta cadena de una forma no pointfree.

Escribamos una función `app` que haga la llamada y coloque los contenidos en la pantalla.

```
const app = compose(Impure.getJSON(Impure.trace('response')), ur1);
app('cats');
```

Esta llama a nuestra función `ur1`, luego pasa la cadena a nuestra función `getJSON`, que ha sido aplicada parcialmente con `trace`. Cuando arranque la aplicación mostrará por consola la respuesta a la llamada a la API.



Nos gustaría construir imágenes a partir de este json. Parece que las `mediaUrls` están enterradas en la propiedad `m` de `media` de cada uno de los `items`.

De cualquier manera, para llegar a estas propiedades anidadas podemos usar una útil función getter universal de rambda llamada `prop`. Aquí tienes una versión casera para que puedas ver lo que está pasando:

```
const prop = curry((property, object) => object[property]);
```

Realmente es bastante aburrida. Tan solo usamos la sintaxis `[]` para acceder a una propiedad de cualquier objeto. Vamos a usar esto para llegar a nuestras `mediaUrls`.

```
const mediaUrl = compose(prop('m'), prop('media'));
const mediaUrls = compose(map(mediaUrl), prop('items'));
```

Una vez hemos reunido a los `items`, debemos aplicarles `map` para extraer cada url. Esto da como resultado un bonito array de `mediaUrls`. Conectemos esto a nuestra aplicación y presentemos las imágenes por pantalla.

```
var renderImages = _.compose(Impure.setHtml("body"), srcs);
var app = _.compose(Impure.getJSON(renderImages), url);
```

Todo lo que hemos hecho es una nueva composición que llamará a nuestras `mediaUrls` y establecerá el html de `<main>` con ellas. Ahora que tenemos algo para mostrar aparte de un json en crudo, hemos reemplazado la llamada a `trace` por la llamada a `renderImages`. Esto mostrará nuestras `mediaUrls` dentro del `body` de forma cruda.

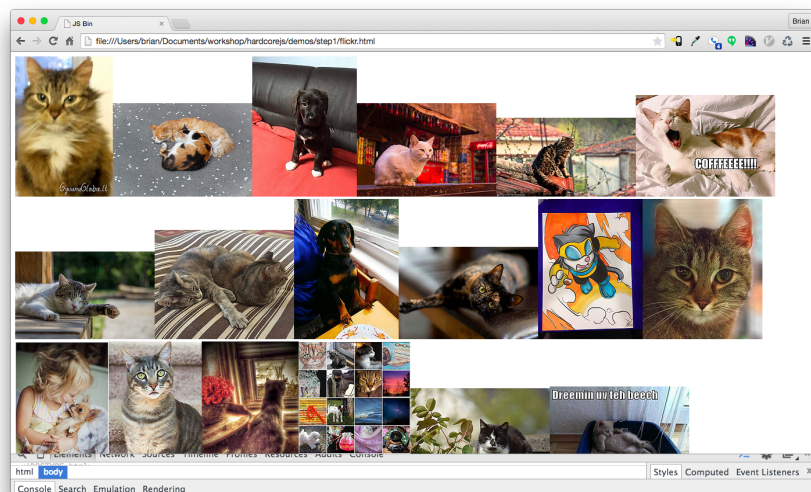
Nuestro paso final es convertir esas `mediaUrls` en imágenes de verdad. En una aplicación más grande, usaríamos una librería de template/dom como Handlebars o React. Sin embargo, para esta aplicación solo necesitamos una etiqueta `img`, así que vamos a seguir con jQuery.

```
const img = src => $('<img />', { src });
```

El método `html` de jQuery aceptará un array de etiquetas. Solo tenemos que transformar nuestras `mediaUrls` en imágenes y enviárselas a `setHtml`.

```
const images = compose(map(img), mediaUrls);
const render = compose(Impure.setHtml('#js-main'), images);
const app = compose(Impure.getJSON(render), url);
```

Y ¡hemos terminado!



Aquí está el script completo:

```

const CDN = s => `https://cdnjs.cloudflare.com/ajax/libs/${s}`;
const ramda = CDN('ramda/0.21.0/ramda.min');
const jquery = CDN('jquery/3.0.0-rc1/jquery.min');

requirejs.config({ paths: { ramda, jquery } });
require(['jquery', 'ramda'], ($, { compose, curry, map, prop }) => {
  // -- Utils -----
  const Impure = {
    trace: curry((tag, x) => { console.log(tag, x); return x; }), // eslint-dis
    getJSON: curry((callback, url) => $.getJSON(url, callback)),
    setHtml: curry((sel, html) => $(sel).html(html)),
  };

  // -- Pure -----
  const host = 'api.flickr.com';
  const path = '/services/feeds/photos_public.gne';
  const query = t => `?tags=${t}&format=json&jsoncallback=?`;
  const url = t => `https://${host}${path}${query(t)}`;

  const img = src => `<img />`, { src };
  const mediaUrl = compose(prop('m'), prop('media'));
  const mediaUrls = compose(map(mediaUrl), prop('items'));
  const images = compose(map(img), mediaUrls);

  // -- Impure -----
  const render = compose(Impure.setHtml('#js-main'), images);
  const app = compose(Impure.getJSON(render), url);

  app('cats');
});

```

Mira eso. Una especificación hermosamente declarativa de lo que son las cosas, no de cómo llegan a serlo. Ahora vemos a cada línea como a una ecuación con sus propiedades. Podemos usar estas propiedades para razonar acerca de nuestra aplicación y refactorizarla.

Una Refactorización Basada En Principios

Hay una optimización disponible; hacemos `map` sobre cada elemento para convertirlo en una url, luego hacemos nuevamente `map` sobre esas `mediaUrls` para convertirlas en etiquetas `img`. He aquí una ley con respecto a `map` y la composición:

```
// ley de composición de map
compose(map(f), map(g)) === map(compose(f, g));
```

Podemos utilizar esta propiedad para optimizar nuestro código. Refactorizemos basándonos en principios.

```
// código original
const mediaUrl = compose(prop('m'), prop('media'));
const mediaUrls = compose(map(mediaUrl), prop('items'));
const images = compose(map(img), mediaUrls);
```

Pongamos en la misma línea nuestras funciones `map`. Podemos incluir el código de `mediaUrls` directamente en `images` gracias al razonamiento ecuacional y a la pureza.

```
const mediaUrl = compose(prop('m'), prop('media'));
const images = compose(map(img), map(mediaUrl), prop('items'));
```

Ahora que tenemos a nuestras funciones `map` en la misma línea podemos aplicar la ley de la composición.

```
/*
compose(map(f), map(g)) === map(compose(f, g));
compose(map(img), map(mediaUrl)) === map(compose(img, mediaUrl));
*/

const mediaUrl = compose(prop('m'), prop('media'));
const images = compose(map(compose(img, mediaUrl)), prop('items'));
```

Ahora, al convertir cada elemento en una `img`, solo recorrerá el bucle una vez. Hagámoslo un poco más legible extrayendo la función.

```
const mediaUrl = compose(prop('m'), prop('media'));
const mediaToImg = compose(img, mediaUrl);
const images = compose(map(mediaToImg), prop('items'));
```

En Resumen

Con una aplicación pequeña, pero real, hemos visto como poner en práctica nuestras nuevas habilidades. Hemos utilizado nuestro marco matemático para razonar sobre nuestro código y refactorizarlo. Pero ¿qué hay del manejo de errores y la ramificación de código? ¿Cómo podemos hacer que toda la aplicación sea completamente pura en vez de tan solo agregar a un espacio de

nombres las funciones destructivas? ¿Cómo podemos hacer que nuestra aplicación sea más segura y expresiva? Estas son las preguntas que abordaremos en la parte 2.

Capítulo 7: Hindley-Milner y Yo

Capítulo 07: Hindley-Milner y Yo

¿Cuál Es Tu Tipo?

Si tu llegada al mundo funcional es reciente, no tardarás en verte de firmas de tipo hasta las rodillas. Los tipos son el meta lenguaje que permite a personas de todos los ámbitos comunicarse de forma sucinta y eficaz. La mayoría de las veces están escritas en un sistema llamado "Hindley-Milner" que examinaremos juntos en este capítulo.

Cuando trabajamos con funciones puras, las firmas de tipo tienen un poder expresivo que el inglés no puede lograr [*ni el español*]. Estas firmas te susurran al oído los íntimos secretos de una función. En una única y compacta línea exponen comportamiento e intención. Podemos derivar "teoremas gratuitos" de ellas. Los tipos pueden ser inferidos, así que no hay necesidad de anotaciones de tipo explícitas. Pueden afinarse al detalle o dejarse como algo general y abstracto. No solo son útiles para comprobaciones en tiempo de compilación, sino que resultan ser la mejor documentación posible disponible. Las firmas de tipos juegan, por tanto, un papel importante en la programación funcional; mucho más de lo que cabría esperar en un principio.

JavaScript es un lenguaje dinámico, pero eso no significa que evitemos los tipos por completo. Seguimos trabajando con strings, números, booleanos, etc. Es solo que no hay una integración a nivel de lenguaje, por lo que mantenemos esta información en nuestras cabezas. No hay de que preocuparse, podemos utilizar comentarios para servir a nuestro propósito dado que usamos las firmas como documentación.

Existen herramientas de comprobación de tipos para JavaScript como [Flow](#) o el dialecto con tipos, [TypeScript](#). La finalidad de este libro es equiparnos con las herramientas para escribir código funcional, así que nos quedaremos con el sistema de tipos estándar utilizado en muchos lenguajes de programación funcional.

Cuentos De Lo Críptico

Desde las polvorientas páginas de los libros de matemáticas, a través del vasto océano de papers académicos, entre publicaciones informales de blogs un sábado por la mañana, hasta en el propio código fuente, encontramos firmas de tipos de Hindley-Milner. El sistema es bien sencillo, pero merece una rápida explicación y algo de práctica para absorber completamente el pequeño lenguaje.

```
// capitalize :: String -> String
const capitalize = s => toUpperCase(head(s)) + toLowerCase(tail(s));

capitalize('smurf'); // 'Smurf'
```

Aquí, `capitalize` recibe un `String` y retorna un `String`. No importa la implementación, es la firma de tipos lo que nos interesa.

En HM, las funciones se escriben como `a -> b` dónde `a` y `b` son variables de cualquier tipo. Así que la firma para `capitalize` se puede leer como "una función de `String` a `String`". En otras palabras, toma un `String` como entrada y retorna un `String` como salida.

Veamos otras firmas de funciones:

```
// strLength :: String -> Number
const strLength = s => s.length;

// join :: String -> [String] -> String
const join = curry((what, xs) => xs.join(what));

// match :: Regex -> String -> [String]
const match = curry((reg, s) => s.match(reg));

// replace :: Regex -> String -> String -> String
const replace = curry((reg, sub, s) => s.replace(reg, sub));
```

`strLength` es la misma idea que antes: tomamos un `String` y te retorna un `Number`.

Las otras pueden dejarte perplejo en un principio. Sin entender del todo los detalles, siempre puedes ver al último tipo como el valor de retorno. Así que podemos interpretar `match` como: Toma un `Regex` y un `String` y te devuelve `[String]`. Pero una cosa interesante ocurre aquí y me gustaría tomarme un tiempo para explicarla, si se me permite.

Para `match` podemos agrupar las firmas como queramos:

```
// match :: Regex -> (String -> [String])
const match = curry((reg, s) => s.match(reg));
```

Ah si, agrupar la última parte con paréntesis revela más información. Ahora se ve como una función que toma un `Regex` y nos devuelve una función de `String` a `[String]`. Gracias al currying, este es claramente el caso: Le entregamos un `Regex` y obtenemos de vuelta una función que espera su argumento `String`. Por supuesto, no tenemos que verlo así, pero es una buena forma de entender por qué el último tipo es el que se devuelve.

```
// match :: Regex -> (String -> [String])
// onHoliday :: String -> [String]
const onHoliday = match(/holiday/ig);
```

Cada argumento hace saltar un tipo de delante de la firma. `onHoliday` es `match` que ya tiene un `Regex`.

```
// replace :: Regex -> (String -> (String -> String))
const replace = curry((reg, sub, s) => s.replace(reg, sub));
```

Como puedes ver en los paréntesis de `replace`, la notación extra puede ser algo ruidosa y redundante así que simplemente la omitimos. Podemos entregarle todos los argumentos a la vez si lo deseamos, así que es más fácil pensar en ello como: `replace` toma un `Regex`, un `String`, otro `String`, y te devuelve un `String`.

Unas últimas cosas:

```
// id :: a -> a
const id = x => x;

// map :: (a -> b) -> [a] -> [b]
const map = curry((f, xs) => xs.map(f));
```

La función `id` toma cualquier tipo `a` y devuelve algo del mismo tipo `a`. Podemos usar variables en los tipos, igual que en el código. Los nombres de las variables como `a` y `b` son una convención, pero son arbitrarios y pueden ser reemplazados por el nombre que prefieras. Si son la misma variable, tienen que ser del mismo tipo. Esta es una regla importante así que vamos a repetirnos: `a -> b` puede ser cualquier tipo `a` a cualquier tipo `b`, pero `a -> a` significa que tienen que ser del mismo tipo. Por ejemplo, `id` puede ser `String -> String` o `Number -> Number`, pero no `String -> Bool`.

`map` usa variables de tipos de forma parecida, pero esta vez introducimos `b` que puede ser del mismo tipo que `a` o no. Podemos leerla como: `map` toma una función de cualquier tipo `a` al mismo o distinto tipo `b`, luego toma un array de `a`s y devuelve un array de `b`s.

Ojalá te hayas dejado llevar por la belleza expresiva de esta firma de tipos. Nos dice literalmente lo que hace la función casi palabra por palabra. Se le da una función de `a` a `b`, un array de `a`, y nos entrega un array de `b`. Lo único sensato que puede hacer es llamar a la maldita función para cada `a`. Cualquier otra cosa sería una mentira descarada.

Ser capaz de razonar sobre los tipos y sus implicaciones es una habilidad que te llevará lejos en el mundo funcional. No solo los artículos académicos, blogs, documentación, etcétera, se volverán más digeribles, sino que la propia firma prácticamente te enseñará su funcionalidad. Lleva práctica ganar fluidez lectora, pero si perseveras, un montón de información se volverá accesible para ti sin necesidad de leer todo el manual.

Aquí hay algunas más para ver si puedes descifrarlas por ti mismo.

```
// head :: [a] -> a
const head = xs => xs[0];

// filter :: (a -> Bool) -> [a] -> [a]
const filter = curry((f, xs) => xs.filter(f));

// reduce :: ((b, a) -> b) -> b -> [a] -> b
const reduce = curry((f, x, xs) => xs.reduce(f, x));
```

`reduce` quizás sea la más expresiva de todas. Sin embargo, es difícil, así que no te sientas mal si te cuesta entenderla. Para los curiosos, intentaré hacer una explicación en castellano, aunque estudiar la firma por tu cuenta es mucho más instructivo.

Ejem, aquí va mi intento... revisando la firma, vemos que el primer argumento es una función que espera un tipo `b` y un tipo `a` y produce un `b`. ¿De dónde tomará todos esos `a`s y `b`s? Bueno, los siguientes argumentos en la firma son un `b`, y un array de `a`, por lo que solo podemos asumir que `b` y cada `a` serán inyectados a la función. También podemos ver que el resultado de la función es `b`, por lo que la conclusión será que el último hechizo de la función que hemos pasado será nuestro valor de salida. Conociendo lo que hace `reduce`, podemos afirmar que la investigación anterior es correcta.

Reduciendo las Posibilidades

Una vez que se introduce una variable de tipo, surge una curiosa propiedad llamada *parametricidad*. Esta propiedad establece que una función *actuará en todos los tipos de manera uniforme*. Investiguemos:

```
// head :: [a] -> a
```

Observando `head`, vemos que toma `[a]` y retorna `a`. Aparte del tipo concreto `array`, no dispone de más información, y, por lo tanto, su funcionalidad se limita a trabajar solamente con el array. ¿Qué podría hacer con la variable `a` si no sabe nada de ella? En otras palabras, `a` dice que no puede ser un tipo *específico*, lo que significa que puede ser *cualquier* tipo, lo que nos deja con una función que debe funcionar de manera uniforme para *todos* los tipos concebibles. Esto es todo en lo que consiste la *parametricidad*. Adivinando la implementación, las únicas suposiciones razonables son que toma el primer, el último o un elemento aleatorio del array. El nombre `head` [*cabeza*] debería servirnos de pista.

He aquí otra más:

```
// reverse :: [a] -> [a]
```

De la firma de tipos por sí sola, ¿qué podríamos inferir de `reverse` ? De nuevo, no puede hacer nada específico a `a` . No puede cambiar `a` a otro tipo o tendríamos que introducir el tipo `b` . ¿Puede ordenar? Bueno, no, no tendría suficiente información para ordenar todos los tipos posibles. ¿Puede reordenar? Sí, supongo que puede hacerlo, pero siempre tendría que hacerlo exactamente de la misma y predecible forma. Otra posibilidad es que decida eliminar o duplicar un elemento. En cualquier caso, la cuestión es que el posible comportamiento se ve masivamente reducido por su tipo polimórfico.

Esta reducción de posibilidades nos permite utilizar buscadores de firmas de tipos como [Hoogle](#) para encontrar la función que estamos buscando. La información contenida en una firma es, en efecto, muy poderosa.

Teoremas Gratuitos

Además de deducir posibilidades de implementación, este tipo de razonamiento nos proporciona *teoremas gratuitamente*. Lo que sigue son algunos teoremas de ejemplo extraídos aleatoria y directamente del [artículo académico de Wadler sobre el tema](#).

```
// head :: [a] -> a
compose(f, head) === compose(head, map(f));

// filter :: (a -> Bool) -> [a] -> [a]
compose(map(f), filter(compose(p, f))) === compose(filter(p), map(f));
```

No necesitas ningún código para entender estos teoremas, se deducen directamente de los tipos. El primero dice que obtener el primer elemento del array con `head` y luego ejecutar una función `f` sobre él, es equivalente y de paso mucho más rápido que primero aplicar `f` sobre cada elemento mediante `map` y luego aplicar `head` sobre el resultado.

Podrías pensar, bueno, eso es de sentido común. Pero la última vez que lo comprobé, los ordenadores no tenían sentido común. De hecho, han de tener una manera formal de automatizar este tipo de optimizaciones de código. Las matemáticas tienen una manera de formalizar lo intuitivo, lo cual es muy útil en medio del rígido terreno de la lógica computacional.

El teorema `filter` es parecido. Dice que si componemos `f` y `p` para comprobar cuál debe ser filtrado, y luego aplicamos `f` via `map` (recuerda que `filter` no transformará a los elementos; su firma dice que `a` no será tocado), siempre será equivalente a mapear nuestra `f` y luego filtrar el resultado con el predicado `p` .

Estos son solo dos ejemplos, pero puedes aplicar este razonamiento a cualquier firma de tipos polimórficos y siempre se mantendrá. En JavaScript, hay disponibles algunas herramientas para declarar reglas de reescritura. También se podría hacer a través de la propia función `compose` . La fruta está al alcance de la mano y las posibilidades son infinitas.

Restricciones

Una última cosa a tener en cuenta es que podemos restringir los tipos a una interfaz.

```
// sort :: Ord a => [a] -> [a]
```

Lo que vemos en el lado izquierdo de nuestra función flecha es la declaración de un hecho: `a` debe ser un `Ord`. O en otras palabras, `a` debe implementar la interfaz `Ord`. ¿Qué es `Ord` y de dónde viene? En un lenguaje con tipos sería una interfaz definida que dice que podemos ordenar los valores. Esto no solo nos dice más acerca de `a` y lo que nuestra función `sort` hace, sino que también restringe el dominio. Llamamos *restricciones de tipo* a estas declaraciones de interfaz.

```
// assertEquals :: (Eq a, Show a) => a -> a -> Assertion
```

Aquí, tenemos dos restricciones: `Eq` y `Show`. Así podremos comprobar la igualdad de nuestras `a` s e imprimir la diferencia si no son iguales.

Veremos más ejemplos de restricciones y la idea debería coger más forma en capítulos posteriores.

En Resumen

Las firmas de tipo Hindley-Milner son omnipresentes en el mundo funcional. Aunque son sencillas de leer y escribir, lleva tiempo dominar la técnica de entender los programas tan solo a través de sus firmas. A partir de ahora añadiremos firmas de tipo a cada línea de código.

[Capítulo 8: Tupperware](#)

Capítulo 08: Tupperware

El Poderoso Contenedor



Hemos visto como escribir programas que canalicen los datos a través de una serie de funciones puras. Son especificaciones declarativas de comportamiento. Pero ¿qué pasa con el control de flujo, el manejo de errores, las acciones asíncronas, el estado y me atrevo a decir, los efectos?! En este capítulo, descubriremos los cimientos sobre los que se construyen todas estas abstracciones tan útiles.

Primero crearemos un contenedor. Este contenedor debe poder contener cualquier tipo de valor; una bolsa de cierre zip que solo acepta pudin de tapioca raramente es útil. Será un objeto, pero no le daremos métodos ni propiedades en el sentido de la orientación de objetos. No, lo trataremos como si de un cofre del tesoro se tratara; una caja especial que envuelve a nuestros valiosos datos.

```
class Container {  
  constructor(x) {  
    this.$value = x;  
  }  
  
  static of(x) {  
    return new Container(x);  
  }  
}
```

He aquí nuestro primer contenedor. Lo hemos llamado, inteligentemente, `Container` [*Contenedor*]. Usaremos `Container.of` como un constructor que nos ahorra tener que escribir la terrible palabra clave `new` por todas partes. Hay algo más en la función `of` de lo que se ve a simple vista, pero por ahora, piensa en ella como la manera correcta de colocar valores en nuestro contenedor.

Examinemos nuestra flamante caja nueva...

```
Container.of(3);  
// Container(3)  
  
Container.of('hotdogs');  
// Container("hotdogs")  
  
Container.of(Container.of({ name: 'yoda' }));  
// Container(Container({ name: 'yoda' }))
```

Si estás usando node, verás `{ $value: x }` aunque tengamos un `Container(x)`. Chrome mostrará el tipo correctamente, pero no importa; mientras entendamos como es un `Container` iremos bien. En algunos entornos se puede sobrescribir el método `inspect` si se quiere, pero no seremos tan minuciosos. Para este libro, escribiremos la salida conceptual como si hubiéramos sobrescrito `inspect`, ya que por razones pedagógicas y también estéticas es mucho más instructivo que `{ $value: x }`.

Aclaremos algunas cosas antes de continuar:

- `Container` es un objeto con una sola propiedad. Aunque no están limitados, muchos contenedores solo contienen una cosa. Arbitrariamente, hemos llamado `$value` [*valor*] a su propiedad.
- El valor no puede ser de un tipo específico o de lo contrario difícilmente nuestro `Container` haría honor a su nombre.
- Una vez los datos entran en nuestro `Container` se quedan ahí. *Podríamos* sacarlos usando `.$value`, pero eso anularía su propósito.

Las razones por las cuales estamos haciendo esto quedarán claras como un tarro de cristal, pero por el momento, tendréis que ser pacientes.

Mi Primer Funtor

Una vez que nuestro valor, sea cual sea, está en el contenedor, necesitamos una forma de ejecutar funciones sobre él.

```
// (a -> b) -> Container a -> Container b
Container.prototype.map = function (f) {
  return Container.of(f(this.$value));
};
```

Es como el famoso `map` de Array, excepto que tenemos un `Container a` en lugar de `[a]`. Y esencialmente funciona de la misma manera:

```
Container.of(2).map(two => two + 2);
// Container(4)

Container.of('flamethrowers').map(s => s.toUpperCase());
// Container('FLAMETHROWERS')

Container.of('bombs').map(append(' away')).map(prop('length'));
// Container(10)
```

Podemos trabajar con nuestro valor sin salir del contenedor. Esto es algo extraordinario. Nuestro valor en `Container` es entregado a la función `map` para que podamos manipularlo y después es devuelto a su contenedor para que esté a salvo. Como resultado de no salir nunca de `Container`, podemos seguir usando `map`, ejecutando funciones a nuestro antojo. Incluso podemos cambiar el tipo sobre la marcha como se demuestra en el último de los tres ejemplos.

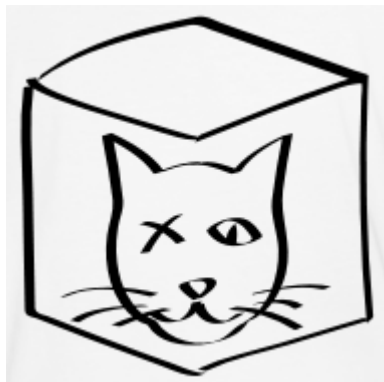
Espera un minuto, si seguimos llamando a `map`, ¡parece ser una especie de composición! ¿Qué magia matemática es esta? Bien amigos, acabamos de descubrir los *Funtores*.

Un Funtor es un tipo que implementa `map` y obedece a algunas leyes

Sí, *Funtor* es simplemente una interfaz con un contrato. También podríamos haberlo llamado *Mapeable*, pero entonces, ¿dónde estaría la diversión? Los Funtores provienen de la teoría de categorías y veremos las matemáticas en detalle hacia el final del capítulo, pero por ahora, trabajemos en la intuición y los usos prácticos de esta interfaz de nombre extraño.

¿Qué razón podríamos tener para embotellar un valor y utilizar `map` para llegar a él? La respuesta se revela por si sola si escogemos mejor la pregunta: ¿Qué ganamos al pedir a nuestro contenedor que aplique funciones en nuestro lugar? Pues la abstracción de la aplicación de funciones. Cuando aplicamos una función mediante `map`, le pedimos al tipo del contenedor que la ejecute de nuestra parte. Este es, de hecho, un concepto muy poderoso.

El Maybe de Schrödinger



`Container` es bastante aburrido. De hecho, se le suele llamar `Identity` [*Identidad*] y tiene más o menos el mismo impacto que nuestra función `id` (de nuevo hay una conexión matemática que veremos en el momento indicado). Sin embargo, hay otros funtores, o sea otros tipos que hacen de contenedor y que tienen su propia función `map`, que pueden proporcionar útiles comportamientos cuando mapean. Definamos uno ahora:

En el [Apéndice B](#) se ofrece una implementación completa

```
class Maybe {
  static of(x) {
    return new Maybe(x);
  }

  get isNothing() {
    return this.$value === null || this.$value === undefined;
  }

  constructor(x) {
    this.$value = x;
  }

  map(fn) {
    return this.isNothing ? this : Maybe.of(fn(this.$value));
  }

  inspect() {
    return this.isNothing ? 'Nothing' : `Just(${inspect(this.$value)})`;
  }
}
```

Ahora, `Maybe` se parece mucho a `Container` pero con un pequeño cambio: comprueba si tiene un valor antes de llamar a la función proporcionada. Esto tiene el efecto de evitar esos molestos nulls cuando aplicamos `map` (Ten en cuenta que esta implementación está simplificada por motivos didácticos).

```
Maybe.of('Malkovich Malkovich').map(match(/a/ig));  
// Just(True)  
  
Maybe.of(null).map(match(/a/ig));  
// Nothing  
  
Maybe.of({ name: 'Boris' }).map(prop('age')).map(add(10));  
// Nothing  
  
Maybe.of({ name: 'Dinah', age: 14 }).map(prop('age')).map(add(10));  
// Just(24)
```

Fíjate en que nuestra app no explota con errores cuando mapeamos funciones sobre nuestros valores nulos. Esto es porque `Maybe` se encargará de comprobar que el valor exista cada vez que le aplique una función.

Esta sintaxis con el punto está bien y es funcional, pero por las razones mencionadas en la Parte 1, nos gustaría mantener nuestro estilo *pointfree*. Así, `map` está totalmente equipada para delegar en cualquier funtor que reciba:

```
// map :: Functor f => (a -> b) -> f a -> f b  
const map = curry((f, anyFunctor) => anyFunctor.map(f));
```

Esto es perfecto, ya que podemos continuar con la composición como de costumbre y `map` funcionará según lo esperado. Este también es el caso con el `map` de ramda. Usaremos la sintaxis con el punto cuando sea instructiva y la versión *pointfree* cuando sea conveniente. ¿Te has dado cuenta? He introducido disimuladamente una notación adicional en nuestra firma de tipos. `Functor f =>` nos dice que `f` debe ser un Funtor. Aunque no es algo que sea complicado, sentí que debía mencionarlo.

Casos de Uso

En la vida real, típicamente vemos a `Maybe` usado en funciones que podrían fallar y no devolver un resultado.

```
// safeHead :: [a] -> Maybe(a)
const safeHead = xs => Maybe.of(xs[0]);

// streetName :: Object -> Maybe String
const streetName = compose(map(prop('street')), safeHead, prop('addresses'));

streetName({ addresses: [] });
// Nothing

streetName({ addresses: [{ street: 'Shady Ln.', number: 4201 }] });
// Just('Shady Ln.')
```

`safeHead` es como nuestra `head` normal, pero con seguridad de tipos añadida. Una cosa curiosa ocurre cuando `Maybe` es introducido en nuestro código; somos forzados a manejar esos escurridizos valores `null`. La función `safeHead` es honesta y directa sobre su posible fallo; realmente no hay nada de lo que avergonzarse, así que nos devuelve un `Maybe` para informarnos sobre este asunto. Sin embargo, estamos más que solo informados, porque nos vemos obligados a aplicar `map` para llegar al valor que queremos, ya que está escondido dentro del objeto `Maybe`. Esencialmente, se trata de una comprobación por parte de la función `safeHead` para ver si tenemos un `null`. Ahora podemos dormir mejor por la noche sabiendo que un valor `null` no levantará su fea y decapitada cabeza cuando menos lo esperemos. Apis como esta harán que una endeble aplicación de papel y chinchetas pase a ser de madera y clavos. Garantizarán un software más seguro.

Para indicar un fallo a veces una función puede devolver `Nothing` explícitamente. Por ejemplo:

```
// withdraw :: Number -> Account -> Maybe(Account)
const withdraw = curry((amount, { balance }) =>
  Maybe.of(balance >= amount ? { balance: balance - amount } : null));

// Esta función es hipotética, no está implementada aquí.. ni en ningún otro s:
// updateLedger :: Account -> Account
const updateLedger = account => account;

// remainingBalance :: Account -> String
const remainingBalance = ({ balance }) => `Your balance is ${balance}`;

// finishTransaction :: Account -> String
const finishTransaction = compose(remainingBalance, updateLedger);

// getTwenty :: Account -> Maybe(String)
const getTwenty = compose(map(finishTransaction), withdraw(20));

getTwenty({ balance: 200.00 });
// Just('Your balance is $180')

getTwenty({ balance: 10.00 });
// Nothing
```

`withdraw` nos hará saber cuando estamos cortos de dinero devolviéndonos un `Nothing`. Esta función también nos comunica su veleidad y no nos deja otra opción que aplicar `map` a todo lo que le sigue. La diferencia es que aquí el `null` es intencionado. En lugar de un `Just('..')` obtenemos el `Nothing` de regreso para señalar el error y luego nuestra aplicación detiene la ejecución de manera efectiva. Es importante destacar esto: si la función `withdraw` falla, entonces `map` cortará el resto de nuestra computación, ya que nunca ejecutará las funciones mapeadas, concretamente `finishTransaction`. Este es precisamente el comportamiento pretendido, dado que preferimos no actualizar nuestra contabilidad o mostrar un nuevo saldo si no hemos retirado fondos con éxito.

Liberando el Valor

Una cosa que a menudo la gente pasa por alto es que siempre habrá un final de línea; alguna función que provoca un efecto como enviar un JSON, imprimir algo en la pantalla o alterar algún archivo en el sistema, o lo que sea. No podemos entregar la salida con `return`, debemos ejecutar alguna u otra función para sacarla al mundo exterior. Podemos expresarlo como un koan del Budismo Zen: "Si un programa no tiene ningún efecto observable, ¿se ejecuta siquiera?". ¿Se ejecuta correctamente para su propia satisfacción? Sospecho que simplemente quema algunos ciclos y vuelve a dormir...

El trabajo de nuestra aplicación es recuperar, transformar y cargar con esos datos hasta el momento de la despedida y la función que lo hace ha de poder ser mapeada, para que así el valor no necesite salir del cálido vientre de su contenedor. De hecho, un error típico es tratar de sacar el valor de nuestro `Maybe` de una manera u otra como si el posible valor de su interior se fuese a materializar de repente y todo fuese a ser perdonado. Debemos entender que puede haber bifurcaciones en el código donde nuestro valor pueda no sobrevivir y alcanzar su destino. Nuestro código, al igual que el gato de Schrödinger, está en dos estados a la vez y debe mantenerse así hasta la función final. Esto da a nuestro código un flujo lineal a pesar de las ramificaciones lógicas.

Existe, sin embargo, una vía de escape. Si preferimos devolver un valor personalizado y proseguir, podemos utilizar un pequeño ayudante llamado

`maybe` .

```
// maybe :: b -> (a -> b) -> Maybe a -> b
const maybe = curry((v, f, m) => {
  if (m.isNothing) {
    return v;
  }

  return f(m.$value);
});

// getTwenty :: Account -> String
const getTwenty = compose(maybe('You\'re broke!', finishTransaction), withdraw)

getTwenty({ balance: 200.00 });
// 'Your balance is $180.00'

getTwenty({ balance: 10.00 });
// 'You\'re broke!'
```

Ahora devolveremos un valor estático (del mismo tipo que devuelve `finishTransaction`) o seguiremos para finalizar alegremente la transacción sin `Maybe` . Con `maybe` , estamos ante el equivalente de una sentencia `if/else` mientras que con `map` , el análogo imperativo sería: `if (x !== null) { return f(x) }` .

Inicialmente, la introducción de `Maybe` puede causar cierta incomodidad. Los usuarios de Swift y Scala sabrán a qué me refiero, ya que está incorporado en las librerías del núcleo bajo la apariencia de `option(al)` . Cuando se nos empuja continuamente a comprobar la presencia de `null` (y hay veces que sabemos con absoluta certeza que el valor existe), la mayoría no podemos evitar que nos parezca un poco laborioso. Sin embargo, con el tiempo, se volverá algo natural y probablemente apreciarás la seguridad. Al fin y al cabo, la mayoría de las veces evitará chapuzas y nos mantendrá a salvo.

Escribir software inseguro es como asegurarse de decorar cada huevo con pinturas pastel antes de lanzarlo al tráfico; como construir una residencia de ancianos con materiales rechazados por los tres cerditos. Nos vendrá bien poner algo de seguridad en nuestras funciones y `Maybe` nos ayuda justamente a hacer eso.

Sería negligente por mi parte si no mencionase que la implementación "real" dividirá a `Maybe` en dos tipos: uno para algo y otro para nada. Esto nos permite obedecer a la parametricidad de `map` para que valores como `null` y `undefined` puedan seguir siendo mapeados y que la calificación universal del valor como funtor siga siendo respetada. A menudo verás tipos como `Some(x) / None` o `Just(x) / Nothing` en lugar de un `Maybe` que comprueba que su valor no sea `null`.

Manejo de Errores Puro



Puede resultar chocante, pero `throw/catch` no es muy puro. Cuando un error es lanzado, en lugar de devolver un valor de salida, ¡hacemos sonar las alarmas! La función ataca, lanzando miles de 0 y 1 como escudos y dardos en una batalla eléctrica contra nuestra intrusa entrada. Con nuestro nuevo amigo `Either`, podemos hacer algo mejor que declarar la guerra a la entrada, podemos responder con un educado mensaje. Echemos un vistazo:

En el [Apéndice B](#) se proporciona una implementación completa

```
class Either {  
  static of(x) {  
    return new Right(x);  
  }  
  
  constructor(x) {  
    this.$value = x;  
  }  
}  
  
class Left extends Either {  
  map(f) {  
    return this;  
  }  
  
  inspect() {  
    return `Left(${inspect(this.$value)})`;  
  }  
}  
  
class Right extends Either {  
  map(f) {  
    return Either.of(f(this.$value));  
  }  
  
  inspect() {  
    return `Right(${inspect(this.$value)})`;  
  }  
}  
  
const left = x => new Left(x);
```

`Left` y `Right` son dos subclases de un tipo abstracto llamado `Either`. Me he saltado la ceremonia de crear la superclase `Either` ya que no la usaremos nunca, pero es bueno saber de su existencia. Ahora bien, no hay nada nuevo aquí aparte de los dos tipos. Veamos como actúan:

```

Either.of('rain').map(str => `b${str}`);
// Right('brain')

left('rain').map(str => `It's gonna ${str}, better bring your umbrella!`);
// Left('rain')

Either.of({ host: 'localhost', port: 80 }).map(prop('host'));
// Right('localhost')

left('rolls eyes...').map(prop('host'));
// Left('rolls eyes...')

```

`Left` es como un adolescente e ignora nuestra solicitud de aplicarse `map` a sí mismo. `Right` funcionará igual que `Container` (también conocido como Identidad). El poder viene de la capacidad de incrustar un mensaje de error dentro de `Left`.

Supón que tenemos una función que podría no tener éxito. Quizás una función que calcule la edad a partir de una fecha de nacimiento. Podríamos utilizar `Nothing` para avisar del fracaso y bifurcar nuestro programa, sin embargo, eso no nos dice mucho. Quizás nos gustaría saber por qué ha fallado. Escribámoslo usando `Either`.

```

const moment = require('moment');

// getAge :: Date -> User -> Either(String, Number)
const getAge = curry((now, user) => {
  const birthDate = moment(user.birthDate, 'YYYY-MM-DD');

  return birthDate.isValid()
    ? Either.of(now.diff(birthDate, 'years'))
    : left('Birth date could not be parsed');
});

getAge(moment(), { birthDate: '2005-12-12' });
// Right(9)

getAge(moment(), { birthDate: 'July 4, 2001' });
// Left('Birth date could not be parsed')

```

Ahora bien, al igual que con `Nothing`, cuando devolvemos un `Left` estamos cortocircuitando nuestra aplicación. La diferencia es que ahora tenemos una pista de por qué nuestro programa ha descarrilado. Algo a tener en cuenta es que devolvemos `Either(String, Number)`, que contiene un `String` como su valor izquierdo (`Left`) y un `Number` como su valor derecho (`Right`). Esta firma de tipos es un poco informal, ya que no hemos dedicado tiempo suficiente para

definir una superclase `Either` real, sin embargo, aprendemos mucho del tipo. Nos informa de que estamos obteniendo o bien un mensaje de error o bien la edad.

```
// fortune :: Number -> String
const fortune = compose(concat('If you survive, you will be '), toString, add(:

// zoltar :: User -> Either(String, _)
const zoltar = compose(map(console.log), map(fortune), getAge(moment()));

zoltar({ birthDate: '2005-12-12' });
// 'If you survive, you will be 10'
// Right(undefined)

zoltar({ birthDate: 'balloons!' });
// Left('Birth date could not be parsed')
```

Cuando `birthdate` es válido, el programa muestra su mística predicción en la pantalla para que la contemplemos. De lo contrario, se nos entrega un `Left` con el mensaje de error tan claro como el día, aunque todavía escondido en su contenedor. Esto actúa igual que si hubiésemos lanzado un error, pero de una manera más tranquila y suave en lugar de perder los estribos y gritar como un niño cuando algo sale mal.

En este ejemplo, estamos haciendo una bifurcación lógica en nuestro flujo de control dependiendo de la validez de la fecha de cumpleaños, leyéndose en una sola línea de derecha a izquierda en vez de escalar a través de las llaves de una declaración condicional. Normalmente, moveríamos el `console.log` fuera de la función `zoltar` y aplicaríamos `map` al momento de llamarla, pero es útil ver como la rama del `Right` diverge. Utilizamos `_` en la firma de la rama derecha para indicar que es un valor que debe ser ignorado (En algunos navegadores debes usar `console.log.bind(console)` para usarlo como función de primera clase).

Me gustaría aprovechar esta oportunidad para resaltar algo que podrías haber pasado por alto: `fortune`, a pesar de su uso con `Either` en este ejemplo, es completamente ignorante de cualquier funtor que tenga a su alrededor. Este era también el caso para `finishTransaction` en el ejemplo anterior. En el momento de la llamada, una función puede ser rodeada por `map`, lo que la transforma, en términos informales, de no ser un funtor a serlo. Llamamos a este proceso *levantamiento* (*lifting*). Es mejor que las funciones trabajen con tipos de datos normales que con tipos contenedor, para luego ser levantadas al contenedor apropiado según se considere necesario. Esto conduce a funciones más simples y reutilizables que pueden ser alteradas a demanda para trabajar con cualquier funtor.

`Either` es genial para errores casuales como validaciones y también para casos más graves como archivos faltantes o sockets rotos. Prueba a reemplazar por `Either` alguno de los ejemplos de `Maybe` para obtener un mejor entendimiento.

Por otro lado, no puedo dejar de pensar que le he hecho un flaco favor a `Either` presentándolo como un simple contenedor de mensajes de error. `Either` captura la disyunción lógica (`||`) en un tipo. También codifica la idea de *Coproducto* de la teoría de categorías, que no se verá en este libro, aunque vale la pena leer sobre ello, ya que contiene varias propiedades a explotar. `Either` es la suma de tipos canónica (o unión disjunta de conjuntos) porque la cantidad total de posibles habitantes es la suma de los dos tipos contenidos (sé que esto suena a magia por lo que acá les entrego un [gran artículo](#)). Hay muchas cosas que `Either` puede ser, pero como funtor, se usa por su manejo de errores.

Igual que con `Maybe`, tenemos a la pequeña `either`, que se comporta de manera similar, pero toma dos funciones en lugar de una y un valor estático. Cada función debe devolver el mismo tipo.

```
// either :: (a -> c) -> (b -> c) -> Either a b -> c
const either = curry((f, g, e) => {
  let result;

  switch (e.constructor) {
    case Left:
      result = f(e.$value);
      break;

    case Right:
      result = g(e.$value);
      break;

    // No Default
  }

  return result;
});

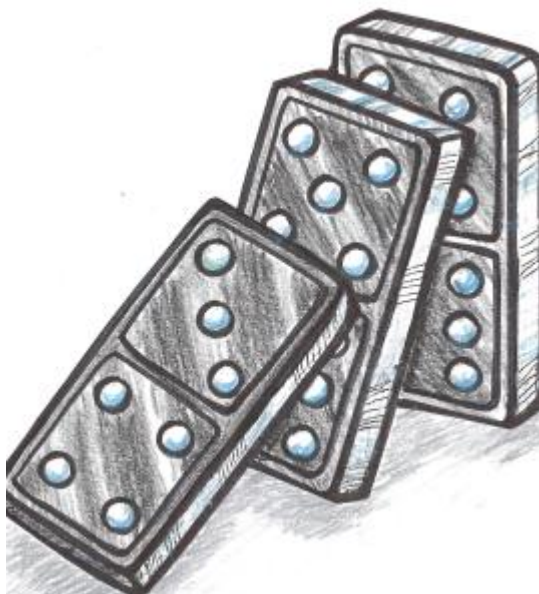
// zoltar :: User -> _
const zoltar = compose(console.log, either(id, fortune), getAge(moment()));

zoltar({ birthDate: '2005-12-12' });
// 'If you survive, you will be 10'
// undefined

zoltar({ birthDate: 'balloons!' });
// 'Birth date could not be parsed'
// undefined
```

Por fin, un uso para esa misteriosa función `id`. Simplemente repite como un loro el valor de `Left` para pasar el mensaje de error a `console.log`. Hemos hecho que nuestra app de videncia sea más robusta imponiendo el manejo de errores desde dentro de `getAge`. O bien abofeteamos al usuario con la dura verdad o bien continuamos con nuestro proceso. Y con esto, podemos pasar a otro tipo de funtor completamente diferente.

El Viejo McDonald Tenía Efectos...



En nuestro capítulo sobre la pureza vimos un peculiar ejemplo de una función pura. Esta función contenía un efecto secundario, pero la convertimos en pura envolviendo su acción en otra función. He aquí otro ejemplo de esto:

```
// getFromStorage :: String -> ( _ -> String )
const getFromStorage = key => () => localStorage[key];
```

Si no hubiéramos rodeado sus entrañas en otra función, `getFromStorage` variaría su salida dependiendo de las circunstancias externas. Con el resistente envoltorio en su sitio, siempre obtendremos la misma salida para cada entrada: una función que, al ser llamada, recuperará un elemento concreto de `localStorage`. Y así, sin más (puede que con algún Ave María) limpiamos nuestra conciencia y todo está perdonado.

Excepto que esto no es particularmente útil. Igual que con una figura de coleccionista en su empaquetado original, no podemos jugar con ella. Si tan solo hubiese una manera de alcanzar el interior del contenedor y tomar su contenido... He aquí `io`.

```
class IO {  
  static of(x) {  
    return new IO(() => x);  
  }  
  
  constructor(fn) {  
    this.$value = fn;  
  }  
  
  map(fn) {  
    return new IO(compose(fn, this.$value));  
  }  
  
  inspect() {  
    return `IO(${inspect(this.$value)})`;  
  }  
}
```

`IO` se diferencia de los anteriores funtores en que su valor es siempre una función. Sin embargo, no pensemos en su `$value` como una función - esto es un detalle de implementación y mejor lo ignoramos. Lo que sucede es exactamente lo que vimos con el ejemplo de `getFromStorage : IO` retrasa la acción impura capturándola en una función envolvente. Como tal, pensamos en `IO` como que contiene el valor devuelto por la acción envuelta y no el envoltorio en sí mismo. Esto es evidente en la función `of` : Tenemos un `IO(x)` , el `IO(() => x)` únicamente es necesario par evitar su evaluación. Nótese que, para simplificar la lectura, mostraremos como resultado el hipotético valor contenido en el `IO` ; sin embargo, en la práctica, ¡no se puede saber cuál es este valor hasta que no se hayan desencadenado los efectos!

Veámoslo en uso:

```
// ioWindow :: IO Window
const ioWindow = new IO(() => window);

ioWindow.map(win => win.innerWidth);
// IO(1430)

ioWindow
  .map(prop('location'))
  .map(prop('href'))
  .map(split('/'));
// IO(['http:', '', 'localhost:8000', 'blog', 'posts'])

// $ :: String -> IO [DOM]
const $ = selector => new IO(() => document.querySelectorAll(selector));

$('#myDiv').map(head).map(div => div.innerHTML);
// IO('I am some inner html')
```

Aquí, `ioWindow` es un `IO` al que podemos aplicar `map` directamente, mientras que `$` es una función que devuelve un `IO` al ser llamada. He escrito los valores de retorno *conceptuales* para expresar mejor el `IO`, aunque, en realidad, siempre será `{ $value: [Function] }`. Cuando aplicamos `map` sobre nuestro `IO`, ponemos esta función al final de una composición que, a su vez, se convierte en el nuevo `$value` y así sucesivamente. Nuestras funciones mapeadas no se ejecutan, sino que quedan adheridas al final del cálculo que estamos construyendo, función a función, como si cuidadosamente colocáramos fichas de dominó que no nos atrevemos a volcar. El resultado recuerda al patrón comando del Gang of Four o a una cola.

Tómate un momento para trabajar tu intuición sobre los funtores. Si miramos más allá de los detalles de implementación, deberíamos sentirnos como en casa mapeando cualquier contenedor, sin importar sus peculiaridades o idiosincrasias. Tenemos que agradecer este poder pseudopsíquico a las leyes de funtores, las cuales exploraremos hacia el final del capítulo. En cualquier caso, por fin podemos jugar con valores impuros sin sacrificar nuestra preciosa pureza.

Ahora bien, hemos enjaulado a la bestia, pero seguimos teniendo que liberarla en algún momento. Aplicar `map` sobre nuestro `IO` ha creado una poderosa computación impura y ejecutarla seguramente perturbará la paz. Entonces, ¿dónde y cuándo podemos apretar el gatillo? ¿Es posible ejecutar nuestro `IO` y seguir vistiendo de blanco en nuestra boda? La respuesta es que sí, si le damos la responsabilidad al código que lo llama. Nuestro código puro, a pesar de las nefastas conspiraciones e intrigas, mantiene su inocencia y es quien lo ejecute quien cargará con la responsabilidad de realmente ejecutar los efectos. Veamos un ejemplo para concretar esto:

```
// url :: IO String
const url = new IO(()) => window.location.href);

// toPairs :: String -> [[String]]
const toPairs = compose(map(split('=')), split('&'));

// params :: String -> [[String]]
const params = compose(toPairs, last, split('?'));

// findParam :: String -> IO Maybe [String]
const findParam = key => map(compose(Maybe.of, find(compose(eq(key), head))), pa

// -- Código impuro que hace la llamada-----

// ejecútala llamando a $value()!
findParam('searchTerm').$value();
// Just(['searchTerm', 'wafflehouse'])
```

Al envolver a `url` en un `IO` y pasar la pelota a quien la llama, nuestra librería mantiene sus manos limpias. También te habrás dado cuenta de que hemos apilado nuestros contenedores; es perfectamente razonable tener un `IO(Maybe([x]))`, que tiene tres funtores de profundidad (`Array` es claramente un contenedor mapeable) y es excepcionalmente expresivo.

Hay algo que ha estado molestándome y que deberíamos rectificar inmediatamente: el `$value` de `IO` realmente no es un valor lo que contiene y tampoco es una propiedad privada. Es el pasador de la granada y está destinado, de la más pública de las maneras, a ser tirado por quien lo llame. Cambiemos el nombre de esta propiedad a `unsafePerformIO` para recordar su volatilidad a nuestros usuarios.

```
class IO {
  constructor(io) {
    this.unsafePerformIO = io;
  }

  map(fn) {
    return new IO(compose(fn, this.unsafePerformIO));
  }
}
```

Ya está, mucho mejor. Ahora nuestro código de llamada se convierte en `findParam('searchTerm').unsafePerformIO()`, que es claro como el agua para quien use (y lea) nuestra aplicación.

`io` será un fiel compañero, ayudándonos a domar esas asilvestradas acciones impuras. A continuación, veremos un tipo similar en espíritu, pero que tiene un caso de uso drásticamente distinto.

Tareas Asíncronas

Las funciones callbacks son la estrecha escalera de caracol hacia el infierno. Son el control de flujo diseñado por M.C. Escher. Con cada callback anidada que estrujamos entre la jungla de llaves y paréntesis, más se parece a jugar al limbo en una mazmorra (¿Cuán bajo podemos llegar?!) Me dan escalofríos claustrofóbicos solo de pensar en ellas. No hay de que preocuparse, tenemos una forma mucho mejor de tratar con código asíncrono y empieza por "F".

Los aspectos internos son demasiado complicados como para derramarlos por toda la página así que utilizaremos `Data.Task` (antes conocido como `Data.Future`) de la fantástica [Folktable](#) de Quildreen Motta. Contempla algunos ejemplos de uso:

```
// -- Ejemplo con readFile de Node -----

const fs = require('fs');

// readFile :: String -> Task Error String
const readFile = filename => new Task((reject, result) => {
  fs.readFile(filename, (err, data) => (err ? reject(err) : result(data)));
});

readFile('metamorphosis').map(split('\n')).map(head);
// Task('One morning, as Gregor Samsa was waking up from anxious dreams, he discovered
// in bed he had been changed into a monstrous verminous bug.')

// -- Ejemplo con getJSON de jQuery -----

// getJSON :: String -> {} -> Task Error JSON
const getJSON = curry((url, params) => new Task((reject, result) => {
  $.getJSON(url, params, result).fail(reject);
}));

getJSON('/video', { id: 10 }).map(prop('title'));
// Task('Family Matters ep 15')

// -- Contexto Mínimo por Defecto -----

// También podemos poner adentro valores normales, no futurísticos
Task.of(3).map(three => three + 1);
// Task(4)
```

Las funciones que estoy llamando `reject` y `result` son nuestras callback de error o éxito respectivamente. Como puedes ver, simplemente `map` eamos sobre `Task` para trabajar sobre el valor futuro como si estuviera ahí mismo a nuestro alcance. A estas alturas, `map` ya debería sernos sobradamente familiar.

Si conoces las promesas, puede que reconozcas a la función `map` como `then`, con `Task` haciendo el papel de nuestra promesa. No te preocupes si no conoces las promesas, de todos modos no las usaremos dado que no son puras, pero la analogía igualmente se mantiene.

Al igual que `IO`, `Task` esperará pacientemente a que le demos luz verde antes de ejecutarse. De hecho, dado que espera nuestra orden, `IO` está efectivamente subsumido en `Task` para todas las cosas asíncronas; `readFile` y `getJSON` no requieren un contenedor `IO` adicional para ser puros. Es más, `Task` funciona de manera similar cuando `map` eamos sobre él: estamos

colocando instrucciones para el futuro igual que si colocáramos una tabla de tareas en una cápsula del tiempo; un acto de sofisticada procrastinación tecnológica.

Para ejecutar nuestra tarea `[Task]`, debemos llamar al método `fork`. Esto funciona como `unsafePerformIO`, pero como su nombre indica, bifurcará nuestro proceso y la evaluación continuará sin bloquear nuestro hilo de ejecución. Esto puede ser implementado de numerosas maneras con hilos y demás, pero aquí actuará como lo haría una llamada asíncrona normal y la gran rueda del bucle de eventos de JavaScript seguirá girando. Echemos un vistazo a `fork`:

```
// -- Aplicación pura -----
// blogPage :: Posts -> HTML
const blogPage = Handlebars.compile(blogTemplate);

// renderPage :: Posts -> HTML
const renderPage = compose(blogPage, sortBy(prop('date')));

// blog :: Params -> Task Error HTML
const blog = compose(map(renderPage), getJSON('/posts'));

// -- Código impuro que hace la llamada -----
blog({}).fork(
  error => $('#error').html(error.message),
  page => $('#main').html(page),
);

$('#spinner').show();
```

Al llamar a `fork`, `Task` se apresura a encontrar algunos posts y renderizar la página. Mientras tanto, mostramos un spinner, ya que `fork` no va a esperar a la respuesta. Finalmente, mostraremos el mensaje de error o renderizaremos la página en la pantalla dependiendo de si la llamada a `getJSON` es exitosa o no.

Tómate un momento para considerar lo lineal que es nuestro flujo de control aquí. Solo leemos de abajo a arriba y de derecha a izquierda, sin importar que realmente el programa vaya a hacer algunos saltos durante su ejecución. Esto hace que leer y razonar sobre nuestra aplicación sea más sencillo que tener que rebotar entre callbacks y bloques de manejo de errores.

¡Santo cielo, mira eso, `Task` también se ha tragado a `Either`! Lo tiene que hacer para manejar fallos futuros, ya que nuestro flujo de control normal no aplica en el mundo asíncrono. Todo esto está muy bien, puesto que proporciona un manejo de errores suficiente y puro listo para ser utilizado.

Incluso con `Task`, nuestros funtores `IO` y `Either` no se quedan sin trabajo. Acompáñame en un rápido ejemplo que se inclina hacia el lado más complejo e hipotético, pero que es útil para fines ilustrativos:

```
// Postgres.connect :: Url -> IO DbConnection
// runQuery :: DbConnection -> ResultSet
// readFile :: String -> Task Error String

// -- Aplicación pura -----

// dbUrl :: Config -> Either Error Url
const dbUrl = ({ uname, pass, host, db }) => {
  if (uname && pass && host && db) {
    return Either.of(`db:pg://${uname}:${pass}@${host}5432/${db}`);
  }

  return left(Error('Invalid config!'));
};

// connectDb :: Config -> Either Error (IO DbConnection)
const connectDb = compose(map(Postgres.connect), dbUrl);

// getConfig :: Filename -> Task Error (Either Error (IO DbConnection))
const getConfig = compose(map(compose(connectDb, JSON.parse)), readFile);

// -- Código impuro que hace la llamada -----

getConfig('db.json').fork(
  logErr('couldn\'t read file'),
  either(console.log, map(runQuery)),
);
```

En este ejemplo, todavía hacemos uso de `Either` e `IO` desde la rama exitosa de `readFile`. `Task` se encarga de lo impuro de leer un archivo de manera asíncrona, pero seguimos teniendo que ocuparnos de validar la configuración con `Either` y lidiar con la conexión a la base de datos con `IO`. Así que ya ves, todavía tenemos trabajo para todo tipo de cosas síncronas.

Podría continuar, pero eso es todo. Tan simple como `map`.

En la práctica, es probable que tengas múltiples tareas asíncronas en un solo flujo de trabajo y aún no hemos adquirido todas las API de contenedores como para afrontar este escenario. No te preocupes, pronto veremos cosas sobre mónadas y demás, pero primero debemos examinar las matemáticas que hacen que todo esto sea posible.

Un Poco de Teoría

Como ya hemos dicho, los funtores proceden de la teoría de categorías y satisfacen una cuantas leyes. Exploremos primero estas útiles propiedades.

```
// identidad
map(id) === id;

// composición
compose(map(f), map(g)) === map(compose(f, g));
```

La ley de la *identidad* es simple pero importante. Estas leyes son trozos de código ejecutables, por lo que podemos probarlas en nuestros propios funtores para validar su legitimidad.

```
const idLaw1 = map(id);
const idLaw2 = id;

idLaw1(Container.of(2)); // Container(2)
idLaw2(Container.of(2)); // Container(2)
```

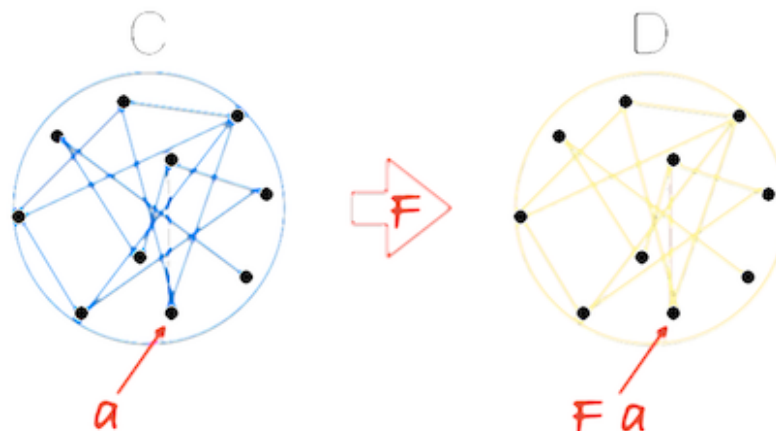
Como ves son iguales. A continuación veamos la composición.

```
const compLaw1 = compose(map(append(' world')), map(append(' cruel')));
const compLaw2 = map(compose(append(' world'), append(' cruel')));

compLaw1(Container.of('Goodbye')); // Container('Goodbye cruel world')
compLaw2(Container.of('Goodbye')); // Container('Goodbye cruel world')
```

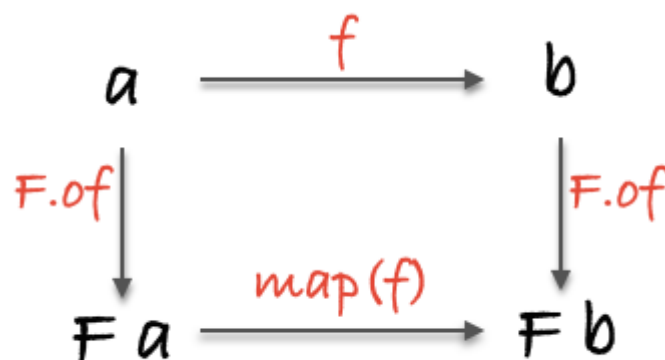
En teoría de categorías los funtores toman los objetos y morfismos de una categoría y los trasladan a otra categoría distinta. Por definición, esta nueva categoría debe tener una identidad y la capacidad de componer morfismos, pero no tenemos que comprobarlo porque las leyes antes mencionadas aseguran que esto ocurre.

Tal vez nuestra definición de categoría sea todavía algo confusa. Puedes pensar en una categoría como en una red de objetos conectados entre sí mediante morfismos. Así que un funtor mapearía una categoría a otra sin romper esta red. Si un objeto `a` está en nuestra categoría de origen `c`, cuando lo mapeamos a la categoría `d` con el funtor `F`, nos referimos a ese objeto como `F a`. Tal vez sea mejor ver un diagrama:



Por ejemplo, `Maybe` mapea nuestra categoría de tipos y funciones a una categoría donde cada objeto puede no existir y cada morfismo tiene una comprobación de `null`. Logramos esto en el código rodeando a cada función con `map` y a cada tipo con nuestro funtor. Sabemos que cada uno de nuestros tipos normales y cada una de nuestras funciones seguirán pudiéndose componer en este nuevo mundo. Técnicamente, cada funtor en nuestro código mapea a una subcategoría de tipos y funciones que hace que todos los funtores sean de un tipo en particular llamado endofuntor, pero para nuestros propósitos consideraremos que son de otra categoría.

Podemos visualizar el mapeo de morfismos y sus correspondientes objetos mediante este diagrama:



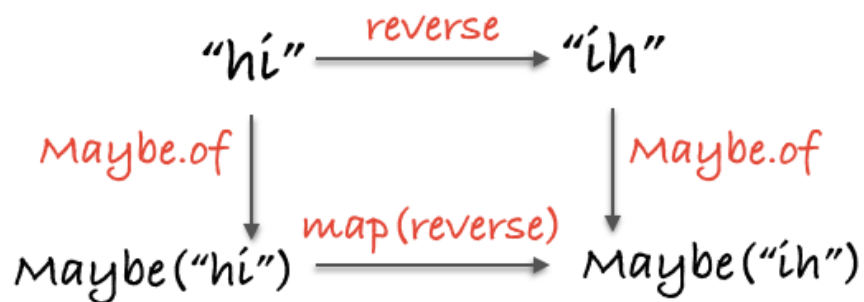
Además de visualizar el morfismo mapeado de una categoría a otra bajo el funtor `F`, vemos que el diagrama conmuta, es decir, si se siguen las flechas cada ruta conduce al mismo resultado. Las distintas rutas significan diferentes comportamientos, pero siempre acabamos en el mismo tipo. Este formalismo nos permite basarnos en principios a la hora de razonar sobre nuestro código; podemos aplicar fórmulas audazmente sin tener que interpretar y examinar cada escenario individualmente. Veamos esto en un ejemplo concreto:

```
// topRoute :: String -> Maybe String
const topRoute = compose(Maybe.of, reverse);

// bottomRoute :: String -> Maybe String
const bottomRoute = compose(map(reverse), Maybe.of);

topRoute('hi'); // Just('ih')
bottomRoute('hi'); // Just('ih')
```

O visualmente:



Instantáneamente, podemos ver y refactorizar código cuando este está basado en las propiedades que todos los funtores tienen.

Los funtores pueden apilarse:

```
const nested = Task.of([Either.of('pillows'), left('no sleep for you')]);

map(map(map(toUpperCase)), nested);
// Task([Right('PILLOWS'), Left('no sleep for you')])
```

Lo que aquí tenemos con `nested` es un futuro array de elementos que pueden ser errores. Aplicamos `map` para pelar cada capa y ejecutar nuestra función en los elementos. No vemos ninguna callback, if/else o bucle for; solo contenido explícito. Sin embargo, tenemos que hacer `map(map(map(f)))`. En lugar de eso podemos componer funtores. Me has oído bien:

```

class Compose {
  constructor(fgx) {
    this.getCompose = fgx;
  }

  static of(fgx) {
    return new Compose(fgx);
  }

  map(fn) {
    return new Compose(map(map(fn), this.getCompose));
  }
}

const tmd = Task.of(Maybe.of('Rock over London'));

const ctmd = Compose.of(tmd);

const ctmd2 = map(append(', rock on, Chicago'), ctmd);
// Compose(Task(Just('Rock over London, rock on, Chicago')))

ctmd2.getCompose;
// Task(Just('Rock over London, rock on, Chicago'))

```

Ahí lo tienes, un solo `map`. La composición de funtores es asociativa y anteriormente definimos `Container` que en realidad se llama funtor `Identidad`. Si tenemos identidad y composición asociativa tenemos una categoría. Esta categoría en particular tiene categorías como objetos y funtores como morfismos, lo que es suficiente para hacer que a uno le transpire el cerebro. No profundizaremos demasiado en esto, pero es bueno apreciar las implicaciones arquitectónicas o incluso tan solo la simple belleza abstracta en el patrón.

En Resumen

Hemos visto unos cuantos funtores distintos, pero hay una infinidad de ellos. Algunas omisiones notables son las estructuras de datos iterables como árboles, listas, mapas, pares, lo que sea. Los event streams [*flujos de eventos*] y los observables son ambos funtores. Otros pueden usarse para encapsular o incluso solo para el modelado de tipos. Los funtores nos rodean y los utilizaremos ampliamente a lo largo del libro.

¿Qué pasa con la llamada a una función con múltiples funtores como argumentos? ¿Qué hay de trabajar con una secuencia ordenada de acciones impuras o asíncronas? Todavía no hemos adquirido el conjunto completo de herramientas para trabajar en este mundo de cajas. A continuación, iremos al grano y veremos las mónadas.

Ejercicios

Exercise

Utiliza `add` y `map` para crear una función que incremente el valor de dentro de un funtor.

```
// incrF :: Functor f => f Int -> f Int
const incrF = undefined;
```

Dado el siguiente objeto User:

```
const user = { id: 2, name: 'Albert', active: true };
```

Exercise

Utiliza `safeProp` y `head` para encontrar la primera inicial del usuario.

```
// initial :: User -> Maybe String
const initial = undefined;
```

Dada las siguientes funciones de soporte:

```
// showWelcome :: User -> String
const showWelcome = compose(concat('Welcome '), prop('name'));

// checkActive :: User -> Either String User
const checkActive = function checkActive(user) {
  return user.active
    ? Either.of(user)
    : left('Your account is not active');
};
```

Exercise

Escribe una función que utilice `checkActive` y `showWelcome` para conceder el acceso o devolver el error.

```
// eitherWelcome :: User -> Either String String
const eitherWelcome = undefined;
```

Ahora consideremos las siguientes funciones:

```
// validateUser :: (User -> Either String ()) -> User -> Either String User
const validateUser = curry((validate, user) => validate(user).map(_ => user));

// save :: User -> IO User
const save = user => new IO(() => ({ ...user, saved: true }));
```

Exercise

Escribe una función `validateName` que compruebe que el nombre del usuario tiene más de 3 caracteres y si no que devuelva un mensaje de error. Luego utiliza `either`, `showWelcome` y `save` para escribir una función `register` para registrar al usuario y darle la bienvenida cuando la validación sea correcta. Recuerda que los dos argumentos de either deben devolver el mismo tipo.

```
// validateName :: User -> Either String ()
const validateName = undefined;

// register :: User -> IO String
const register = compose(undefined, validateUser(validateName));
```

Capítulo 09: Cebollas Monádicas

Fábrica de Funtores Puntiagudos

Antes de seguir avanzando, tengo algo que confesar: No he sido completamente honesto sobre ese método `of` que hemos colocado en cada uno de nuestros tipos. Resulta que no está ahí para evitar la palabra clave `new`, si no para colocar los valores en lo que se llama *contexto mínimo por defecto*. Sí, `of` no sustituye a un constructor, sino que forma parte de una importante interfaz a la que llamamos *Pointed*.

Un *functor pointed* es un functor con un método `of`

Lo importante aquí es la capacidad de dejar caer cualquier valor dentro de nuestro tipo y poder empezar a aplicar `map`.

```
IO.of('tetris').map(concat(' master'));
// IO('tetris master')

Maybe.of(1336).map(add(1));
// Maybe(1337)

Task.of([ { id: 2 }, { id: 3 } ]).map(map(prop('id')));
// Task([2,3])

Either.of('The past, present and future walk into a bar...').map(concat('it was tense'));
// Right('The past, present and future walk into a bar...it was tense.')
```

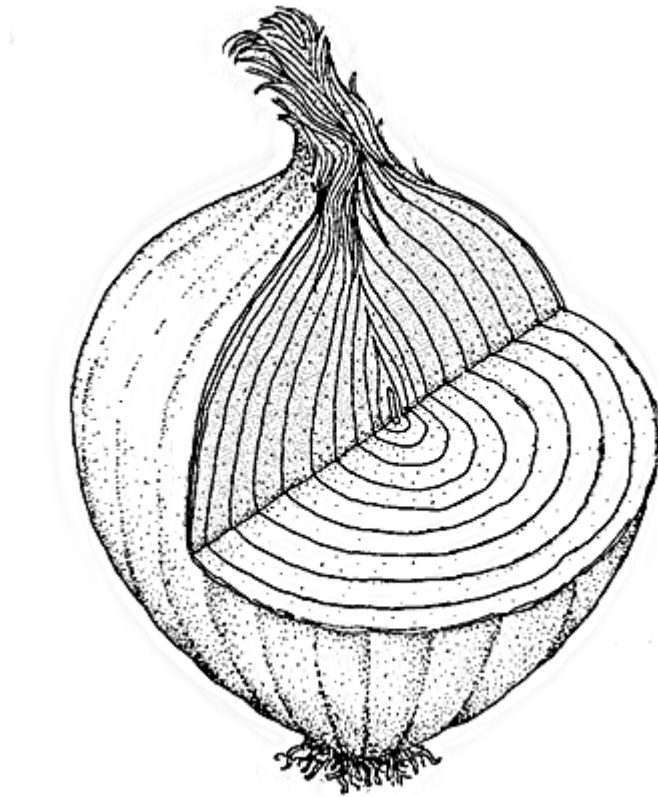
Si recuerdas, los constructores de `IO` y `Task` esperan una función como argumento, pero `Maybe` y `Either` no. La motivación para esta interfaz es tener una forma común y consistente de colocar un valor en nuestro functor sin las complejidades y demandas específicas de cada constructor. El término "contexto mínimo por defecto" carece de precisión, pero recoge bien la idea: nos gustaría levantar cualquier valor dentro de nuestro tipo y aplicarle `map` como de costumbre, obteniendo el comportamiento esperado de cualquier functor.

Una corrección importante que debo hacer llegados a este punto, es que `Left.of` no tiene ningún sentido. Cada functor debe tener una forma de colocarle dentro un valor y en `Either` eso se hace con `new Right(x)`. Definimos `of` usando `Right` porque si nuestro tipo *puede* aplicar `map`, debe aplicar `map`. Viendo los ejemplos anteriores, deberíamos intuir como funcionará `of` normalmente y `Left` rompe ese molde.

Es posible que hayas oído hablar de funciones como `pure`, `point`, `unit`, y `return`. Estos son varios alias para nuestro método `of`, la función internacional del misterio. `of` será importante cuando empecemos a usar mónadas porque, como veremos, es nuestra responsabilidad volver a colocar los valores en el tipo manualmente.

Para evitar la palabra clave `new`, hay varios trucos estándar en JavaScript o en librerías así que los utilizaremos y de ahora en adelante usaremos `of` como adultos responsables que somos. Recomendando usar funtores de `folktaile`, `ramda` o `fantasy-land` ya que proporcionan el método `of` correcto así como amables constructores que no dependen de `new`.

Mezclando Metáforas



Verás, además de burritos espaciales (si has oído los rumores), las mónadas son como las cebollas. Permíteme demostrarlo con una situación muy común:

```
const fs = require('fs');

// readFile :: String -> IO String
const readFile = filename => new IO(() => fs.readFileSync(filename, 'utf-8'));

// print :: String -> IO String
const print = x => new IO(() => {
  console.log(x);
  return x;
});

// cat :: String -> IO (IO String)
const cat = compose(map(print), readFile);

cat('.git/config');
// IO(IO('[core]\nrepositoryformatversion = 0\n'))
```

Lo que hemos obtenido es un `IO` atrapado dentro de otro `IO` porque `print` introdujo un segundo `IO` al aplicarla con `map`. Para seguir trabajando con nuestro string, debemos hacer `map(map(f))` y para ver el efecto debemos hacer `unsafePerformIO().unsafePerformIO()`.

```
// cat :: String -> IO (IO String)
const cat = compose(map(print), readFile);

// catFirstChar :: String -> IO (IO String)
const catFirstChar = compose(map(map(head)), cat);

catFirstChar('.git/config');
// IO(IO('['))
```

Aunque es bueno ver que en nuestra aplicación tenemos dos efectos empaquetados y listos para salir, se parece a trabajar con dos trajes de seguridad contra materiales peligrosos y acabamos con una extraña e incómoda API. Veamos otra situación:

```
// safeProp :: Key -> {Key: a} -> Maybe a
const safeProp = curry((x, obj) => Maybe.of(obj[x]));

// safeHead :: [a] -> Maybe a
const safeHead = safeProp(0);

// firstAddressStreet :: User -> Maybe (Maybe (Maybe Street))
const firstAddressStreet = compose(
  map(map(safeProp('street'))),
  map(safeHead),
  safeProp('addresses'),
);

firstAddressStreet({
  addresses: [{ street: { name: 'Mulburry', number: 8402 }, postcode: 'WC2N' }],
});
// Maybe(Maybe(Maybe({name: 'Mulburry', number: 8402})))
```

De nuevo vemos esta situación en la que tenemos funtores anidados donde es bueno poder ver que hay tres posibilidades de fallo en nuestra función, pero es un poco presuntuoso suponer que quien nos llame va a aplicar `map` tres veces para llegar al valor; acabamos de conocernos. Este patrón aparecerá una y otra vez y es la razón principal por la que necesitaremos hacer brillar en el cielo nocturno el poderoso símbolo de la mónada.

He dicho que las mónadas son como cebollas porque se nos saltan las lágrimas cuando pelamos con `map` cada capa de funtor anidado para llegar al valor del interior. Podemos secar nuestros ojos, respirar hondo, y utilizar un método llamado `join` [unir].

```

const mmo = Maybe.of(Maybe.of('nunchucks'));
// Maybe(Maybe('nunchucks'))

mmo.join();
// Maybe('nunchucks')

const ioio = IO.of(IO.of('pizza'));
// IO(IO('pizza'))

ioio.join();
// IO('pizza')

const ttt = Task.of(Task.of(Task.of('sewers')));
// Task(Task(Task('sewers')));

ttt.join();
// Task(Task('sewers'))

```

Si tenemos dos capas del mismo tipo, podemos unir las aplastando la una con la otra mediante `join`. Esta capacidad de unir, este matrimonio de funtores, es lo que hace mónada a una mónada. Avancemos hacia la definición completa con algo un poco más preciso:

Las mónadas son funtores pointed que pueden aplanar

Cualquier funtor que defina un método `join`, que tenga un método `of`, y que obedezca unas pocas leyes, es una mónada. Definir `join` no es muy difícil así que hagámoslo para `Maybe`:

```

Maybe.prototype.join = function join() {
  return this.isNothing() ? Maybe.of(null) : this.$value;
};

```

Ahí está, tan simple como absorber a nuestro propio gemelo en el vientre. Si tenemos `Maybe(Maybe(x))` entonces `.$value` simplemente eliminará la innecesaria capa adicional y a partir de ahí podremos aplicar `map` con seguridad. De lo contrario, solo tendremos el `Maybe` ya que no se habría mapeado nada en primer lugar.

Ahora que tenemos un método `join`, vamos a espolvorear algo de polvo de mónada mágica sobre el ejemplo de `firstAddressStreet` y a verlo en acción:

```
// join :: Monad m => m (m a) -> m a
const join = mma => mma.join();

// firstAddressStreet :: User -> Maybe Street
const firstAddressStreet = compose(
  join,
  map(safeProp('street')),
  join,
  map(safeHead), safeProp('addresses'),
);

firstAddressStreet({
  addresses: [{ street: { name: 'Mulberry', number: 8402 }, postcode: 'WC2N' }],
});
// Maybe({name: 'Mulberry', number: 8402})
```

Hemos añadido `join` allá donde nos hemos encontrado `Maybe` s anidados para evitar que se nos vayan de las manos. Hagamos lo mismo con `IO` para asentar la idea.

```
IO.prototype.join = () => this.unsafePerformIO();
```

De nuevo, solo hemos eliminado una capa. O sea, no nos hemos deshecho de la pureza, sino que simplemente hemos eliminado una capa sobrante de embalaje.

```
// log :: a -> IO a
const log = x => new IO(() => {
  console.log(x);
  return x;
});

// setStyle :: Selector -> CSSProps -> IO DOM
const setStyle =
  curry((sel, props) => new IO(() => jQuery(sel).css(props)));

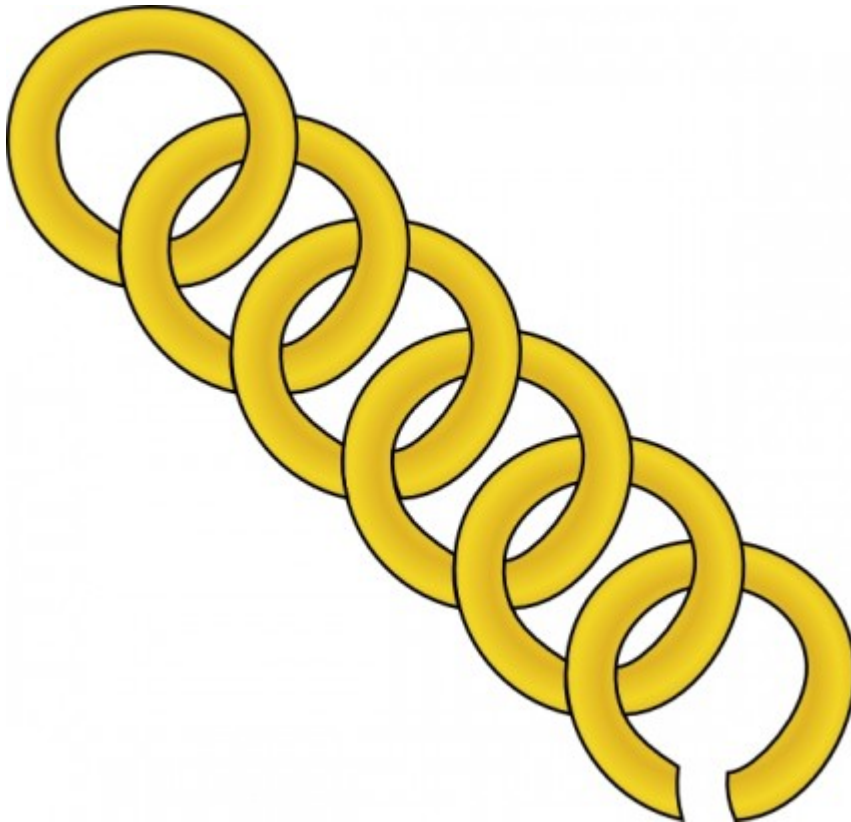
// getItem :: String -> IO String
const getItem = key => new IO(() => localStorage.getItem(key));

// applyPreferences :: String -> IO DOM
const applyPreferences = compose(
  join,
  map(setStyle('#main')),
  join,
  map(log),
  map(JSON.parse),
  getItem,
);

applyPreferences('preferences').unsafePerformIO();
// Object {backgroundColor: "green"}
// <div style="background-color: 'green'"/>
```

`getItem` devuelve un `IO String` así que aplicamos `map` para parsearlo. Tanto `log` como `setStyle` devuelven `IO` por lo que hemos de aplicar `join` para mantener nuestro anidamiento bajo control.

Mi Cadena Me Golpea El Pecho



Puede que hayas notado un patrón. A menudo acabamos llamando a `join` justo después de un `map`. Abstraigamos esto en una función llamada `chain` [cadena].

```
// chain :: Monad m => (a -> m b) -> m a -> m b
const chain = curry((f, m) => m.map(f).join());

// or

// chain :: Monad m => (a -> m b) -> m a -> m b
const chain = f => compose(join, map(f));
```

Tan solo hemos agrupado este combo `map/join` en una sola función. Si has leído sobre mónadas anteriormente, puede que también hayas visto a `chain` llamada como `>=>` (pronunciado `bind`) o `flatMap` que son todo alias para el mismo concepto. Personalmente, creo que `flatMap` es el nombre más preciso, pero continuaremos con `chain` ya que es el nombre ampliamente aceptado en JS. Refactoricemos los dos ejemplos anteriores con `chain`:

```
// map/join
const firstAddressStreet = compose(
  join,
  map(safeProp('street')),
  join,
  map(safeHead),
  safeProp('addresses'),
);

// chain
const firstAddressStreet = compose(
  chain(safeProp('street')),
  chain(safeHead),
  safeProp('addresses'),
);

// map/join
const applyPreferences = compose(
  join,
  map(setStyle('#main')),
  join,
  map(log),
  map(JSON.parse),
  getItem,
);

// chain
const applyPreferences = compose(
  chain(setStyle('#main')),
  chain(log),
  map(JSON.parse),
  getItem,
);
```

He reemplazado cualquier `map/join` por nuestra nueva función `chain` para ordenar un poco las cosas. Lo de limpiar está muy bien y tal, pero hay más cosas en `chain` de las que se ven a simple vista; es más un tornado que una aspiradora. Como `chain` anida efectos sin esfuerzo alguno, podemos capturar de una forma puramente funcional tanto la *secuencia* como la *asignación de variables*.

```
// getJSON :: Url -> Params -> Task JSON
getJSON('/authenticate', { username: 'stale', password: 'crackers' })
  .chain(user => getJSON('/friends', { user_id: user.id }));
// Task([{name: 'Seimith', id: 14}, {name: 'Ric', id: 39}]);

// querySelector :: Selector -> IO DOM
querySelector('input.username')
  .chain(({ value: uname }) =>
    querySelector('input.email')
      .chain(({ value: email }) => IO.of(`Welcome ${uname} prepare for spam at
    `)));
// IO('Welcome Olivia prepare for spam at olivia@tremorcontrol.net');

Maybe.of(3)
  .chain(three => Maybe.of(2).map(add(three)));
// Maybe(5);

Maybe.of(null)
  .chain(safeProp('address'))
  .chain(safeProp('street'));
// Maybe(null);
```

Podríamos haber escrito estos ejemplos con `compose`, pero habríamos necesitado unas cuantas funciones de soporte y, de todos modos, este estilo se presta a la asignación explícita de variables a través de closures. En vez de esto estamos usando la versión infija de `chain` que, por cierto, puede ser derivada automáticamente de `map` y `join` para cualquier tipo: `t.prototype.chain = function(f) { return this.map(f).join(); }`. También podemos definir `chain` manualmente si queremos una falsa sensación de rendimiento, aunque deberemos tener cuidado con mantener la funcionalidad correcta, es decir, debe ser igual que `map` seguido de `join`. Un hecho interesante es que si hemos creado `chain` podemos derivar `map` sin mucho esfuerzo simplemente embotellando de nuevo el valor con `of` cuando hemos terminado. Con `chain`, también podemos definir `join` como `chain(id)`. Puede parecer que estamos jugando al "Texas Hold em" con un mago de la bisutería en el sentido de que nos estamos sacando cosas de la espalda, pero, como en la mayoría de las matemáticas, todas estas construcciones basadas en principios están interrelacionadas. Muchas de estas derivaciones se mencionan en el repo de [fantasyland](#), que es la especificación oficial en JavaScript para tipos de datos algebraicos.

De todos modos, vamos a los ejemplos anteriores. En el primer ejemplo vemos dos tareas encadenadas en una secuencia de acciones asíncronas; primero recupera a la persona usuaria y luego con su id encuentra a sus amistades. Usamos `chain` para evitar vernos en la situación de `Task(Task([Friend]))`.

A continuación, utilizamos `querySelector` para encontrar diferentes entradas y crear un mensaje de bienvenida. Date cuenta de que en la función más interna tenemos acceso tanto a `uname` como a `email`; eso es asignación funcional de variables en su máxima expresión. Dado que `IO` nos presta amablemente su valor, tenemos la responsabilidad de dejarlo como lo encontramos, pues no querríamos corromper su credibilidad (ni nuestro programa). `IO.of` es la herramienta perfecta para el trabajo y es la razón por la que `Pointed` es un prerequisite importante para la interfaz `Mónada`. Sin embargo, podríamos optar por aplicar `map` ya que eso también devolvería el tipo correcto.

```
querySelector('input.username').chain({ value: uname }) =>
  querySelector('input.email').map({ value: email }) =>
    `Welcome ${uname} prepare for spam at ${email}`);
// IO('Welcome Olivia prepare for spam at olivia@tremorcontrol.net');
```

Por último, tenemos dos ejemplos que usan `Maybe`. Dado que `chain` está usando `map` por debajo, si cualquier valor es nulo, detenemos en seco la computación.

No te preocupes si estos ejemplos son difíciles de entender al principio. Juega con ellos. Incórdiales con un palo. Rómpelos en trozos y únelos de nuevo. Recuerda aplicar `map` cuando lo devuelto sea un valor "normal" y `chain` cuando lo devuelto sea otro funtor. En el próximo capítulo, nos acercaremos a los `Aplicativos` y veremos buenos trucos para hacer que este tipo de expresiones sean más bonitas y altamente legibles.

Como recordatorio, esto no funciona con dos tipos anidados diferentes. La composición de funtores y, posteriormente, los transformadores de mónadas, pueden ayudarnos en esa situación.

Borrachera de Poder

Programar utilizando contenedores puede llegar a ser confuso. En ocasiones nos vemos luchando por entender dentro de cuantos contenedores está un valor o si tenemos que utilizar `map` o `chain` (pronto veremos más métodos de contenedores). Podemos mejorar mucho la depuración con trucos como implementar `inspect` y aprenderemos a crear una pila [*stack*] que pueda manejar cualquier efecto que le lancemos, pero aún y así hay veces que nos preguntamos si merecen la pena tantas molestias.

Me gustaría blandir por un momento la ardiente espada monádica para exhibir el poder de programar de esta manera.

Leamos un archivo para después subirlo directamente:

```
// readFile :: Filename -> Either String (Task Error String)
// httpPost :: String -> String -> Task Error JSON
// upload :: Filename -> Either String (Task Error JSON)
const upload = compose(map(chain(httpPost('/uploads'))), readFile);
```

Aquí estamos bifurcando varias veces nuestro código. Mirando las firmas de tipo puedo ver que nos protegemos contra 3 errores. `readFile` utiliza `Either` para validar la entrada (quizás asegurándose de que el archivo está presente), `readFile` puede fallar cuando accede al archivo como expresa el primer parámetro de tipo de `Task`, y la subida puede fallar por cualquier razón tal y como expresa el `Error` en `httpPost`. Con `chain` hemos llevado a cabo dos acciones asíncronas anidadas y secuenciales sin mucho esfuerzo.

Todo esto se ha conseguido con un solo flujo lineal de derecha a izquierda. Todo es puro y declarativo. Contiene razonamiento ecuacional y propiedades fiables. No nos vemos forzados a añadir confusos e innecesarios nombres de variables. Nuestra función `upload` está escrita con una interfaz genérica y no con una API específica de un solo uso. Es una maldita línea por dios.

Para contrastar, veamos la forma imperativa estándar de llevar esto a cabo:

```
// upload :: Filename -> (String -> a) -> Void
const upload = (filename, callback) => {
  if (!filename) {
    throw new Error('You need a filename!');
  } else {
    readFile(filename, (errF, contents) => {
      if (errF) throw errF;
      httpPost('/uploads', contents, (errH, json) => {
        if (errH) throw errH;
        callback(json);
      });
    });
  }
};
```

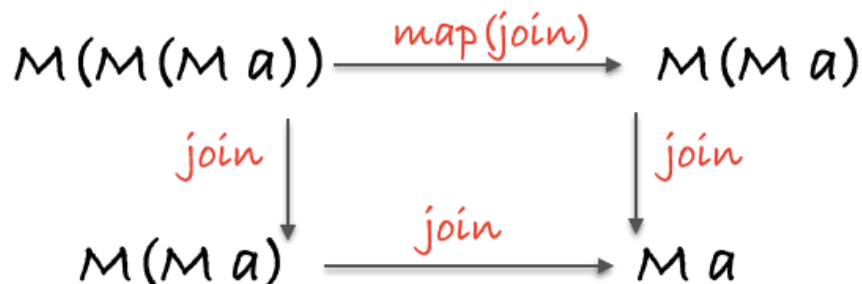
Bueno, ¿no es esto la aritmética del diablo? Se nos hace rebotar a través de un volátil laberinto de locura. ¡Imagina que además fuese la típica app que va mutando variables sobre la marcha! Verdaderamente estaríamos en un pozo de alquitrán.

Teoría

La primera ley que veremos es la asociatividad, pero puede que no de la forma acostumbrada.

```
// asociatividad
compose(join, map(join)) === compose(join, join);
```

Estas leyes atacan al anidamiento característico de las mónadas por lo que la asociatividad se centra en unir primero los tipos más internos o primero los más externos para llegar al mismo resultado. Una imagen puede ser más instructiva:

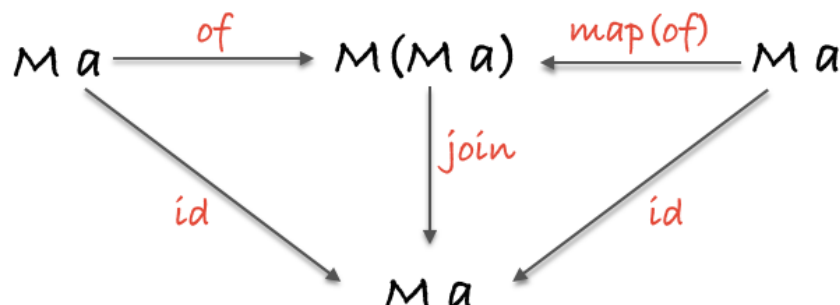


Empezando por la parte superior izquierda y moviéndonos hacia abajo, primero podemos unir con `join` las dos `M` más externas en `M(M(M a))` para luego llegar hasta nuestra deseada `M a` con otro `join`. Alternativamente, podemos abrir el capó y aplanar las dos `M` más internas con `map(join)`. Acabamos con la misma `M a` independientemente de si unimos primero las `M` más internas o primero las más externas y eso es todo sobre lo que trata la asociatividad. Hay que tener en cuenta que `map(join) != join`. Los pasos intermedios pueden variar en valor, pero el resultado final del último `join` será el mismo.

La segunda ley es similar:

```
// identidad para todo (M a)
compose(join, of) === compose(join, map(of)) === id;
```

Afirma que, para cualquier mónada `M`, `of` y `join` equivale a `id`. Podemos incluso hacer `map(of)` y atacarla de dentro hacia afuera. A esto lo llamamos "identidad triangular" porque tiene esa forma cuando lo visualizamos:



Si comenzamos por arriba a la izquierda y vamos hacia la derecha, podemos ver que `of` deja caer nuestro `M a` dentro de otro contenedor `M`. Luego, si nos movemos hacia abajo y aplicamos `join`, obtenemos lo mismo que si hubiésemos llamado a `id` desde el principio. Moviéndonos de derecha a

izquierda, vemos que si nos escabullimos bajo las mantas con `map` y llamamos a `of` con `a` tal cual, igualmente acabaremos con `M (M a)` y aplicando `join` volveremos al punto de partida.

Debo mencionar que acabo de escribir `of`, sin embargo, ha de ser el `M.of` específico para cualquier mónada que estemos utilizando.

Un momento, he visto estas leyes, identidad y asociatividad, en algún otro sitio antes... Espera, estoy pensando... ¡Por supuesto! Son las leyes de una categoría. Pero eso significaría que necesitamos una función de composición para completar la definición. Contempla:

```
const mcompose = (f, g) => compose(chain(f), g);

// identidad por la izquierda
mcompose(M, f) === f;

// identidad por la derecha
mcompose(f, M) === f;

// asociatividad
mcompose(mcompose(f, g), h) === mcompose(f, mcompose(g, h));
```

Estas son las leyes de la categoría después de todo. Las mónadas forman una categoría llamada "categoría Kleisli" en la que todos los objetos son mónadas y los morfismos son funciones encadenadas. No pretendo burlarme de ti con trozos de teoría de categorías sin dar mucha explicación de como encaja el rompecabezas. La intención es arañar la superficie lo suficiente como para mostrar su relevancia, y despertar cierto interés mientras nos concentramos en las propiedades prácticas que podremos usar cada día.

En Resumen

Las mónadas nos permiten perforar a través de computaciones anidadas. Podemos asignar variables, ejecutar efectos secuenciales, realizar tareas asíncronas, todo ello sin colocar un solo ladrillo en la pirámide del terror. Vienen al rescate cuando un valor se encuentra encarcelado bajo múltiples capas del mismo tipo. Con la ayuda del fiel compañero "pointed", las mónadas son capaces de prestarnos un valor sin su caja sabiendo que podremos colocarlo de nuevo donde estaba cuando hayamos terminado.

Sí, las mónadas son muy potentes, pero aún y así seguimos viendo que necesitamos algunas funciones de contenedor adicionales. Por ejemplo, ¿y si necesitamos ejecutar a la vez una lista de llamadas a una api y luego reunir los resultados? Podemos realizar esta tarea con mónadas, pero tendríamos que esperar a que cada una terminase antes de llamar a la siguiente. ¿Qué hay de

combinar diversas validaciones? Nos gustaría seguir validando para ir recopilando la lista de errores, pero las mónadas detendrán el espectáculo nada más entrar a escena el primer `Left`.

En el próximo capítulo, veremos como encajan los funtores aplicativos en el mundo de los contenedores y por qué en muchos casos los preferimos a las mónadas.

Capítulo 10: Funtores Aplicativos

Ejercicios

Considerando un objeto `User` como el que sigue:

```
const user = {
  id: 1,
  name: 'Albert',
  address: {
    street: {
      number: 22,
      name: 'Walnut St',
    },
  },
};
```

Exercise

Utiliza `safeProp` y `map/join` o `chain` para obtener de manera segura el nombre de la calle cuando se proporciona un usuario

```
// getStreetName :: User -> Maybe String
const getStreetName = undefined;
```

Consideremos ahora los siguientes elementos:

```
// getFile :: IO String
const getFile = IO.of('/home/mostly-adequate/ch09.md');

// pureLog :: String -> IO ()
const pureLog = str => new IO(() => console.log(str));
```

Exercise

Utiliza `getFile` para obtener la ruta del archivo, eliminar el directorio y mantener solo el nombre base y luego muéstralo de forma pura. Sugerencia: podrías querer usar ``split`` y ``last`` para obtener el nombre base de una ruta de archivo.

```
// logFilename :: IO ()
const logFilename = undefined;
```

Para este ejercicio, consideremos las funciones de soporte con las siguientes firmas:

```
// validateEmail :: Email -> Either String Email
// addToMailingList :: Email -> IO([Email])
// emailBlast :: [Email] -> IO ()
```

Exercise

Utiliza ``validateEmail``, ``addToMailingList`` y ``emailBlast`` para crear una función que añada un nuevo correo electrónico a la lista de correo si este es válido, y que luego lo notifique a toda la lista.

```
// joinMailingList :: Email -> Either String (IO ())
const joinMailingList = undefined;
```

Capítulo 10: Funtores Aplicativos

Aplicando Aplicativos

El nombre **funtor aplicativo** es placenteramente descriptivo dados sus orígenes funcionales. Los programadores funcionales son conocidos por aparecer con nombres como `mappend` o `liftA4`, que parecen perfectamente naturales cuando se ven en el laboratorio de matemáticas, pero que en cualquier otro contexto son claros como un Darth Vader indeciso en el autoservicio.

En cualquier caso, el nombre debería revelar lo que esta interfaz nos da: *la capacidad de aplicar funtores entre ellos*.

Pero, ¿por qué una persona normal y racional como tú querría una cosa así? Incluso, ¿qué *significa* aplicar un funtor a otro?

Para responder a estas preguntas, comenzaremos con una situación en la que ya te habrás visto en tus viajes funcionales. Digamos que, hipotéticamente, tenemos dos funtores (del mismo tipo) con sus respectivos valores, y que queremos llamar a una función con esos dos valores como argumentos. Algo simple, como sumar los valores de dos `Container`.

```
// No podemos hacer esto porque los números están embotellados.
add(Container.of(2), Container.of(3));

// NaN

// Usemos nuestra función map en la que tanto confiamos
const containerOfAdd2 = map(add, Container.of(2));
// Container(add(2))
```

Ahora tenemos un `Container` con una función dentro que está parcialmente aplicada. Más específicamente, tenemos un `Container(add(2))` y queremos aplicar su `add(2)` al `3` de `Container(3)` para completar la llamada. En otras palabras, queremos aplicar un funtor a otro funtor.

Pues resulta que ya tenemos las herramientas para llevar a cabo esta tarea. Podemos aplicar `chain` y luego `map` a la función parcialmente aplicada `add(2)`, tal que así:

```
Container.of(2).chain(two => Container.of(3).map(add(two)));
```

El problema aquí es que estamos atrapados en el mundo secuencial de las mónadas en el que nada puede ser evaluado hasta que la mónada anterior haya terminado su trabajo. Tenemos dos valores fuertes e independientes y parece innecesario retrasar la creación de `Container(3)` tan solo para satisfacer las demandas secuenciales de las mónadas.

De hecho, si nos viésemos en este aprieto, sería maravilloso si pudiéramos, sucintamente, aplicar el contenido de un funtor al valor de otro, sin esas funciones y variables innecesarias.

Barcos en Botellas



`ap` es una función que puede aplicar la función contenida en un funtor al valor contenido en otro. Di esto cinco veces más rápido.

```
Container.of(add(2)).ap(Container.of(3));  
// Container(5)  
  
// ahora todo junto  
  
Container.of(2).map(add).ap(Container.of(3));  
// Container(5)
```

Así sí, bonito y limpio. Buenas noticias para `Container(3)` dado que ha sido liberado de la cárcel de la función monádica anidada. Vale la pena mencionar de nuevo que `add`, en este caso, resulta parcialmente aplicada durante el primer `map` así que esto solo funciona cuando `add` está currificada.

Podemos definir `ap` como:

```
Container.prototype.ap = function (otherContainer) {  
  return otherContainer.map(this.$value);  
};
```

Recuerda, `this.$value` será una función y aceptaremos otro funtor por lo que solo necesitaremos mapearlo. Y con eso tenemos definida nuestra interfaz:

Un *funtor aplicativo* es un funtor **pointed** con un método `ap`

Observa la dependencia en **pointed**. La interfaz **pointed** es aquí crucial, tal y como veremos en los próximos ejemplos.

Percibo tu escepticismo (o quizás confusión y horror), pero mantén la mente abierta; este personaje `ap` demostrará ser de utilidad. Antes de meternos en ello, exploremos una bonita propiedad.

```
F.of(x).map(f) === F.of(f).ap(F.of(x));
```

En correcto español, mapear `f` equivale a usar la función `ap` de un funtor de `f`. O en un español más correcto, podemos colocar `x` en nuestro contenedor y hacer `map(f)`, o, podemos levantar tanto `f` como `x` en nuestro contenedor y luego aplicarles `ap`. Esto nos permite escribir de izquierda a derecha:

```
Maybe.of(add).ap(Maybe.of(2)).ap(Maybe.of(3));
// Maybe(5)

Task.of(add).ap(Task.of(2)).ap(Task.of(3));
// Task(5)
```

Entrecerrando los ojos, se puede incluso reconocer vagamente la manera normal de llamar a una función. Más adelante en el capítulo veremos la versión **pointfree**, pero por ahora, esta es la manera preferida de escribir un código como este. Usando `of`, cada valor es transportado al mágico mundo de los contenedores, ese universo paralelo donde cada aplicación puede ser asíncrona o nula o lo que sea y donde `ap` aplicará funciones dentro de ese lugar de fantasía. Es como construir un barco dentro de una botella.

¿Has visto? Hemos utilizado `Task` en nuestro ejemplo. Esta es una de las principales situaciones en las que los funtores aplicativos muestran su fuerza. Veamos un ejemplo más en profundidad.

Motivación para la Coordinación

Digamos que estamos construyendo un sitio web de viajes y que queremos recuperar tanto una lista de destinos turísticos como de eventos locales. Cada una es una llamada separada e independiente.

```
// Http.get :: String -> Task Error HTML

const renderPage = curry((destinations, events) => { /* generar página */ });

Task.of(renderPage).ap(Http.get('/destinations')).ap(Http.get('/events'));
// Task("<div>una página con destinos y eventos</div>")
```

Ambas llamadas `Http` se harán a la vez y `renderPage` será llamada cuando ambas se hayan resuelto. Contrasta esto con la versión monádica en la que una tarea `Task` debe finalizar antes de que se inicie la siguiente. Dado que no necesitamos los destinos para recuperar los eventos, nos libramos de la evaluación secuencial.

De nuevo, como estamos utilizando aplicación parcial para alcanzar este resultado, debemos asegurarnos de que `renderPage` está currificada o no esperará a que terminen ambas tareas. Por cierto, si alguna vez has tenido que hacer algo así manualmente, apreciarás la asombrosa simplicidad de esta interfaz. Esta es la bonita forma del código que nos acerca un paso más hacia la singularidad.

Veamos otro ejemplo.

```
// $ :: String -> IO DOM
const $ = selector => new IO(()) => document.querySelector(selector));

// getVal :: String -> IO String
const getVal = compose(map(prop('value')), $);

// signIn :: String -> String -> Bool -> User
const signIn = curry((username, password, rememberMe) => { /* signing in */ });

IO.of(signIn).ap(getVal('#email')).ap(getVal('#password')).ap(IO.of(false));
// IO({ id: 3, email: 'gg@allin.com' })
```

`signIn` es una función currificada de 3 argumentos por lo que tenemos que aplicar `ap` en consecuencia. Con cada `ap`, `signIn` recibe un argumento más hasta que está completa y luego se ejecuta. Podemos seguir este patrón con tantos argumentos como sea necesario. Otra cosa a tener en cuenta es que dos argumentos terminan de manera natural en `io` mientras que el último necesita una pequeña ayuda de `of` para levantarlo a `io` ya que `ap` espera que la función y todos sus argumentos sean del mismo tipo.

Colega, ¿Alguna Vez Levantas?

Examinemos una manera *pointfree* de escribir esas llamadas aplicativas. Dado que sabemos que `map` equivale a `of/ap`, podemos escribir funciones genéricas que aplicarán `ap` tantas veces como especifiquemos:

```
const liftA2 = curry((g, f1, f2) => f1.map(g).ap(f2));

const liftA3 = curry((g, f1, f2, f3) => f1.map(g).ap(f2).ap(f3));

// liftA4, etc
```

`liftA2` es un nombre extraño. Suena a uno de esos ascensores de carga poco fiables de una fábrica en decadencia o a una matrícula para una empresa de limusinas baratas. Sin embargo, una vez aclarado, se explica por sí mismo: levanta esas piezas al mundo del funtor aplicativo.

Cuando vi por primera vez ese sin sentido de 2-3-4 me pareció feo e innecesario. Después de todo, podemos comprobar la aridez de las funciones en JavaScript y construirlas dinámicamente. Sin embargo, suele ser útil aplicar parcialmente a `liftA(N)` para que no pueda variar en la cantidad de argumentos.

Veámoslo en uso:

```
// checkEmail :: User -> Either String Email
// checkName  :: User -> Either String String

const user = {
  name: 'John Doe',
  email: 'blurp_blurp',
};

// createUser :: Email -> String -> IO User
const createUser = curry((email, name) => { /* creando... */ });

Either.of(createUser).ap(checkEmail(user)).ap(checkName(user));
// Left('email inválido')

liftA2(createUser, checkEmail(user), checkName(user));
// Left('email inválido')
```

Como `createUser` toma dos argumentos, utilizamos el correspondiente `liftA2`. Las dos sentencias son equivalentes, pero la versión `liftA2` no menciona a `Either`. Esto la hace más genérica y flexible, ya que dejamos de casarnos con un tipo específico.

Veamos los ejemplos anteriores escritos de esta manera:

```
liftA2(add, Maybe.of(2), Maybe.of(3));  
// Maybe(5)  
  
liftA2(renderPage, Http.get('/destinations'), Http.get('/events'));  
// Task('<div>una página con destinos y eventos</div>')  
  
liftA3(signIn, getVal('#email'), getVal('#password'), IO.of(false));  
// IO({ id: 3, email: 'gg@allin.com' })
```

Operadores

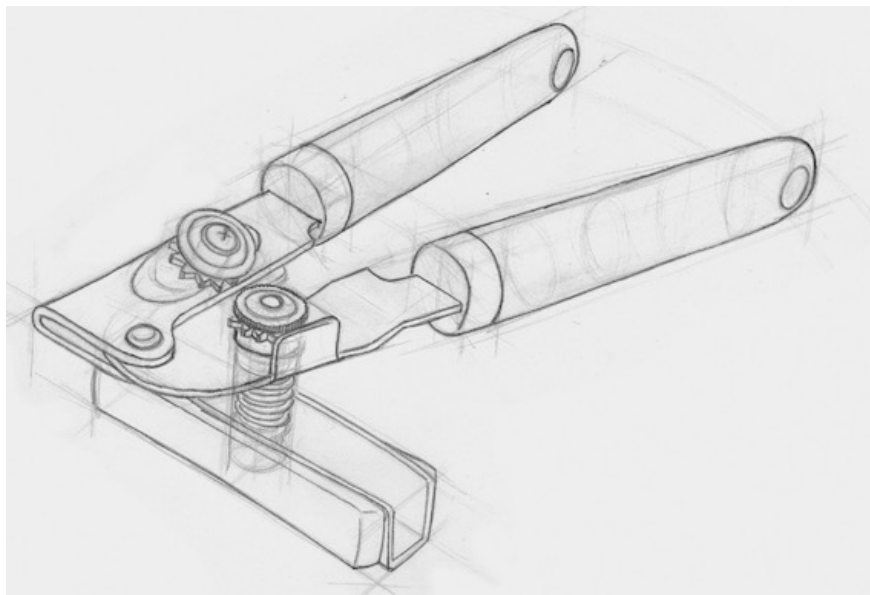
En lenguajes como Haskell, Scala, PureScript y Swift, en los cuales es posible crear tus propios operadores infijos, puedes ver sintaxis como esta:

```
-- Haskell / PureScript  
add <$> Right 2 <*> Right 3
```

```
// JavaScript  
map(add, Right(2)).ap(Right(3));
```

Es útil saber que `<$>` es `map` (también conocido como `fmap`) y que `<*>` es simplemente `ap`. Esto permite un estilo de aplicación de funciones más natural y puede ayudar a eliminar algunos paréntesis.

Abrelatas Gratis



No hemos hablado mucho de las funciones derivadas. Dado que todas estas interfaces se construyen a partir de otras y obedecen a un conjunto de leyes, podemos definir algunas interfaces más débiles en términos de las más fuertes.

Por ejemplo, sabemos que un aplicativo es primero un funtor, así que, si tenemos un ejemplar de aplicativo, seguramente podamos definir un funtor para nuestro tipo.

Esta clase de perfecta armonía computacional es posible porque estamos trabajando dentro de un marco matemático. Mozart no podría haberlo hecho mejor aún teniendo Ableton de niño.

Antes mencioné que `of/ap` equivale a `map`. Podemos utilizar este conocimiento para definir `map`:

```
// map derivado de of/ap
X.prototype.map = function map(f) {
  return this.constructor.of(f).ap(this);
};
```

Las mónadas están, por así decirlo, arriba del todo de la cadena alimenticia, así que si tenemos a `chain`, obtenemos funtor y aplicativo de forma gratuita:

```
// map derivada de chain
X.prototype.map = function map(f) {
  return this.chain(a => this.constructor.of(f(a)));
};

// ap derivada de chain/map
X.prototype.ap = function ap(other) {
  return this.chain(f => other.map(f));
};
```

Si podemos definir una mónada, podemos definir tanto la interfaz de aplicativo como la de funtor. Esto es bastante notable, ya que obtenemos todos estos abrelatas sin coste alguno. Podemos incluso examinar un tipo y automatizar este proceso.

Hay que señalar que parte del atractivo de `ap` es su capacidad para ejecutar cosas de manera concurrente, por lo que definirla mediante `chain` hace que se pierda esa optimización. A pesar de esto, es bueno tener una interfaz funcionando inmediatamente mientras uno trabaja en la mejor implementación posible.

¿Por qué no utilizar mónadas y así ya estar listos?, te preguntarás. Es una buena práctica trabajar con el nivel de potencia que necesitas en cada momento, ni más, ni menos. Al descartar posibles funcionalidades, mantenemos la carga cognitiva al mínimo. Es por esto que es bueno favorecer a los aplicativos por encima de las mónadas.

Las mónadas tienen la capacidad única de secuenciar el cálculo, asignar variables, y detener la ejecución siguiente, todo ello gracias a la estructura de anidamiento descendente. Cuando vemos que se usan aplicativos, no tenemos que estar atentos a nada de eso.

Y ahora, sobre los aspectos legales...

Leyes

Al igual que el resto de construcciones matemáticas que hemos explorado, los funtores aplicativos tienen algunas propiedades que pueden sernos útiles en nuestro día a día programando. En primer lugar, debes saber que los aplicativos están "cerrados bajo composición", lo que significa que `ap` nunca nos cambiará el tipo de los contenedores (otra razón más para favorecerlos por encima de las mónadas). Eso no quiere decir que no podamos tener múltiples efectos diferentes; podemos apilar nuestros tipos sabiendo que seguirán siendo los mismos durante toda nuestra aplicación.

Para demostrarlo:

```
const tOfM = compose(Task.of, Maybe.of);

liftA2(liftA2(concat), tOfM('Rainy Days and Mondays'), tOfM(' always get me down'))
// Task(Maybe(Rainy Days and Mondays always get me down))
```

Lo ves, no hay que preocuparse de que los diferentes tipos se mezclen.

Llega el momento de ver nuestra ley favorita de la teoría de categorías: *identidad*:

Identidad

```
// identidad
A.of(id).ap(v) === v;
```

Bien, así que aplicar `id` desde el interior de un funtor no debería alterar el valor en `v`. Por ejemplo:

```
const v = Identity.of('Pillow Pets');
Identity.of(id).ap(v) === v;
```

`Identity.of(id)` me hace reír por lo inútil que es. De todos modos, lo que resulta aquí interesante, como ya establecimos antes, es que `of/ap` es lo mismo que `map`, por lo que esta ley se deduce directamente de la identidad del funtor:

```
map(id) == id .
```

La belleza de usar estas leyes es que, igual que un entrenador de gimnasia de guardería, obligan a todas nuestras interfaces a jugar bien entre ellas.

Homomorfismo

```
// Homomorfismo
A.of(f).ap(A.of(x)) === A.of(f(x));
```

Un *homomorfismo* tan solo es un map que preserva la estructura. De hecho, un funtor solo es un *homomorfismo* entre categorías, ya que preserva la estructura original de la categoría que está siendo mapeada.

Realmente, tan solo estamos metiendo nuestras funciones y valores normales en un contenedor y ejecutando el cálculo en su interior, así que no debería sorprendernos que alcancemos el mismo resultado si aplicamos todo dentro del contenedor (lado izquierdo de la ecuación) o lo aplicamos fuera y luego lo colocamos dentro (lado derecho).

Un ejemplo rápido:

```
Either.of(toUpperCase).ap(Either.of('oreos')) === Either.of(toUpperCase('oreos'))
```

Intercambio

La ley del *intercambio* establece que no importa si elegimos levantar nuestra función en el lado izquierdo o en el derecho de `ap`.

```
// intercambio
v.ap(A.of(x)) === A.of(f => f(x)).ap(v);
```

He aquí un ejemplo:

```
const v = Task.of(reverse);
const x = 'Sparklehorse';

v.ap(Task.of(x)) === Task.of(f => f(x)).ap(v);
```

Composición

Y, por último, la composición, que no es más que una manera de comprobar que nuestra composición estándar de funciones se mantiene cuando se aplica dentro de los contenedores.

```
// composición
A.of(compose).ap(u).ap(v).ap(w) === u.ap(v.ap(w));
```

```
const u = IO.of(toUpperCase);
const v = IO.of(concat(' & beyond'));
const w = IO.of('blood bath ');

IO.of(compose).ap(u).ap(v).ap(w) === u.ap(v.ap(w));
```

En Resumen

Un buen caso de uso para los aplicativos es cuando tenemos múltiples argumentos de functor. Nos dan la posibilidad de aplicar funciones a los argumentos todo dentro del mundo de los funtores. Aunque ya podíamos hacer esto con las mónadas, preferiremos los funtores aplicativos cuando no necesitemos ninguna funcionalidad monádica específica.

Casi hemos terminado con las apis de contenedores. Hemos aprendido a como aplicar `map`, `chain`, y ahora `ap`, a funciones. En el próximo capítulo aprenderemos a como trabajar mejor con múltiples funtores y a como desmontarlos siguiendo unos principios.

[Capítulo 11: Transforma Otra Vez, Naturalmente](#)

Ejercicios

Exercise

Escribe una función, utilizando ``Maybe`` y ``ap``, que sume dos números que pueden ser null.

```
// safeAdd :: Maybe Number -> Maybe Number -> Maybe Number
const safeAdd = undefined;
```

Exercise

Reescribe ``safeAdd`` de `exercise_a` para que utilice ``liftA2`` en vez de ``ap``.

```
// safeAdd :: Maybe Number -> Maybe Number -> Maybe Number
const safeAdd = undefined;
```

Para el próximo ejercicio, tendremos en cuenta las siguientes funciones de soporte:

```
const localStorage = {  
  player1: { id:1, name: 'Albert' },  
  player2: { id:2, name: 'Theresa' },  
};  
  
// getFromCache :: String -> IO User  
const getFromCache = x => new IO(() => localStorage[x]);  
  
// game :: User -> User -> String  
const game = curry((p1, p2) => `${p1.name} vs ${p2.name}`);
```

Exercise

Escribe un IO que tome de la caché tanto a player1 como a player2 y que inicie el juego.

```
// startGame :: IO String  
const startGame = undefined;
```

Capítulo 11: Transforma Otra Vez, Naturalmente

[El título en inglés es 'Transform Again, Naturally' que recuerda a la canción *Alone Again (Naturally)* de Gilbert O'Sullivan]

Estamos a punto de dialogar sobre las *transformaciones naturales* en cuanto a su utilidad práctica en nuestro día a día programando. Sucede que son un pilar de la teoría de categorías y absolutamente indispensables a la hora de aplicar las matemáticas para razonar sobre nuestro código y para refactorizarlo. Como tal, creo que es mi deber informarte sobre la lamentable injusticia que estás a punto de presenciar, indudablemente debido a mi limitado dominio del tema. Empecemos.

Maldice Este Nido

Me gustaría abordar el tema del anidamiento. No el instintivo impulso que sienten quienes están a punto de ser padres cuando limpian y reordenan obsesiva e impulsivamente, sino del... bueno, ahora que lo pienso, eso no está tan lejos de la realidad, tal y como veremos en los próximos capítulos... En cualquier caso, lo que quiero decir con *anidamiento* es cuando se tienen dos o más tipos distintos, todos acurrucados en torno a un valor, acunándolo, por así decirlo, como a un recién nacido.

```
Right(Maybe('b'));  
  
IO(Task(IO(1000)));  
  
[Identity('bee thousand')];
```

Hasta ahora hemos logrado, mediante ejemplos cuidadosamente elaborados, evadirnos de tan típico escenario, pero en la práctica, mientras programamos, los tipos tienden a enredarse entre ellos como el cable de los auriculares en un exorcismo. Si no mantenemos a nuestros tipos meticulosamente bien organizados a medida que avanzamos, nuestro código se leerá más peludo que un hipster en un café de gatos.

Una Comedia de Situación

```
// getValue :: Selector -> Task Error (Maybe String)
// postComment :: String -> Task Error Comment
// validate :: String -> Either ValidationError String

// saveComment :: () -> Task Error (Maybe (Either ValidationError (Task Error Comment)))
const saveComment = compose(
  map(map(map(postComment))),
  map(map(validate)),
  getValue('#comment'),
);
```

La pandilla está aquí al completo, para consternación de nuestra firma de tipos. Permíteme explicar brevemente el código. Con `getValue('#comment')`, que es una acción que recupera el texto de un elemento, comenzamos obteniendo lo proporcionado por el usuario. Ahora bien, cabe la posibilidad de que se produzca un error al buscar el elemento o que la cadena de texto no exista, así que devuelve `Task Error (Maybe String)`. Después de esto, debemos aplicar `map` tanto sobre `Task` como sobre `Maybe` para pasarle el texto a `validate`, quien a su vez nos entrega mediante `Either` un `ValidationError` o nuestro `String`. A continuación, mapeamos durante días para enviar el `String` de nuestro `Task Error (Maybe (Either ValidationError String))` a `postComment`, que nos devuelve el `Task` resultante.

Qué desorden tan espantoso. Un collage de tipos abstractos, expresionismo de tipos amateur, un Pollock polimórfico, un Mondrian monolítico. Hay numerosas soluciones para este problema tan común. Podemos componer los tipos en un monstruoso contenedor, ordenarlos y aplicar `join` sobre algunos, homogeneizarlos, deconstruirlos, etc. En este capítulo nos centraremos en homogeneizarlos mediante *transformaciones naturales*.

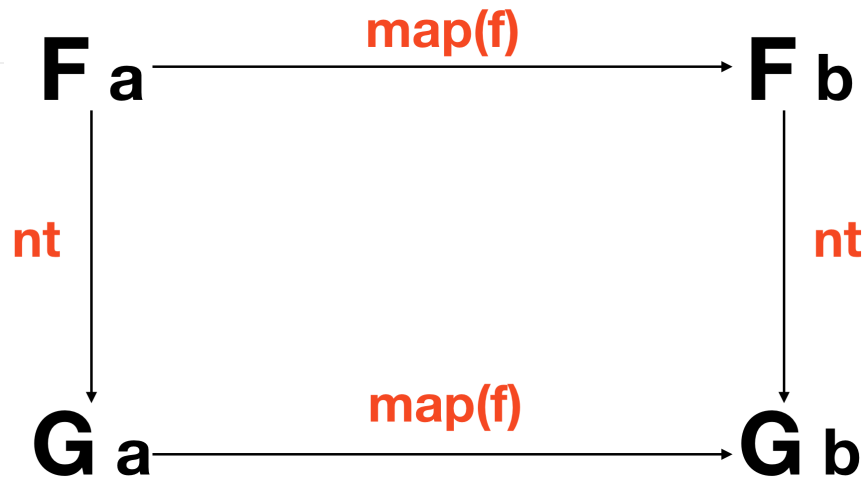
Todo Natural

Una *Transformación Natural* es un "morfismo entre funtores", o sea, una función que opera en los contenedores mismos. Tipológicamente, es una función

```
(Functor f, Functor g) => f a -> g a .
```

Lo que la hace especial es que no podemos, bajo ningún concepto, asomarnos al contenido de nuestro funtor.

Piensa en ello como un intercambio de información clasificada; las dos partes ignoran lo que hay en el sobre de manila sellado con "top secret". Es una operación estructural. Un cambio funcional de vestuario. Formalmente, una *transformación natural* es cualquier función para la que se cumple lo siguiente:



o en código:

```
// nt :: (Functor f, Functor g) => f a -> g a
compose(map(f), nt) === compose(nt, map(f));
```

Tanto el diagrama como el código dicen lo mismo: Podemos ejecutar nuestra transformación natural y luego aplicar `map` o podemos aplicar `map` y luego ejecutar nuestra transformación natural y obtener el mismo resultado. Casualmente esto se desprende de un [teorema gratuito](#), aunque las transformaciones naturales (y los funtores) no están limitadas a funciones sobre tipos.

Conversión de Tipos Basada en Principios

Como programadores estamos familiarizados con la conversión de tipos. Transformamos tipos como `String a Boolean` e `Integer a Float` (aunque JavaScript solo tiene `Number`). Simplemente aquí la diferencia es que estamos trabajando con contenedores algebraicos y tenemos algo de teoría a nuestra disposición.

Veamos ejemplos de algunas de estas conversiones:

```
// idToMaybe :: Identity a -> Maybe a
const idToMaybe = x => Maybe.of(x.$value);

// idToIO :: Identity a -> IO a
const idToIO = x => IO.of(x.$value);

// eitherToTask :: Either a b -> Task a b
const eitherToTask = either(Task.rejected, Task.of);

// ioToTask :: IO a -> Task () a
const ioToTask = x => new Task((reject, resolve) => resolve(x.unsafePerform()));

// maybeToTask :: Maybe a -> Task () a
const maybeToTask = x => (x.isNothing ? Task.rejected() : Task.of(x.$value));

// arrayToMaybe :: [a] -> Maybe a
const arrayToMaybe = x => Maybe.of(x[0]);
```

¿Ves la idea? Solo estamos cambiando un funtor por otro. Se nos permite perder información por el camino, siempre y cuando el valor al que aplicaremos `map` no se pierda con tanto cambio de forma. Ese es el punto: `map` debe continuar, según nuestra definición, incluso después de la transformación.

Una manera de ver todo esto es que estamos transformando a nuestros efectos. Bajo esta luz, podemos ver a `ioToTask` como convertir de síncrono a asíncrono o a `arrayToMaybe` como convertir de no determinístico a posible fallo. Date cuenta que no podemos convertir de asíncrono a síncrono en JavaScript, así que no podemos escribir `taskToIO`; eso sería una transformación supernatural.

Envidia de Características

Supongamos que queremos utilizar algunas características de otro tipo, como por ejemplo `sortBy` en un `List`. Las *Transformaciones naturales* proporcionan una buena forma de convertir al tipo objetivo sabiendo que nuestro `map` será válido.

```
// arrayToList :: [a] -> List a
const arrayToList = List.of;

const doListyThings = compose(sortBy(h), filter(g), arrayToList, map(f));
const doListyThings_ = compose(sortBy(h), filter(g), map(f), arrayToList); // ...
```

Un movimiento de nuestra nariz, tres toques de nuestra varita, dejamos caer `arrayToList`, y ¡voilà!, nuestro `[a]` es un `List a` y podemos utilizar `sortBy` si queremos.

Además, se vuelve más fácil de optimizar / fusionar operaciones al mover

`map(f)` hacia la izquierda de la *transformación natural*, como se muestra en `doListyThings_`.

JavaScript Isomórfico

Cuando podemos ir completamente hacia atrás y hacia adelante sin perder ninguna información, se considera que es un *isomorfismo*. Esta solo es una palabra elegante para decir que "mantiene los mismos datos". Decimos que dos tipos son *isomorfos* si podemos proporcionar las *transformaciones naturales* "hacia" y "desde" como demuestra:

```
// promiseToTask :: Promise a b -> Task a b
const promiseToTask = x => new Task((reject, resolve) => x.then(resolve).catch(reject))

// taskToPromise :: Task a b -> Promise a b
const taskToPromise = x => new Promise((resolve, reject) => x.fork(reject, resolve))

const x = Promise.resolve('ring');
taskToPromise(promiseToTask(x)) === x;

const y = Task.of('rabbit');
promiseToTask(taskToPromise(y)) === y;
```

Q.E.D. `Promise` y `Task` son *isomorfos*. También podemos escribir una función `listToArray` para complementar nuestra `arrayToList` y demostrar que también lo son. Como contraejemplo, `arrayToMaybe` no es un *isomorfismo* dado que pierde información:

```
// maybeToArray :: Maybe a -> [a]
const maybeToArray = x => (x.isNothing ? [] : [x.$value]);

// arrayToMaybe :: [a] -> Maybe a
const arrayToMaybe = x => Maybe.of(x[0]);

const x = ['elvis costello', 'the attractions'];

// no isomorfa
maybeToArray(arrayToMaybe(x)); // ['elvis costello']

// pero es una transformación natural
compose(arrayToMaybe, map(replace('elvis', 'lou')))(x); // Just('lou costello')
// ==
compose(map(replace('elvis', 'lou')), arrayToMaybe)(x); // Just('lou costello')
```

Sin embargo, sí que son *transformaciones naturales*, ya que la función `map` de cada lado da el mismo resultado. Menciono los *isomorfismos* aquí, a mitad del capítulo, porque estamos hablando de ello, pero no te dejes engañar, son un concepto enormemente poderoso y omnipresente. De todos modos, sigamos adelante.

Una Definición Más Amplia

Estas funciones estructurales no se limitan en absoluto a la conversión de tipos.

He aquí otras distintas:

```
reverse :: [a] -> [a]

join :: (Monad m) => m (m a) -> m a

head :: [a] -> a

of :: a -> f a
```

Las leyes de la transformación natural también son válidas para estas funciones.

Una cosa que puede confundirte es que `head :: [a] -> a` puede verse como `head :: [a] -> Identity a`. Somos libres de insertar `Identity` donde queramos mientras demostramos las leyes, ya que podemos a su vez demostrar que `a` es isomorfo con `Identity a` (lo ves, te dije que los *isomorfismos* eran omnipresentes).

Una Solución Para El Anidamiento

Volviendo a nuestra cómica firma de tipos. Para aplicar coerción de tipos a cada tipo que varíe podemos espolvorear en ella algunas *transformaciones naturales* a lo largo del código que la llama para que así sean todos uniformes y que, por lo tanto, se puedan unir con `join`.

```
// getValue :: Selector -> Task Error (Maybe String)
// postComment :: String -> Task Error Comment
// validate :: String -> Either ValidationError String

// saveComment :: () -> Task Error Comment
const saveComment = compose(
  chain(postComment),
  chain(eitherToTask),
  map(validate),
  chain(maybeToTask),
  getValue('#comment'),
);
```

¿Qué tenemos aquí? Tan solo hemos añadido `chain(maybeToTask)` y `chain(eitherToTask)`. Ambas tienen el mismo efecto; de manera natural, transforman el funtor que guarda nuestro `Task`, en otro `Task` y luego une los dos con `join`. De esta manera evitamos el anidamiento justo desde el origen igual que los pinchos para palomas en el alféizar de una ventana. Como dicen en la Ciudad de la Luz, "Mieux vaut prévenir que guérir"; más vale prevenir que curar.

En Resumen

Las *Transformaciones Naturales* son funciones sobre nuestros funtores. Son un concepto extremadamente importante en la teoría de categorías y empezarán a aparecer por todas partes una vez que adoptemos más abstracciones, pero, por ahora, las hemos limitado a unas pocas aplicaciones concretas. Como hemos visto, podemos conseguir diferentes efectos al convertir tipos, con la garantía de que mantendremos nuestra composición. También pueden ayudarnos con el anidamiento de tipos, aunque tienen el efecto general de homogeneizar nuestros funtores al mínimo común denominador que, en la práctica, es el funtor con los efectos más volátiles (`Task` en la mayoría de los casos).

Este continuo y tedioso ordenamiento de tipos es el precio que pagamos por haberlos materializado; convocados desde el éter. Por supuesto, los efectos implícitos son mucho más insidiosos, así que aquí estamos, librando esta justa batalla. Necesitaremos algunas herramientas más en nuestro equipamiento antes de poder enrollar las más largas amalgamas de tipos. A continuación, veremos cómo reordenar nuestros tipos con *Traversable*.

[Capítulo 12: Atravesando la Piedra](#)

Ejercicios

Exercise

Escribe una transformación natural que convierta `Either b a` en `Maybe a`

```
// eitherToMaybe :: Either b a -> Maybe a
const eitherToMaybe = undefined;
```

```
// eitherToTask :: Either a b -> Task a b
const eitherToTask = either(Task.rejected, Task.of);
```

Exercise

Utilizando `eitherToTask`, simplifica `findNameById` para eliminar el `Either` anidado.

```
// findNameById :: Number -> Task Error (Either Error User)
const findNameById = compose(map(map(prop('name'))), findUserById);
```

Como recordatorio, las siguientes funciones están disponibles en el contexto del ejercicio:

```
split :: String -> String -> [String]
intercalate :: String -> [String] -> String
```

Exercise

Escribe los isomorfismos entre String y [Char].

```
// strToList :: String -> [Char]
const strToList = undefined;

// listToStr :: [Char] -> String
const listToStr = undefined;
```

Capítulo 12: Atravesando la Piedra

Hasta ahora, en nuestro circo de contenedores, nos has visto domar al feroz **funtor**, doblegándolo a nuestra voluntad para realizar cualquier operación que se nos antojara. Te han deslumbrado los malabares hechos simultáneamente con múltiples efectos peligrosos, utilizando **aplicación** de funciones para reunir los resultados. Presenciaste con asombro como los contenedores se desvanecían en el aire al ser **unidos** entre ellos. En el espectáculo de efectos secundarios, los vimos **componerse** en uno solo. Y más recientemente, nos aventuramos más allá de lo natural y **transformamos** un tipo en otro ante tus propios ojos.

Y ahora, para nuestro siguiente truco, veremos los "traversables". Veremos tipos volar unos sobre otros como si fuesen trapevistas, manteniendo nuestro valor intacto. Reordenaremos los efectos como a las cabinas de una atracción de feria. Cuando nuestros contenedores se entrelacen como las extremidades de un contorsionista podremos utilizar esta interfaz para enderezar las cosas. Con distintas disposiciones presenciaremos distintos efectos. Tráeme mis bombachos y mi flauta de émbolo, comencemos.

Tipos y Tipos

Pongámonos raros:

```
// readFile :: FileName -> Task Error String

// firstWords :: String -> String
const firstWords = compose(intercalate(' '), take(3), split(' '));

// tldr :: FileName -> Task Error String
const tldr = compose(map(firstWords), readFile);

map(tldr, ['file1', 'file2']);
// [Task('hail the monarchy'), Task('smash the patriarchy')]
```

Aquí estamos leyendo un grupo de archivos y terminamos con un inútil array de tareas. ¿Cómo podríamos ejecutar cada una de ellas? Sería de lo más agradable si pudiésemos cambiar los tipos de sitio para tener `Task Error [String]` en vez de `[Task Error String]`. De esta manera tendríamos un valor futuro conteniendo todos los resultados, que es más agradable para nuestras necesidades asíncronas que varios valores futuros llegando a su antojo.

He aquí un último ejemplo de una situación complicada:

```
// getAttribute :: String -> Node -> Maybe String
// $ :: Selector -> IO Node

// getControlNode :: Selector -> IO (Maybe (IO Node))
const getControlNode = compose(map(map($)), map(getAttribute('aria-controls'))),
```

Mira esos `IO` anhelando estar juntos. Sería simplemente encantador poderlos unir con `join` permitiéndoles bailar mejilla con mejilla, pero, por desgracia, un `Maybe` se interpone entre ellos como una carabina en el baile de graduación. Nuestro mejor movimiento aquí sería colocarlos uno junto al otro para que sus tipos estuviesen al fin juntos, y así simplificar nuestra firma a `IO (Maybe Node)`.

Feng Shui de Tipos

La interfaz *Traversable* consiste en dos gloriosas funciones: `sequence` y `traverse`.

Reordenemos nuestros tipos utilizando `sequence`:

```
sequence(List.of, Maybe.of(['the facts'])); // [Just('the facts')]
sequence(Task.of, new Map({ a: Task.of(1), b: Task.of(2) })); // Task(Map({ a:
sequence(IO.of, Either.of(IO.of('buckle my shoe')))); // IO(Right('buckle my sho
sequence(Either.of, [Either.of('wing')])); // Right(['wing'])
sequence(Task.of, left('wing')); // Task(Left('wing'))
```

¿Ves lo que ha ocurrido aquí? Nuestro tipo con anidamiento es dado la vuelta como a unos pantalones de piel en una húmeda noche de verano. El funtor de dentro es movido hacia el exterior y viceversa. Has de saber que `sequence` es un poco particular en cuanto a sus argumentos. Tiene el siguiente aspecto:

```
// sequence :: (Traversable t, Applicative f) => (a -> f a) -> t (f a) -> f (t
const sequence = curry((of, x) => x.sequence(of));
```

Comencemos por el segundo argumento. Ha de ser un *Traversable* conteniendo un *Aplicativo* que, aun sonando bastante restrictivo, suele ser lo más común. Es el `t (f a)` quien es transformado en `f (t a)`. ¿No es expresivo? Queda claro como el agua que los dos tipos bailan dos-à-dos el uno alrededor del otro. El primer argumento es tan solo una ayuda y solo es necesario en un lenguaje sin tipos. Es un constructor de tipo (nuestro *of*) proporcionado para que podamos invertir tipos reacios a `map`, como `Left`; más sobre esto en un minuto.

Utilizando `sequence` podemos mover tipos de un lado a otro con la precisión de un trilerero. Pero ¿cómo funciona esto? Veamos como un tipo, digamos `Either`, la implementaría.

```
class Right extends Either {  
  // ...  
  sequence(of) {  
    return this.$value.map(Either.of);  
  }  
}
```

Ah, sí, si nuestro valor `$value` es un funtor (de hecho debe ser un aplicativo), podemos simplemente aplicarle nuestro constructor mediante `map` para que salte por encima del tipo.

Puede que te hayas dado cuenta de que hemos ignorado por completo el `of`. Se pasa como argumento para cuando el mapeo es inútil, como es en el caso de `Left`:

```
class Left extends Either {  
  // ...  
  sequence(of) {  
    return of(this);  
  }  
}
```

Queremos que los tipos acaben siempre en la misma disposición, por lo que es necesario que tipos como `Left`, que no contienen a nuestro aplicativo interno, reciban algo de ayuda para hacerlo. La interfaz *Aplicativo* requiere que primero tengamos un *Functor Pointed* para que siempre tengamos un `of` que pasar. En un lenguaje con sistema de tipos, el tipo externo puede ser inferido de la firma y no necesita ser proporcionado explícitamente.

Surtido de Efectos

Distintas disposiciones tienen distintos resultados en cuanto a nuestros contenedores se refiere. Si tengo `[Maybe a]`, es una colección de posibles valores mientras que si tengo un `Maybe [a]`, es una posible colección de valores. Lo primero indica que seremos indulgentes y nos quedaremos con "los buenos", mientras que lo último significa que es una situación del tipo "todo o nada". De igual manera, `Either Error (Task Error a)` puede representar una validación del lado del cliente y `Task Error (Either Error a)` puede ser del lado del servidor. Los tipos pueden ser intercambiados para proporcionarnos diferentes efectos.

```
// fromPredicate :: (a -> Bool) -> a -> Either e a

// partition :: (a -> Bool) -> [a] -> [Either e a]
const partition = f => map(fromPredicate(f));

// validate :: (a -> Bool) -> [a] -> Either e [a]
const validate = f => traverse(Either.of, fromPredicate(f));
```

Aquí tenemos dos funciones distintas según se basan en aplicar `map` o `traverse`. La primera, `partition`, nos dará un array de `Left`s y `Right`s de acuerdo con la función predicado. Esto es útil para mantener los preciosos datos a mano para futuros usos en vez de descartarlos junto con el agua del baño. En cambio, `validate` nos devolverá en `Left` el primer elemento que no supere el predicado, o todos los elementos en un `Right` si todo está bien. Al escoger un orden diferente de tipos, obtenemos un comportamiento diferente:

Veamos la función `traverse` de `List` para ver como está hecho el método `validate`.

```
traverse(of, fn) {
  return this.$value.reduce(
    (f, a) => fn(a).map(b => bs => bs.concat(b)).ap(f),
    of(new List([])),
  );
}
```

Esto tan solo ejecuta un `reduce` en la lista. La función `reduce` es `(f, a) => fn(a).map(b => bs => bs.concat(b)).ap(f)`, que da algo de miedo, así que veámosla paso a paso.

1. `reduce(..., ...)`

Recuerda la firma de `reduce :: [a] -> (f -> a -> f) -> f -> f`. El primer argumento es en realidad proporcionado por la notación con punto en `$value`, así que es una lista de cosas. Después necesitamos una función desde un `f` (el acumulador) y un `a` (el iterado) para devolvernos un nuevo acumulador.

2. `of(new List([]))`

El valor semilla es `of(new List([]))`, el cual en nuestro caso es `Right([]) :: Either e [a]`. ¡Fíjate que `Either e [a]` también será nuestro tipo resultante!

3. `fn :: Applicative f => a -> f a`

Si lo aplicamos a nuestro ejemplo de arriba, `fn` es en realidad

```
fromPredicate(f) :: a -> Either e a
```

```
fn(a) :: Either e a
```

4. `.map(b => bs => bs.concat(b))`

Cuando es `Right`, `Either.map` pasa el valor correcto a la función y devuelve un nuevo `Right` con el resultado. En este caso, la función tiene un parámetro (`b`), y devuelve otra función (`bs => bs.concat(b)`), donde `b` está al alcance gracias a la closure). Cuando es `Left`, el valor de `left` es devuelto.

```
fn(a).map(b => bs => bs.concat(b)) :: Either e ([a] -> [a])
```

5. `.ap(f)`

Recuerda que aquí `f` es un Aplicativo, así que podemos aplicar la función `bs => bs.concat(b)` a cualquier valor `bs :: [a]` que esté en `f`. Afortunadamente, `f` proviene de nuestra semilla inicial y tiene el siguiente tipo: `f :: Either e [a]` que, por cierto, se conserva cuando aplicamos `bs => bs.concat(b)`. Cuando `f` es `Right` llama a `bs => bs.concat(b)`, quien a su vez devuelve un `Right` con el elemento añadido a la lista. Cuando es `Left`, el valor izquierdo (del paso anterior o de la iteración anterior respectivamente) es devuelto.

```
fn(a).map(b => bs => bs.concat(b)).ap(f) :: Either e [a]
```

Esta transformación aparentemente milagrosa se consigue con tan solo 6 miserables líneas de código en `List.traverse`, y se logra con `of`, `map` y `ap` por lo que funcionará para cualquier Funtor Aplicativo. Este es un gran ejemplo de cómo estas abstracciones pueden ayudar a escribir código altamente genérico con solo unas pocas suposiciones (¡que pueden, por cierto, ser declaradas y comprobadas a nivel de tipos!)

El Vals de los Tipos

Es momento de revisitar y limpiar nuestros ejemplos iniciales.

```
// readFile :: FileName -> Task Error String

// firstWords :: String -> String
const firstWords = compose(intercalate(' '), take(3), split(' '));

// tldr :: FileName -> Task Error String
const tldr = compose(map(firstWords), readFile);

traverse(Task.of, tldr, ['file1', 'file2']);
// Task(['hail the monarchy', 'smash the patriarchy']);
```

Utilizando `traverse` en vez de `map`, hemos conseguido formar un rebaño con esas revoltosas `Task`, convirtiéndolas en un bonito y coordinado array de resultados. Esto es como `Promise.all()`, si estás familiarizado, excepto que no es una única función personalizada, no, esto funciona para cualquier tipo

traversable. Estas apis matemáticas tienden a capturar de forma interoperable y reusable la mayor parte de las cosas que nos gustaría hacer, en vez de que cada librería reinvente estas funciones para un solo tipo.

Limpiemos el último ejemplo de closure:

```
// getAttribute :: String -> Node -> Maybe String
// $ :: Selector -> IO Node

// getControlNode :: Selector -> IO (Maybe Node)
const getControlNode = compose(chain(traverse(IO.of, $)), map(getAttribute('ar:

```

En vez de `map(map($))` tenemos `chain(traverse(IO.of, $))`, que invierte nuestros tipos dado que, mediante `chain`, aplica `map` y luego aplana los dos `IO`.

Sin Ley Ni Orden

Bien, ahora, antes de que te pongas a juzgar y golpees la tecla de borrar como con un mazo para olvidar el capítulo, tómame un momento para reconocer que todas estas leyes son útiles garantías de código. Es conjetura mía que la finalidad de las arquitecturas de muchos programas es intentar poner restricciones útiles en nuestro código para así reducir las posibilidades, para guiarnos hacia las respuestas cuando lo diseñamos y cuando lo leemos.

Una interfaz sin leyes es simple indirección. Como cualquier otra estructura matemática, debemos exponer las propiedades para nuestra propia cordura. Esto tiene un efecto similar a la encapsulación, dado que protege a los datos, permitiéndonos intercambiar la interfaz por otro ciudadano ejemplar.

Acompañame, tenemos algunas leyes que averiguar.

Identidad

```
const identity1 = compose(sequence(Identity.of), map(Identity.of));
const identity2 = Identity.of;

// pruébalo con Right
identity1(Either.of('stuff'));
// Identity(Right('stuff'))

identity2(Either.of('stuff'));
// Identity(Right('stuff'))
```

Esto debería ser sencillo. Si colocamos un `Identity` dentro de nuestro funtor, y luego le damos la vuelta con `sequence` es lo mismo que colocarlo por fuera desde el principio. Hemos elegido a `Right` como conejillo de indias porque con

él es fácil probar a aplicar la ley e inspeccionarlo. Podríamos haber usado cualquier otro funtor, sin embargo, el uso de un funtor concreto como `Identity` en la ley misma, puede que haya levantado algunas cejas. Recuerda que una [categoría](#) se define como morfismos entre sus objetos con composición asociativa e identidad. Cuando se trata de la categoría de funtores, las transformaciones naturales son los morfismos e `Identity` es, bueno, la identidad. El funtor `Identity` es tan fundamental para demostrar las leyes como nuestra función `compose`. De hecho, deberíamos dejar aquí este tema y pasar a hacer lo mismo con nuestro tipo [Compose](#):

Composición

```
const comp1 = compose(sequence(Compose.of), map(Compose.of));
const comp2 = (Fof, Gof) => compose(Compose.of, map(sequence(Gof)), sequence(Fof));

// Pruébalo con algunos tipos que tengamos por ahí
comp1(Identity(Right([true])));
// Compose(Right([Identity(true)]))

comp2(Either.of, Array)(Identity(Right([true])));
// Compose(Right([Identity(true)]))
```

Tal y como podríamos esperar, esta ley preserva la composición: si intercambiamos la composición de funtores, no deberíamos tener ninguna sorpresa dado que la composición es un funtor en sí mismo. Arbitrariamente hemos escogido `true`, `Right`, `Identity` y `Array` para probarlo. Librerías como [quickcheck](#) o [jsverify](#) pueden ayudarnos a comprobar la ley mediante pruebas con datos aleatorios en las entradas.

Como consecuencia natural de la ley de arriba, obtenemos la capacidad de [fusionar traversals](#), lo que es bueno desde el punto de vista del rendimiento.

Naturalidad

```
const natLaw1 = (of, nt) => compose(nt, sequence(of));
const natLaw2 = (of, nt) => compose(sequence(of), map(nt));

// test con una transformación natural al azar y nuestros amigables funtores Id

// maybeToEither :: Maybe a -> Either () a
const maybeToEither = x => (x.$value ? new Right(x.$value) : new Left());

natLaw1(Maybe.of, maybeToEither)(Identity.of(Maybe.of('barlow one')));
// Right(Identity('barlow one'))

natLaw2(Either.of, maybeToEither)(Identity.of(Maybe.of('barlow one')));
// Right(Identity('barlow one'))
```

Esto es parecido a nuestra ley de la identidad. Si primero hacemos girar a los tipos y luego ejecutamos una transformación natural en el exterior, debería ser lo mismo que mapear una transformación natural y después voltear los tipos.

Una consecuencia natural de esta ley es:

```
traverse(A.of, A.of) === A.of;
```

Lo cual, de nuevo, es bueno desde el punto de vista del rendimiento.

En Resumen

Traversable es una poderosa interfaz que nos provee con la capacidad de reordenar nuestros tipos con la facilidad de un interiorista con telequinesis. Con distintas disposiciones podemos conseguir distintos efectos, así como planchar esas feas arrugas en los tipos que nos impiden unirlos con `join`. A continuación, nos desviaremos un poco para ver una de las interfaces más poderosas de la programación funcional y puede que incluso del propio álgebra: [Los Monoides lo unen todo](#)

Ejercicios

Teniendo en cuenta los siguientes elementos:

```
// httpGet :: Route -> Task Error JSON

// routes :: Map Route Route
const routes = new Map({ '/': '/', '/about': '/about' });
```

Exercise

Utiliza la interfaz `Traversable` para cambiar la firma de tipo de `getJsons`` a `Map Route Route → Task Error (Map Route JSON)`

```
// getJsons :: Map Route Route -> Map Route (Task Error JSON)
const getJsons = map(httpGet);
```

Ahora definimos la siguiente función de validación:

```
// validate :: Player -> Either String Player
const validate = player => (player.name ? Either.of(player) : left('must have name'))
```

Exercise

Usando `Traversable` y la función `validate``, actualiza `startGame`` (y su firma) para que solo comience el juego si todos los jugadores son válidos

```
// startGame :: [Player] -> [Either Error String]
const startGame = compose(map(map(always('game started!'))), map(validate))
```

Finalmente, teniendo en cuenta algunas funciones de soporte para el sistema de archivos:

```
// readFile :: String -> String -> Task Error String
// readdir :: String -> Task Error [String]
```

Exercise

Utiliza `Traversable` para reordenar y aplanar los `Task` y `Maybe` anidados

```
// readFirst :: String -> Task Error (Maybe (Task Error String))
const readFirst = compose(map(map(readfile('utf-8'))), map(safeHead), readDir)
```

Capítulo 13: Los Monoides Lo Unen Todo

Salvaje Combinación

En este capítulo examinaremos los *monoides* utilizando *semigrupos*. Los *monoides* son el chicle en el pelo de la abstracción matemática. Capturan una idea que abarca múltiples disciplinas, uniéndolas figurativa y literalmente todas en una. Son la fuerza ominosa que conecta todo aquello que tiene la capacidad de calcular. Son el oxígeno en nuestra base de código, la tierra sobre la que se ejecuta, entrelazamiento cuántico codificado.

Los *monoides* tratan sobre combinar. Pero, ¿qué es combinar? Puede significar muchas cosas, desde acumulación hasta concatenación pasando por multiplicación o elección, composición, ordenación, ¡incluso evaluación! Veremos numerosos ejemplos, pero tan solo pasaremos de soslayo por la falda de la montaña de los monoides. Los ejemplares son abundantes y las aplicaciones, enormemente amplias. El objetivo de este capítulo es proporcionarte una buena intuición para que puedas crear tus propios *monoides*.

Abstrayendo La Suma

La suma tiene algunas interesantes cualidades que me gustaría discutir. Démosle un vistazo a través de nuestras gafas de abstracción.

Para empezar, es una operación binaria, o sea, es una operación que toma dos valores y devuelve uno solo, todo dentro del mismo conjunto.

```
// una operación binaria  
1 + 1 = 2
```

¿Lo ves? Dos valores en el dominio, un valor en el codominio, todos del mismo conjunto; números, por así decirlo. Algunos dirían que los números están "cerrados bajo la suma", queriendo decir que el tipo nunca cambiará, sin importar cuál se eche en la mezcla. Esto quiere decir que podemos encadenar la operación, puesto que el resultado siempre es otro número:

```
// podemos ejecutar esto con cualquier cantidad de números  
1 + 7 + 5 + 4 + ...
```

Además, tenemos la asociatividad, que nos da la capacidad de agrupar operaciones como nos plazca. Por cierto, una operación binaria asociativa es la receta para la computación paralela porque podemos trocear y distribuir el trabajo.

```
// asociatividad
(1 + 2) + 3 = 6
1 + (2 + 3) = 6
```

No vayas a confundir esto con conmutatividad, la cual nos permite cambiar el orden. Aunque se mantiene para la suma, ahora mismo no estamos especialmente interesados en esta propiedad; demasiado específica para nuestras necesidades de abstracción.

Ahora que lo pienso, ¿qué propiedades deben estar si o si en nuestra superclase abstracta? ¿Qué rasgos son específicos de la suma y cuáles pueden ser generalizados? ¿Hay otras abstracciones en medio de esta jerarquía o es todo un mismo trozo? Este es el tipo de razonamiento que nuestros antepasados matemáticos aplicaban cuando concibieron las interfaces en el álgebra abstracta.

Como era de esperar, cuando estos "abstraccionistas" de la vieja escuela abstraieron la suma llegaron al concepto de *grupo*. Un *grupo* tiene de todo, incluyendo el concepto de números negativos. Ahora mismo solo estamos interesados en el operador binario asociativo así que elegiremos una interfaz menos específica, *Semigrupo*. Un *Semigrupo* es un tipo con un método `concat` que hace de operador binario asociativo.

Implementémoslo para la suma y llamémosle `Sum` :

```
const Sum = x => ({
  x,
  concat: other => Sum(x + other.x)
})
```

Fíjate que con `concat` concatenamos con otro `Sum` y siempre devolvemos un `Sum` .

He utilizado una factoría en vez de nuestra típica ceremonia con el prototipo, principalmente porque `Sum` no es *pointed* y no queremos tener que teclear `new` . De todos modos, aquí está en acción:

```
Sum(1).concat(Sum(3)) // Sum(4)
Sum(4).concat(Sum(37)) // Sum(41)
```

Así podemos programar para la interfaz, no para la implementación. Dado que esta interfaz viene de la teoría de grupos, tiene siglos de literatura respaldándola. ¡Documentación sin esfuerzo adicional!

Como mencionaba antes, `Sum` no es *pointed*, y tampoco es un *functor*. Como ejercicio, vuelve atrás y comprueba las leyes para ver por qué. Vale, yo te lo diré: únicamente puede mantener un número, así que `map` no tiene sentido aquí dado que no podemos transformar al valor subyacente en otro tipo. ¡Ese sería un `map` muy limitado de hecho!

Y entonces, ¿por qué es útil? Bien, como con cualquier interfaz, podemos cambiar nuestra implementación para conseguir distintos resultados:

```
const Product = x => ({ x, concat: other => Product(x * other.x) })

const Min = x => ({ x, concat: other => Min(x < other.x ? x : other.x) })

const Max = x => ({ x, concat: other => Max(x > other.x ? x : other.x) })
```

Y esto no está limitado a números. Veamos otros tipos:

```
const Any = x => ({ x, concat: other => Any(x || other.x) })
const All = x => ({ x, concat: other => All(x && other.x) })

Any(false).concat(Any(true)) // Any(true)
Any(false).concat(Any(false)) // Any(false)

All(false).concat(All(true)) // All(false)
All(true).concat(All(true)) // All(true)

[1,2].concat([3,4]) // [1,2,3,4]

"miracle grow".concat("n") // miracle grown"

Map({day: 'night'}).concat(Map({white: 'nikes'})) // Map({day: 'night', white:
```

Si los miras fijamente el tiempo suficiente, el patrón aparecerá como en un estereograma. Está en todas partes. Estamos fusionando estructuras de datos, combinando lógica, construyendo cadenas de texto... Parece que podemos meter a golpes casi cualquier tarea dentro de esta interfaz basada en la combinación.

Ya he usado `Map` unas cuantas veces. Perdóname si no te lo he presentado adecuadamente. `Map` tan solo envuelve a `Object` para así poder embellecerlo con algunos métodos extra sin alterar el tejido del universo.

Todos Mis Funtores Favoritos Son Semigrupos

Los tipos que hemos visto hasta ahora y que implementan la interfaz functor también implementan la interfaz semigrupo. Veamos a `Identity` (el artista antes conocido como Contenedor):

```
Identity.prototype.concat = function(other) {
  return new Identity(this.__value.concat(other.__value))
}

Identity.of(Sum(4)).concat(Identity.of(Sum(1))) // Identity(Sum(5))
Identity.of(4).concat(Identity.of(1)) // TypeError: this.__value.concat is not
```

Es un *semigrupo* si y solo si su valor `__value` es un *semigrupo*. Como un torpe piloto de ala delta, solo lo es mientras está agarrado a una.

Otros tipos tienen un comportamiento similar:

```
// combinar con manejo de errores
Right(Sum(2)).concat(Right(Sum(3))) // Right(Sum(5))
Right(Sum(2)).concat(Left('some error')) // Left('some error')

// combinar asincronia
Task.of([1,2]).concat(Task.of([3,4])) // Task([1,2,3,4])
```

Esto es particularmente útil cuando apilamos estos semigrupos en una combinación en cascada:

```
// formValues :: Selector -> IO (Map String String)
// validate :: Map String String -> Either Error (Map String String)

formValues('#signup').map(validate).concat(formValues('#terms').map(validate))
formValues('#signup').map(validate).concat(formValues('#terms').map(validate))

serverA.get('/friends').concat(serverB.get('/friends')) // Task([friend1, friend2])

// loadSetting :: String -> Task Error (Maybe (Map String Boolean))
loadSetting('email').concat(loadSetting('general')) // Task(Maybe(Map({background}))
```

En el ejemplo de arriba, para validar y fusionar los valores del formulario hemos combinado un `IO` que contiene un `Either` que a su vez contiene un `Map`. Después hemos llamado a un par de servidores distintos y hemos combinado sus resultados de manera asíncrona utilizando `Task` y `Array`. Finalmente hemos apilado `Task`, `Maybe` y `Map` para cargar, parsear y fusionar múltiples ajustes.

Estos ejemplos podrían haber utilizado `chain` o `ap`, pero los *semigrupos* capturan lo que queremos de forma mucho más concisa.

Esto se extiende más allá de los funtores. De hecho, resulta que cualquier cosa hecha enteramente de semigrupos es ella misma un semigrupo.

```
const Analytics = (clicks, path, idleTime) => ({
  clicks,
  path,
  idleTime,
  concat: other =>
    Analytics(clicks.concat(other.clicks), path.concat(other.path), idleTime.co
  })

Analytics(Sum(2), ['/home', '/about'], Right(Max(2000))).concat(Analytics(Sum(
// Analytics(Sum(3), ['/home', '/about', '/contact'], Right(Max(2000)))
```

Como ves, todo sabe como combinarse. Resulta que podemos hacer lo mismo sin ningún esfuerzo adicional con tan solo utilizar el tipo `Map` :

```
Map({clicks: Sum(2), path: ['/home', '/about'], idleTime: Right(Max(2000))}).co
// Map({clicks: Sum(3), path: ['/home', '/about', '/contact'], idleTime: Right(
```

Podemos apilar y combinar tantos como queramos. Solo es cuestión de añadir otro árbol al bosque, u otra llama al incendio del bosque dependiendo de tu base de código.

El comportamiento intuitivo por defecto es combinar lo que el tipo contiene, sin embargo, hay casos en los que ignoramos lo que hay dentro y combinamos el contenedor en sí mismo. Considera un tipo como `Stream` [*Flujo*]:

```
const submitStream = Stream.fromEvent('click', $('#submit'))
const enterStream = filter(x => x.key === 'Enter', Stream.fromEvent('keydown',

submitStream.concat(enterStream).map(submitForm) // Stream()
```

Podemos combinar flujos de eventos [*event streams*] capturando los eventos de ambos flujos en un nuevo flujo. Alternativamente, podríamos haberlos combinado insistiendo en que contienen un semigrupo. De hecho, hay muchos ejemplares posibles para cada tipo. Considera `Task` . Podríamos combinar las tareas eligiendo la que termina más pronto o la que termina más tarde. Siempre podemos elegir el primer `Right` en vez de cortocircuitar con `Left` , haciendo que se ignoren los errores. Hay una interfaz llamada *Alternative* [*Alternativa*] que implementa alguno de estos... bueno... ejemplares alternativos, que típicamente se concentran más en elegir que en combinar en cascada. Vale la pena estudiarla si te ves en la necesidad de una funcionalidad como esta.

Monoides A Cambio De Nada

Estamos abstrayendo la suma pero, como a los babilonios, nos falta el concepto de cero (hubo cero menciones sobre él).

El cero actúa como la *identidad* queriendo decir que cualquier elemento añadido a `0` devolverá ese mismo elemento. En términos de abstracción, sirve de ayuda pensar en el `0` como en un elemento neutro o *vacío*. Es importante el hecho de que actúa de la misma manera tanto en el lado izquierdo como en el lado derecho de nuestra operación binaria:

```
// identidad
1 + 0 = 1
0 + 1 = 1
```

Denominemos `empty` [*vacío*] a este concepto y creemos con él una nueva interfaz. Como tantas startups, escogeremos un abominable y poco informativo a la vez que googleable nombre: *Monoide*. La receta para *Monoide* es coger cualquier *semigrupo* y añadirle un elemento *identidad* especial. Vamos a implementar esto con una función `empty` en el propio tipo:

```
Array.empty = () => []
String.empty = () => ""
Sum.empty = () => Sum(0)
Product.empty = () => Product(1)
Min.empty = () => Min(Infinity)
Max.empty = () => Max(-Infinity)
All.empty = () => All(true)
Any.empty = () => Any(false)
```

¿Cuándo podría demostrar ser útil un valor identidad vacío? Eso es como preguntar por qué el cero es útil. Es como no preguntar nada en absoluto...

Cuando no tenemos nada más, ¿con quién podemos contar? Cero. ¿Cuántos bugs queremos? Cero. Esta es nuestra tolerancia hacia el código no confiable. Un nuevo comienzo. La etiqueta de precio definitiva. Puede aniquilar todo lo que haya en su camino o salvarnos de un apuro. Un salvavidas dorado y un pozo de desesperación.

En cuanto al código, corresponden a sensatos valores por defecto:

```
const settings = (prefix="", overrides=[], total=0) => ...

const settings = (prefix=String.empty(), overrides=Array.empty(), total=Sum.empty()) => ...
```

O para devolver un valor útil cuando no tenemos nada más:

```
sum([]) // 0
```

También son el valor inicial perfecto para un acumulador...

Plegando La Casa

Resulta que `concat` y `empty` encajan perfectamente con los dos primeros huecos de `reduce`. De hecho podemos aplicar `reduce` a un array de *semigrupos* ignorando el valor vacío, pero, como puedes ver, esto conduce a una precaria situación:

```
// concat :: Semigroup s => s -> s -> s
const concat = x => y => x.concat(y)

[Sum(1), Sum(2)].reduce(concat) // Sum(3)

[].reduce(concat) // TypeError: Reduce of empty array with no initial value
```

Y la dinamita explota. Como un tobillo torcido en una maratón, obtenemos un error de ejecución. JavaScript es más que feliz dejando que nos atemos pistolas a nuestras zapatillas deportivas antes de salir a correr; es algo así como un lenguaje conservador, supongo, pero que nos detiene en seco cuando el array es estéril. ¿Qué podría devolver si no? `Nan`, `false`, `-1`? Si fuésemos a continuar con nuestro programa, querríamos un resultado del tipo correcto. Podría devolver un `Maybe` para indicar la posibilidad de fallo, pero podemos hacer algo mejor.

Vamos a utilizar nuestra versión currificada de `reduce` y a hacer una versión segura donde el valor vacío no sea opcional. En lo sucesivo será conocida como `fold`:

```
// fold :: Monoid m => m -> [m] -> m
const fold = reduce(concat)
```

La `m` inicial es nuestro valor vacío; nuestro punto neutro inicial, y luego tomamos un array de `m` s y las aplastamos hasta llegar a un hermoso valor diamantino.

```
fold(Sum.empty(), [Sum(1), Sum(2)]) // Sum(3)
fold(Sum.empty(), []) // Sum(0)

fold(Any.empty(), [Any(false), Any(true)]) // Any(true)
fold(Any.empty(), []) // Any(false)

fold(Either.of(Max.empty()), [Right(Max(3)), Right(Max(21)), Right(Max(11))]) // Right(Max(21))
fold(Either.of(Max.empty()), [Right(Max(3)), Left('error retrieving value'), Right(Max(11))]) // Left('error retrieving value')

fold(IO.of([]), ['.link', 'a'].map($)) // IO([<a>, <button class="link"/>, <a>])
```

Hemos proporcionado manualmente un valor vacío para estos dos últimos, ya que no podemos definir uno en el propio tipo. Eso es totalmente correcto. Los lenguajes tipados pueden averiguarlo por ellos mismos, pero aquí tenemos que pasarlo.

No Un Monoide Exactamente

Hay algunos *semigrupos* que no pueden convertirse en *monoides*, o sea, que no pueden proporcionar un valor inicial. Fíjate en `First` :

```
const First = x => ({ x, concat: other => First(x) })

Map({id: First(123), isPaid: Any(true), points: Sum(13)}).concat(Map({id: First(14), isPaid: Any(true), points: Sum(14)}))
// Map({id: First(123), isPaid: Any(true), points: Sum(14)})
```

Fusionaremos un par de cuentas y mantendremos el primer id. No hay manera de definir un valor vacío para ello. Esto no significa que no sea útil.

Gran Teoría Unificadora

¿Teoría De Grupos O Teoría De Categorías?

En el álgebra abstracta el concepto de operación binaria está en todas partes. Para una *categoría* es, de hecho, la operación primaria. Sin embargo, en la teoría de categorías no podemos modelar nuestra operación sin una *identidad*. Esta es la razón por la que comenzamos con un semigrupo de la teoría de grupos para luego, una vez tenemos el elemento *vacío*, saltar a un monoide de la teoría de categorías.

Los monoides forman una categoría de un solo objeto donde el morfismo es `concat`, la identidad es `empty` y donde la composición está garantizada.

Composición Como Monoide

Las funciones de tipo `a -> a`, donde el dominio es del mismo conjunto que el codominio, son llamadas *endomorfismos*. Podemos crear un *monoide* llamado *Endo* que capture esta idea:

```
const Endo = run => ({
  run,
  concat: other =>
    Endo(compose(run, other.run))
})

Endo.empty = () => Endo(identity)

// en acción

// thingDownFlipAndReverse :: Endo [String] -> [String]
const thingDownFlipAndReverse = fold(Endo(() => []), [Endo(reverse), Endo(sort)]

thingDownFlipAndReverse.run(['let me work it', 'is it worth it?'])
// ['thing down', 'let me work it', 'is it worth it?']
```

Dado que son todos del mismo tipo podemos concatenarlos a través de `compose`, que los tipos siempre se alinearán.

Mónada Como Monoide

Puede que hayas notado que `join` es una operación que toma dos mónadas (anidadas) y las aplasta en una sola de manera asociativa. Es también una transformación natural o una "función funtor". Como establecimos anteriormente, podemos crear una categoría con funtores como objetos y transformaciones naturales como morfismos. Si la especializamos en *Endofuntores*, o sea, funtores del mismo tipo, entonces `join` nos provee de un monoide en la categoría de Endofuntores también conocido como Mónada. Mostrar en código la formulación exacta requiere de algunos trucos que te animo a googlear, pero esta es la idea general.

Aplicativo Como Monoide

Incluso lo funtores aplicativos tienen una formulación monoidal conocida en teoría de categorías como *funtor monoidal laxo*. Podemos implementar la interfaz como un monoide y recuperar `ap` de él:

```
// concat :: f a -> f b -> f [a, b]
// empty :: () -> f ()

// ap :: Functor f => f (a -> b) -> f a -> f b
const ap = compose(map(([f, x]) => f(x)), concat)
```

En Resumen

Como puedes ver, todo está conectado, o puede estarlo. Este profundo hecho convierte a los *Monoïdes* en una poderosa herramienta de modelado para amplias franjas de la arquitectura de aplicaciones llegando hasta las piezas de datos más pequeñas. Te animo a pensar en *monoides* cuándo la acumulación o combinación directas formen parte de tu aplicación, y a que, una vez lo tengas claro, empieces a ampliar la definición a más aplicaciones (te sorprenderá lo mucho que se puede modelar con un *monoïde*).

Ejercicios

Apéndice A: Funciones Esenciales de Soporte

En este apéndice, encontrarás algunas implementaciones básicas en Javascript de varias funciones descritas en el libro. Ten en cuenta que estas implementaciones pueden no ser las más rápidas o las más eficientes; *tienen únicamente una finalidad educativa*.

Para encontrar funciones más preparadas para un entorno en producción, échale un vistazo a [ramda](#), [lodash](#), o [falktale](#).

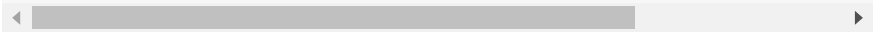
Ten en cuenta que algunas funciones también hacen referencia a estructuras algebraicas definidas en el [Apéndice B](#)

always

```
// always :: a -> b -> a
const always = curry((a, b) => a);
```

compose

```
// compose :: ((y -> z), (x -> y), ..., (a -> b)) -> a -> z
const compose = (...fns) => (...args) => fns.reduceRight((res, fn) => [fn.call(
```



curry

```
// curry :: ((a, b, ...) -> c) -> a -> b -> ... -> c
function curry(fn) {
  const arity = fn.length;

  return function $curry(...args) {
    if (args.length < arity) {
      return $curry.bind(null, ...args);
    }

    return fn.call(null, ...args);
  };
}
```

either

```
// either :: (a -> c) -> (b -> c) -> Either a b -> c
const either = curry((f, g, e) => {
  if (e.isLeft) {
    return f(e.$value);
  }

  return g(e.$value);
});
```

identity

```
// identity :: x -> x
const identity = x => x;
```

inspect

```
// inspect :: a -> String
const inspect = (x) => {
  if (x && typeof x.inspect === 'function') {
    return x.inspect();
  }

  function inspectFn(f) {
    return f.name ? f.name : f.toString();
  }

  function inspectTerm(t) {
    switch (typeof t) {
      case 'string':
        return `${t}`;
      case 'object': {
        const ts = Object.keys(t).map(k => [k, inspect(t[k])]);
        return `${ts.map(kv => kv.join(': ')).join(', ')}`;
      }
      default:
        return String(t);
    }
  }

  function inspectArgs(args) {
    return Array.isArray(args) ? `[${args.map(inspect).join(', ')}]` : inspect`
  }

  return (typeof x === 'function') ? inspectFn(x) : inspectArgs(x);
};
```

left

```
// left :: a -> Either a b
const left = a => new Left(a);
```

liftA2

```
// liftA2 :: (Applicative f) => (a1 -> a2 -> b) -> f a1 -> f a2 -> f b
const liftA2 = curry((fn, a1, a2) => a1.map(fn).ap(a2));
```

liftA3

```
// liftA3 :: (Applicative f) => (a1 -> a2 -> a3 -> b) -> f a1 -> f a2 -> f a3 -> f b
const liftA3 = curry((fn, a1, a2, a3) => a1.map(fn).ap(a2).ap(a3));
```

maybe

```
// maybe :: b -> (a -> b) -> Maybe a -> b
const maybe = curry((v, f, m) => {
  if (m.isNothing) {
    return v;
  }

  return f(m.$value);
});
```

nothing

```
// nothing :: Maybe a
const nothing = Maybe.of(null);
```

reject

```
// reject :: a -> Task a b
const reject = a => Task.rejected(a);
```

Apéndice B: Estructuras Algebraicas de Soporte

En este apéndice, encontrarás algunas implementaciones básicas en Javascript de varias estructuras algebraicas descritas en el libro. Ten en cuenta que estas implementaciones pueden no ser las más rápidas o las más eficientes; *tienen únicamente una finalidad educativa*.

Para encontrar estructuras más preparadas para un entorno en producción, échale un vistazo a [ramda](#), [lodash](#), o [folktale](#).

Ten en cuenta que algunos métodos hacen referencia a funciones definidas en el [Apéndice A](#)

Compose

```
const createCompose = curry((F, G) => class Compose {
  constructor(x) {
    this.$value = x;
  }

  [util.inspect.custom]() {
    return `Compose(${inspect(this.$value)})`;
  }

  // ----- Pointed (Compose F G)
  static of(x) {
    return new Compose(F(G(x)));
  }

  // ----- Functor (Compose F G)
  map(fn) {
    return new Compose(this.$value.map(x => x.map(fn)));
  }

  // ----- Applicative (Compose F G)
  ap(f) {
    return f.map(this.$value);
  }
});
```

Either

```
class Either {  
  constructor(x) {  
    this.$value = x;  
  }  
  
  // ----- Pointed (Either a)  
  static of(x) {  
    return new Right(x);  
  }  
}
```

Left

```
class Left extends Either {  
  get isLeft() {  
    return true;  
  }  
  
  get isRight() {  
    return false;  
  }  
  
  static of(x) {  
    throw new Error("`of` called on class Left (value) instead of Either (type)");  
  }  
  
  [util.inspect.custom]() {  
    return `Left(${inspect(this.$value)})`;  
  }  
  
  // ----- Functor (Either a)  
  map() {  
    return this;  
  }  
  
  // ----- Applicative (Either a)  
  ap() {  
    return this;  
  }  
  
  // ----- Monad (Either a)  
  chain() {  
    return this;  
  }  
  
  join() {  
    return this;  
  }  
  
  // ----- Traversable (Either a)  
  sequence(of) {  
    return of(this);  
  }  
  
  traverse(of, fn) {  
    return of(this);  
  }  
}
```

```
}  
}
```



Right

```
class Right extends Either {  
  get isLeft() {  
    return false;  
  }  
  
  get isRight() {  
    return true;  
  }  
  
  static of(x) {  
    throw new Error("`of` called on class Right (value) instead of Either (type)");  
  }  
  
  [util.inspect.custom]() {  
    return `Right(${inspect(this.$value)})`;  
  }  
  
  // ----- Functor (Either a)  
  map(fn) {  
    return Either.of(fn(this.$value));  
  }  
  
  // ----- Applicative (Either a)  
  ap(f) {  
    return f.map(this.$value);  
  }  
  
  // ----- Monad (Either a)  
  chain(fn) {  
    return fn(this.$value);  
  }  
  
  join() {  
    return this.$value;  
  }  
  
  // ----- Traversable (Either a)  
  sequence(of) {  
    return this.traverse(of, identity);  
  }  
  
  traverse(of, fn) {  
    return fn(this.$value).map(Either.of);  
  }  
}
```

```
}  
}
```



Identity

```
class Identity {
  constructor(x) {
    this.$value = x;
  }

  [util.inspect.custom]() {
    return `Identity(${inspect(this.$value)})`;
  }

  // ----- Pointed Identity
  static of(x) {
    return new Identity(x);
  }

  // ----- Functor Identity
  map(fn) {
    return Identity.of(fn(this.$value));
  }

  // ----- Applicative Identity
  ap(f) {
    return f.map(this.$value);
  }

  // ----- Monad Identity
  chain(fn) {
    return this.map(fn).join();
  }

  join() {
    return this.$value;
  }

  // ----- Traversable Identity
  sequence(of) {
    return this.traverse(of, identity);
  }

  traverse(of, fn) {
    return fn(this.$value).map(Identity.of);
  }
}
```

IO

```
class IO {
  constructor(fn) {
    this.unsafePerformIO = fn;
  }

  [util.inspect.custom]() {
    return 'IO(?)';
  }

  // ----- Pointed IO
  static of(x) {
    return new IO(() => x);
  }

  // ----- Functor IO
  map(fn) {
    return new IO(compose(fn, this.unsafePerformIO));
  }

  // ----- Applicative IO
  ap(f) {
    return this.chain(fn => f.map(fn));
  }

  // ----- Monad IO
  chain(fn) {
    return this.map(fn).join();
  }

  join() {
    return new IO(() => this.unsafePerformIO().unsafePerformIO());
  }
}
```

List

```
class List {  
  constructor(xs) {  
    this.$value = xs;  
  }  
  
  [util.inspect.custom]() {  
    return `List(${inspect(this.$value)})`;  
  }  
  
  concat(x) {  
    return new List(this.$value.concat(x));  
  }  
  
  // ----- Pointed List  
  static of(x) {  
    return new List([x]);  
  }  
  
  // ----- Functor List  
  map(fn) {  
    return new List(this.$value.map(fn));  
  }  
  
  // ----- Traversable List  
  sequence(of) {  
    return this.traverse(of, identity);  
  }  
  
  traverse(of, fn) {  
    return this.$value.reduce(  
      (f, a) => fn(a).map(b => bs => bs.concat(b)).ap(f),  
      of(new List([])),  
    );  
  }  
}
```

Map

```
class Map {
  constructor(x) {
    this.$value = x;
  }

  [util.inspect.custom]() {
    return `Map(${inspect(this.$value)})`;
  }

  insert(k, v) {
    const singleton = {};
    singleton[k] = v;
    return Map.of(Object.assign({}, this.$value, singleton));
  }

  reduceWithKeys(fn, zero) {
    return Object.keys(this.$value)
      .reduce((acc, k) => fn(acc, this.$value[k], k), zero);
  }

  // ----- Functor (Map a)
  map(fn) {
    return this.reduceWithKeys(
      (m, v, k) => m.insert(k, fn(v)),
      new Map({}),
    );
  }

  // ----- Traversable (Map a)
  sequence(of) {
    return this.traverse(of, identity);
  }

  traverse(of, fn) {
    return this.reduceWithKeys(
      (f, a, k) => fn(a).map(b => m => m.insert(k, b)).ap(f),
      of(new Map({})),
    );
  }
}
```

Maybe

Ten en cuenta que `Maybe` también podría definirse con dos clases hijas `Just` y `Nothing`, de una manera similar a como lo hicimos con `Either`. Esta tan solo es una forma diferente.

```
class Maybe {  
  get isNothing() {  
    return this.$value === null || this.$value === undefined;  
  }  
  
  get isJust() {  
    return !this.isNothing;  
  }  
  
  constructor(x) {  
    this.$value = x;  
  }  
  
  [util.inspect.custom]() {  
    return this.isNothing ? 'Nothing' : `Just(${inspect(this.$value)})`;  
  }  
  
  // ----- Pointed Maybe  
  static of(x) {  
    return new Maybe(x);  
  }  
  
  // ----- Functor Maybe  
  map(fn) {  
    return this.isNothing ? this : Maybe.of(fn(this.$value));  
  }  
  
  // ----- Applicative Maybe  
  ap(f) {  
    return this.isNothing ? this : f.map(this.$value);  
  }  
  
  // ----- Monad Maybe  
  chain(fn) {  
    return this.map(fn).join();  
  }  
  
  join() {  
    return this.isNothing ? this : this.$value;  
  }  
  
  // ----- Traversable Maybe  
  sequence(of) {  
    return this.traverse(of, identity);  
  }  
  
  traverse(of, fn) {
```

```
    return this.isNothing ? of(this) : fn(this.$value).map(Maybe.of);  
  }  
}
```

Task

```
class Task {  
  constructor(fork) {  
    this.fork = fork;  
  }  
  
  [util.inspect.custom]() {  
    return 'Task(?)';  
  }  
  
  static rejected(x) {  
    return new Task((reject, _) => reject(x));  
  }  
  
  // ----- Pointed (Task a)  
  static of(x) {  
    return new Task((_, resolve) => resolve(x));  
  }  
  
  // ----- Functor (Task a)  
  map(fn) {  
    return new Task((reject, resolve) => this.fork(reject, compose(resolve, fn),  
  }  
  
  // ----- Applicative (Task a)  
  ap(f) {  
    return this.chain(fn => f.map(fn));  
  }  
  
  // ----- Monad (Task a)  
  chain(fn) {  
    return new Task((reject, resolve) => this.fork(reject, x => fn(x).fork(reject,  
  }  
  
  join() {  
    return this.chain(identity);  
  }  
}
```

Apéndice C: Utilidades Pointfree

En este apéndice, encontrarás versiones pointfree de funciones clásicas de JavaScript descritas en el libro. Todas las siguientes funciones están disponibles en los ejercicios como parte de su contexto global. Ten en cuenta que estas implementaciones pueden no ser las más rápidas o las más eficientes; *tienen únicamente una finalidad educativa*.

Para encontrar funciones más preparadas para un entorno en producción, échale un vistazo a [ramda](#), [lodash](#), o [folktale](#).

Ten en cuenta que las funciones hacen referencia a las funciones `curry` y `compose` definidas en el [Apéndice A](#)

add

```
// add :: Number -> Number -> Number
const add = curry((a, b) => a + b);
```

append

```
// append :: String -> String -> String
const append = flip(concat);
```

chain

```
// chain :: Monad m => (a -> m b) -> m a -> m b
const chain = curry((fn, m) => m.chain(fn));
```

concat

```
// concat :: String -> String -> String
const concat = curry((a, b) => a.concat(b));
```

eq

```
// eq :: Eq a => a -> a -> Boolean
const eq = curry((a, b) => a === b);
```

filter

```
// filter :: (a -> Boolean) -> [a] -> [a]
const filter = curry((fn, xs) => xs.filter(fn));
```

flip

```
// flip :: (a -> b -> c) -> b -> a -> c
const flip = curry((fn, a, b) => fn(b, a));
```

forEach

```
// forEach :: (a -> ()) -> [a] -> ()
const forEach = curry((fn, xs) => xs.forEach(fn));
```

head

```
// head :: [a] -> a
const head = xs => xs[0];
```

intercalate

```
// intercalate :: String -> [String] -> String
const intercalate = curry((str, xs) => xs.join(str));
```

join

```
// join :: Monad m => m (m a) -> m a
const join = m => m.join();
```

last

```
// last :: [a] -> a
const last = xs => xs[xs.length - 1];
```

map

```
// map :: Functor f => (a -> b) -> f a -> f b
const map = curry((fn, f) => f.map(fn));
```

match

```
// match :: RegExp -> String -> Boolean
const match = curry((re, str) => re.test(str));
```

prop

```
// prop :: String -> Object -> a
const prop = curry((p, obj) => obj[p]);
```

reduce

```
// reduce :: (b -> a -> b) -> b -> [a] -> b
const reduce = curry((fn, zero, xs) => xs.reduce(fn, zero));
```

replace

```
// replace :: RegExp -> String -> String -> String
const replace = curry((re, rpl, str) => str.replace(re, rpl));
```

reverse

```
// reverse :: [a] -> [a]
const reverse = x => (Array.isArray(x) ? x.reverse() : x.split('').reverse().join(''));
```

safeHead

```
// safeHead :: [a] -> Maybe a
const safeHead = compose(Maybe.of, head);
```

safeLast

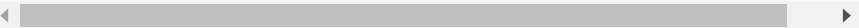
```
// safeLast :: [a] -> Maybe a
const safeLast = compose(Maybe.of, last);
```

safeProp

```
// safeProp :: String -> Object -> Maybe a
const safeProp = curry((p, obj) => compose(Maybe.of, prop(p))(obj));
```

sequence

```
// sequence :: (Applicative f, Traversable t) => (a -> f a) -> t (f a) -> f (t
const sequence = curry((of, f) => f.sequence(of));
```



sortBy

```
// sortBy :: Ord b => (a -> b) -> [a] -> [a]
const sortBy = curry((fn, xs) => xs.sort((a, b) => {
  if (fn(a) === fn(b)) {
    return 0;
  }

  return fn(a) > fn(b) ? 1 : -1;
})));
```

split

```
// split :: String -> String -> [String]
const split = curry((sep, str) => str.split(sep));
```

take

```
// take :: Number -> [a] -> [a]
const take = curry((n, xs) => xs.slice(0, n));
```

toLowerCase

```
// toLowerCase :: String -> String
const toLowerCase = s => s.toLowerCase();
```

toString

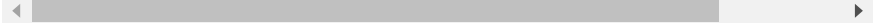
```
// toString :: a -> String
const toString = String;
```

toUpperCase

```
// toUpperCase :: String -> String
const toUpperCase = s => s.toUpperCase();
```

traverse

```
// traverse :: (Applicative f, Traversable t) => (a -> f a) -> (a -> f b) -> t
const traverse = curry((of, fn, f) => f.traverse(of, fn));
```



unsafePerformIO

```
// unsafePerformIO :: IO a -> a
const unsafePerformIO = io => io.unsafePerformIO();
```